

Introduction to Algorithm

기초 알고리즘

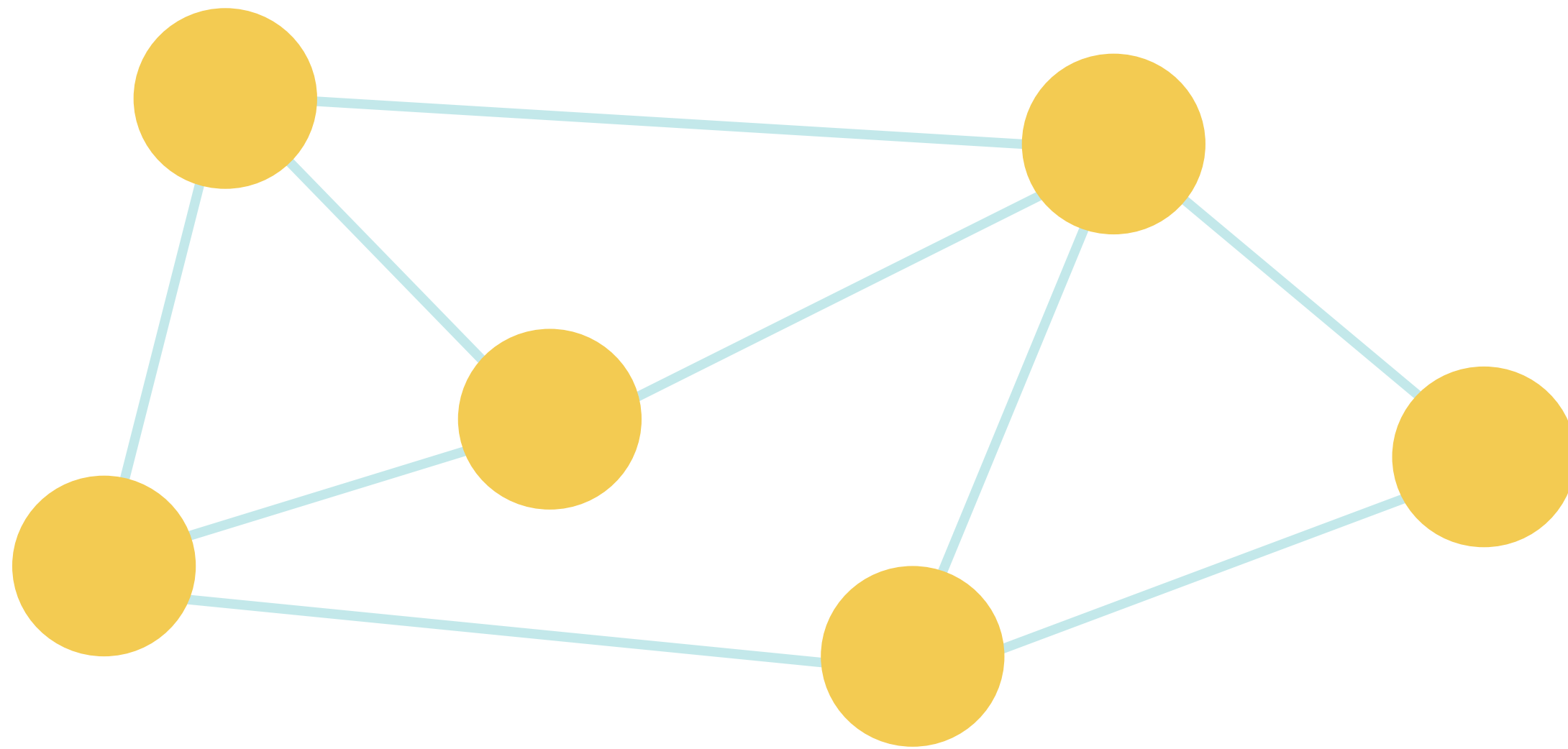
그래프 이론 기초

그래프 개념



그래프란?

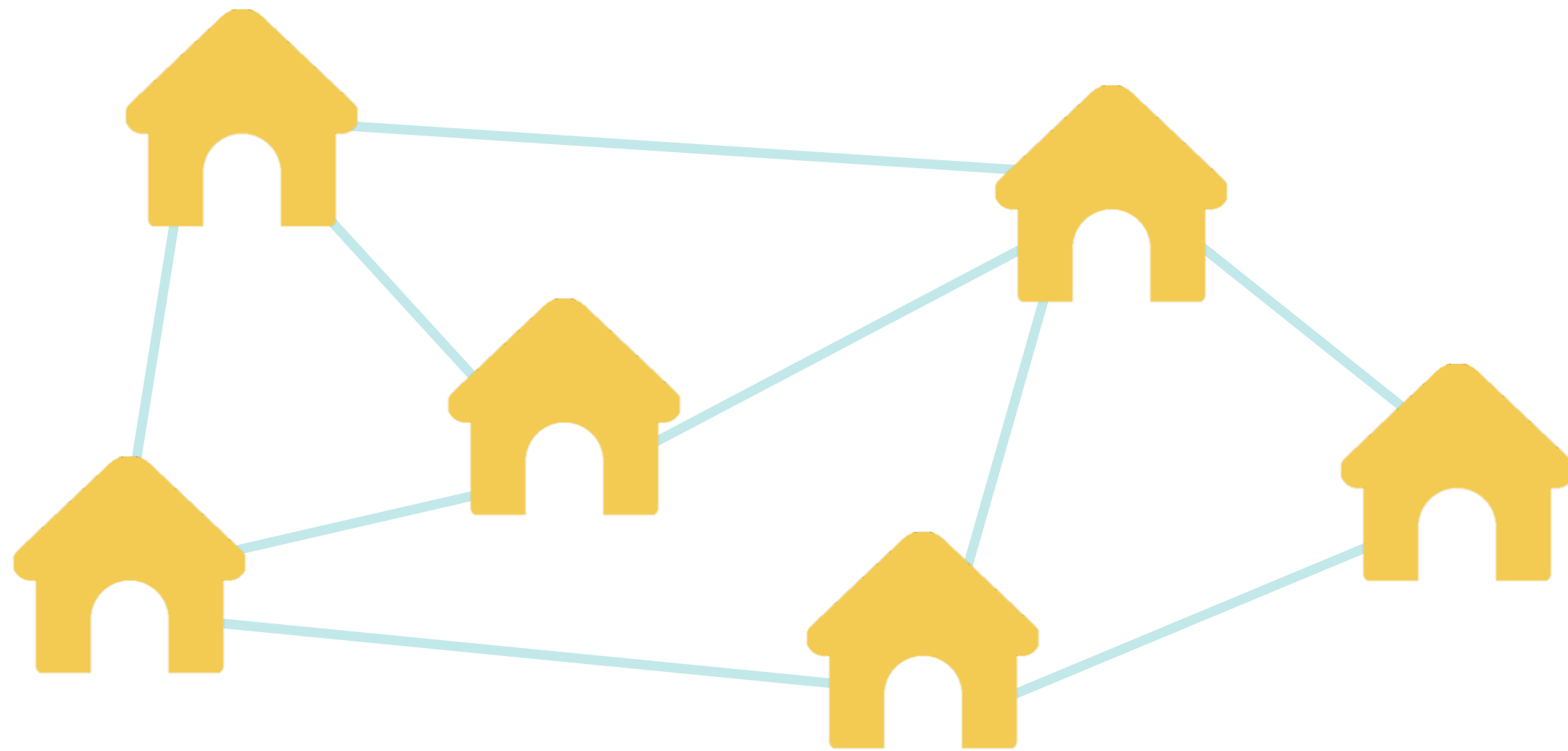
정점과 간선으로 이루어진 자료구조로
정점간의 관계를 조직도로 표현합니다





정점이란?

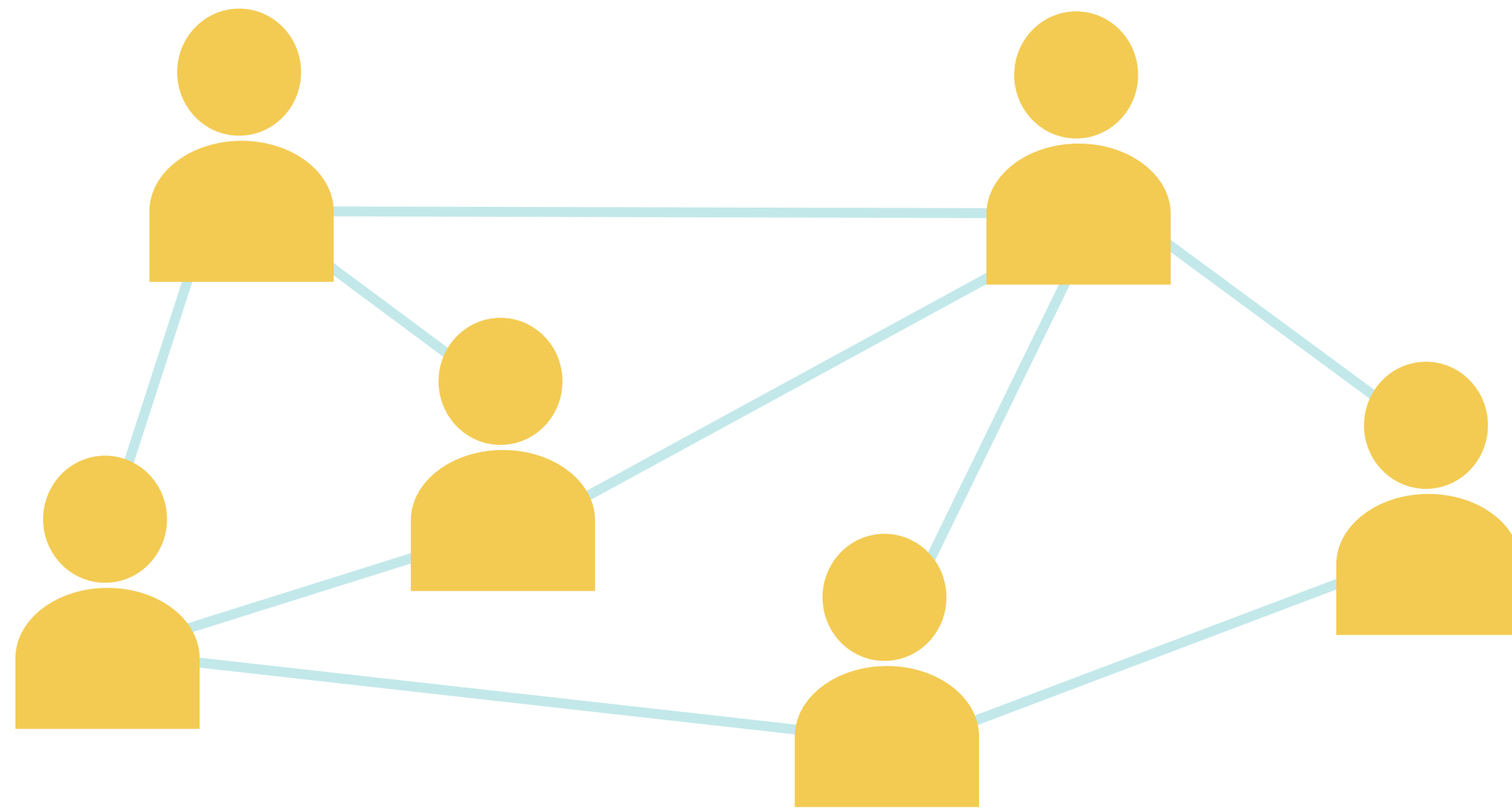
정점은 여러가지 특성을 가질 수 있는 객체를 의미합니다





정점이란?

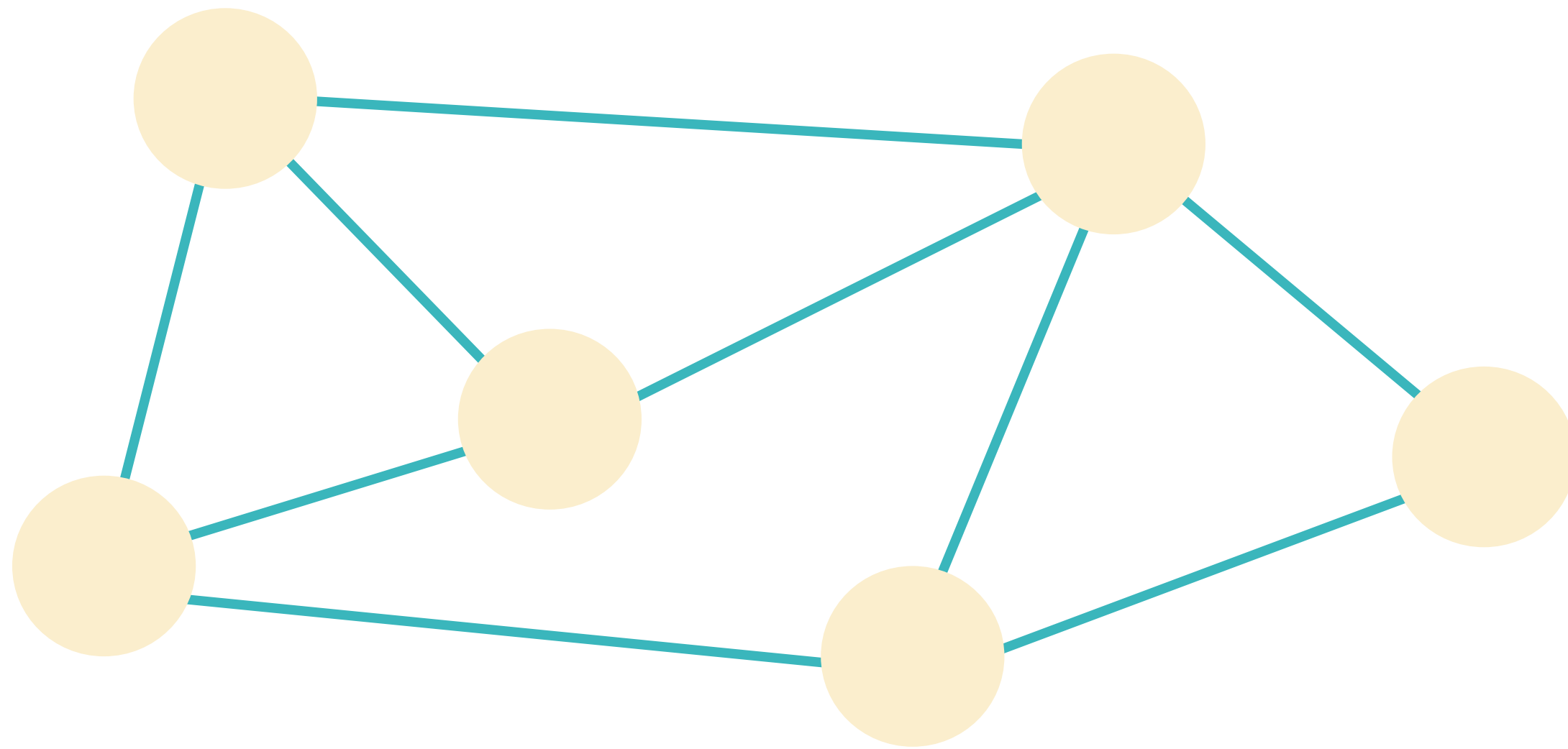
정점은 여러가지 특성을 가질 수 있는 객체를 의미합니다





간선이란?

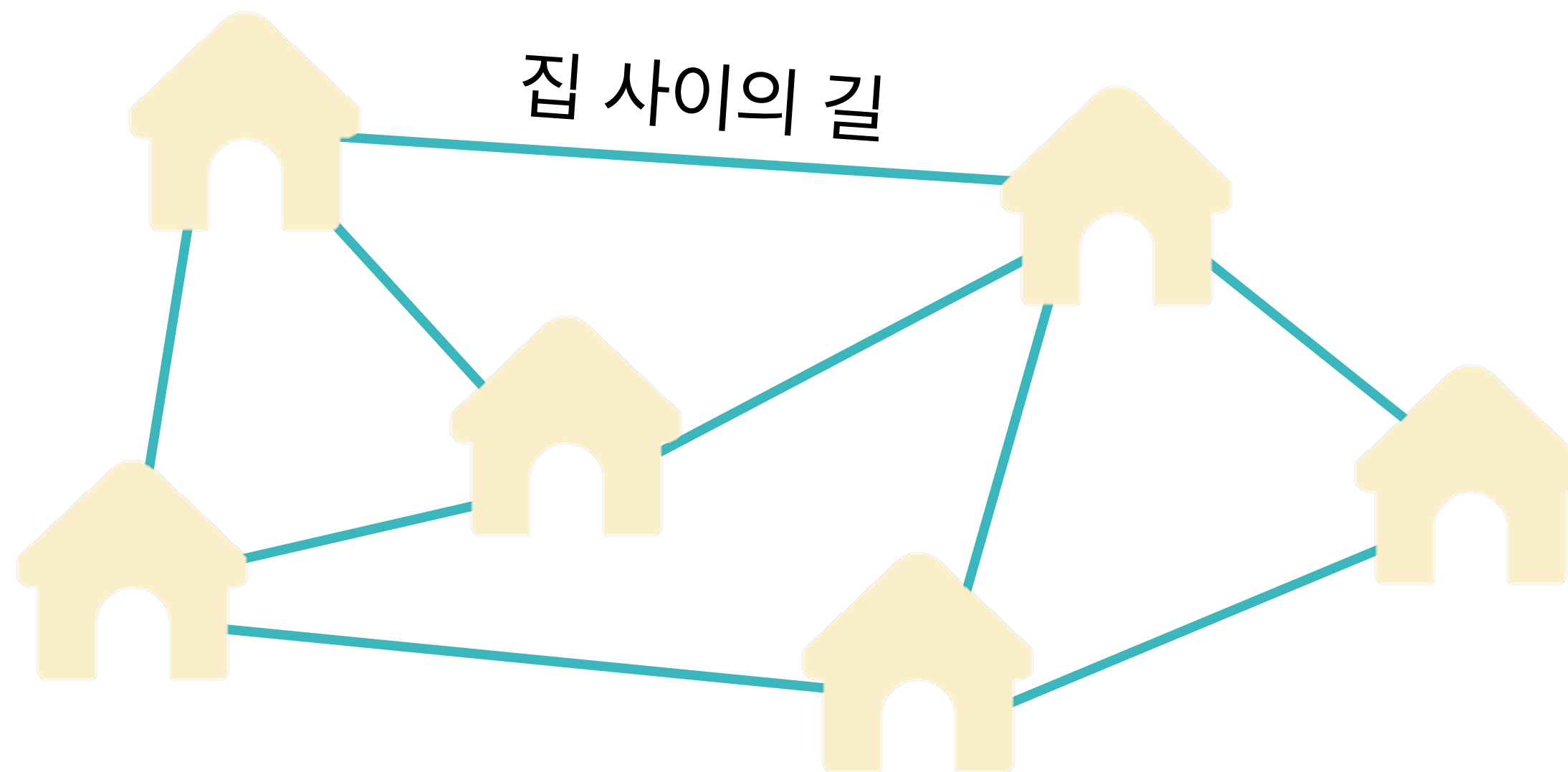
간선은 정점들 간의 관계를 의미합니다.





간선이란?

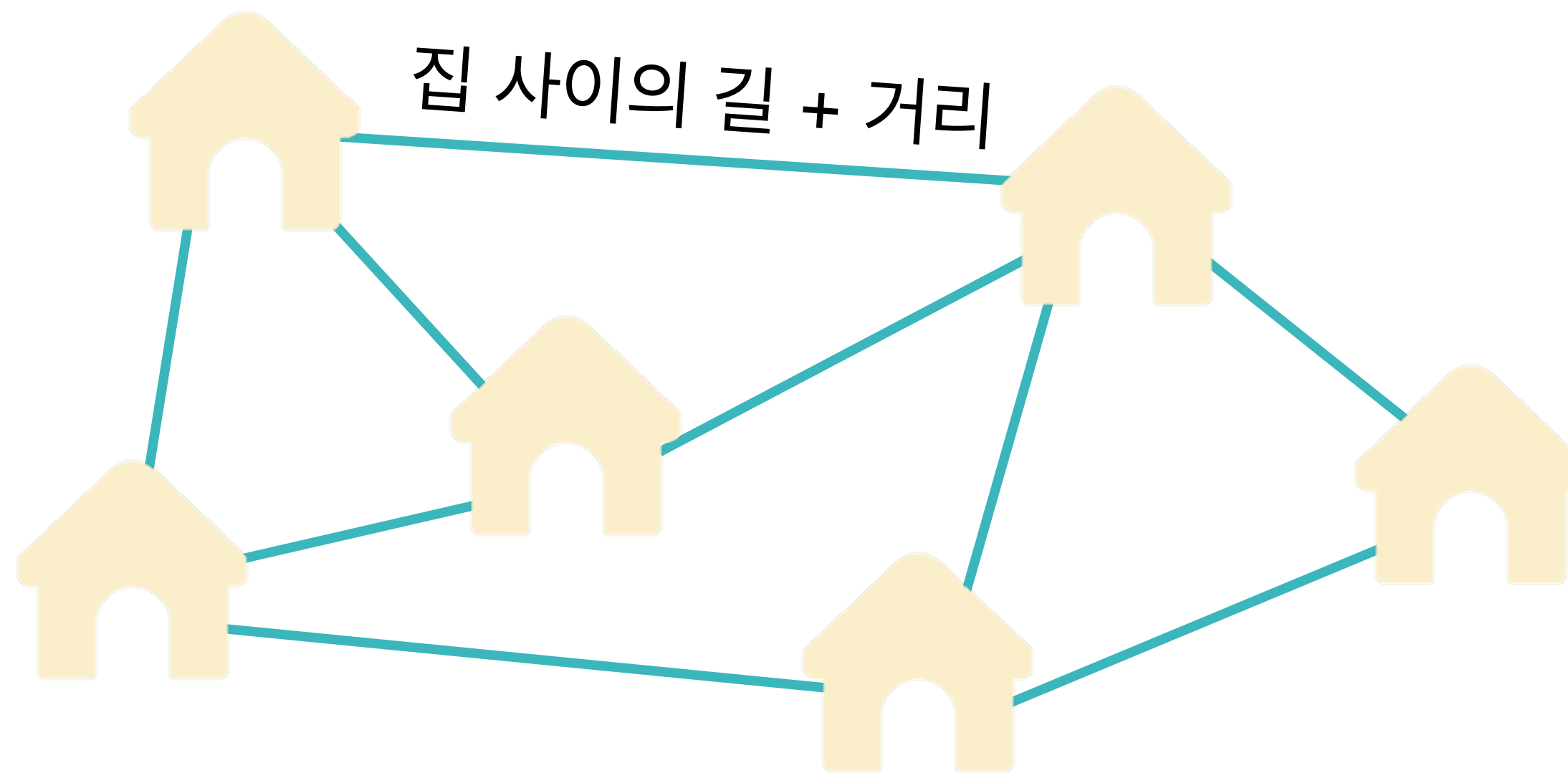
간선은 정점들 간의 관계를 의미합니다.





간선이란?

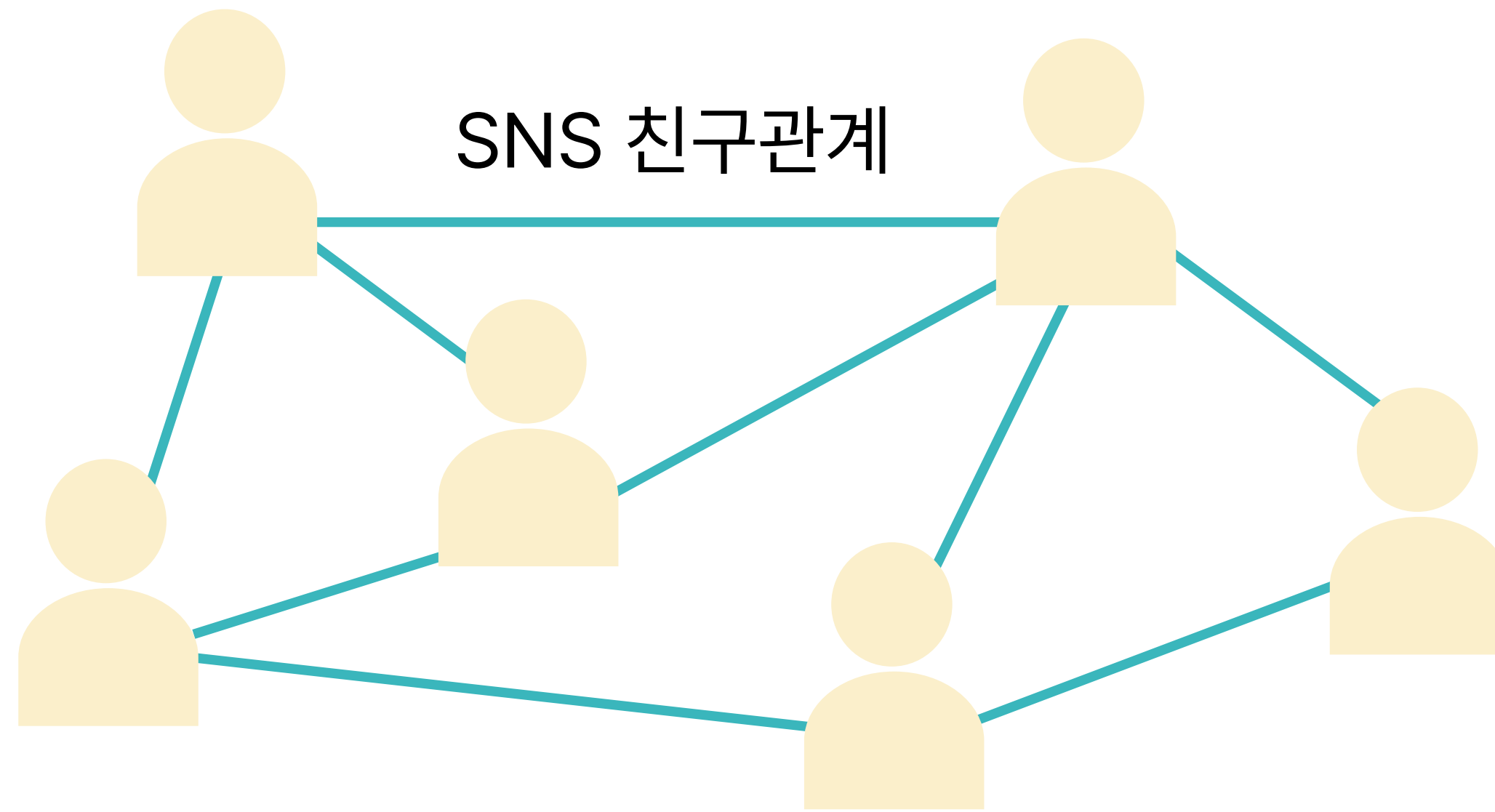
간선은 정점들 간의 관계를 의미합니다.





간선이란?

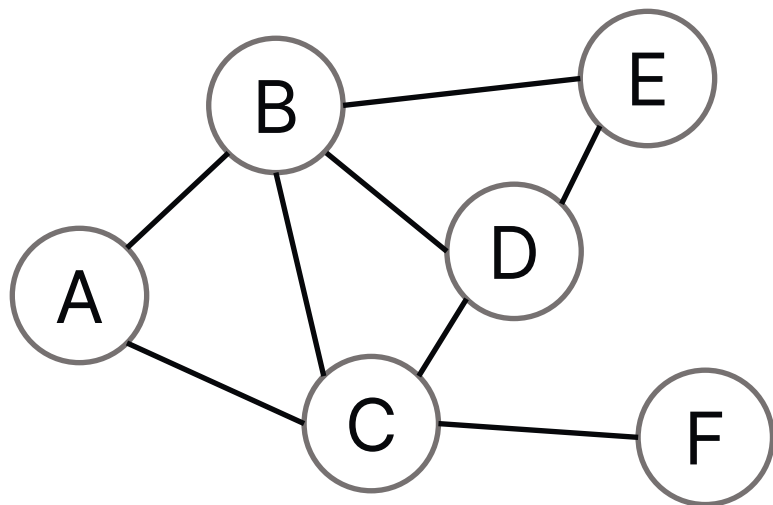
간선은 정점들 간의 관계를 의미합니다.



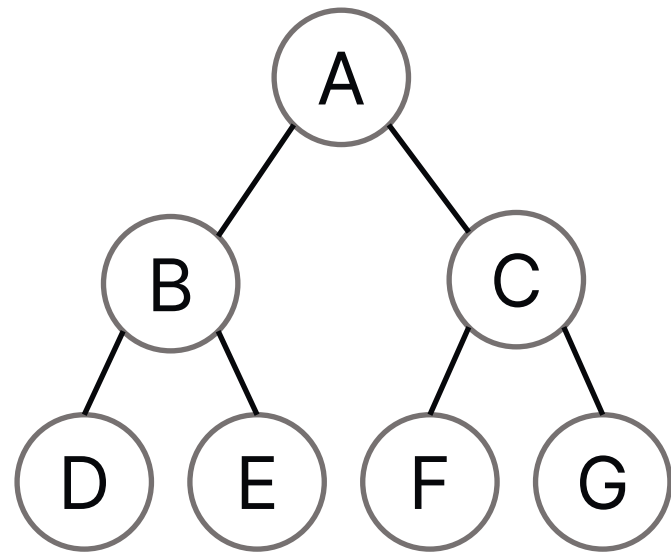


그래프의 종류

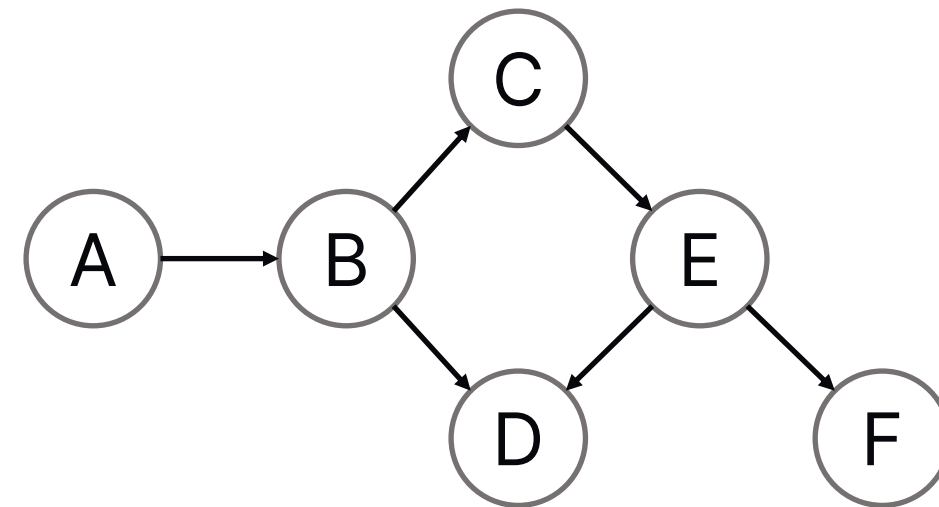
무방향 그래프



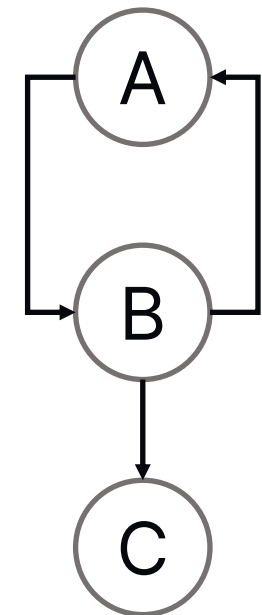
정점간 간선에 **방향**이
없는 그래프



방향 그래프



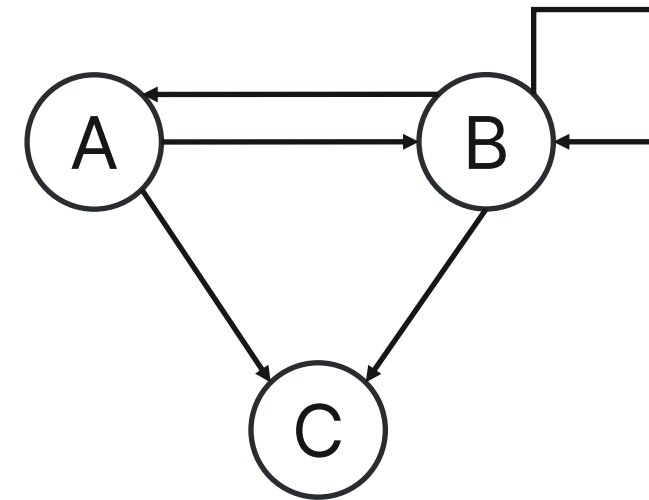
정점간 간선에 **방향**이
있는 그래프



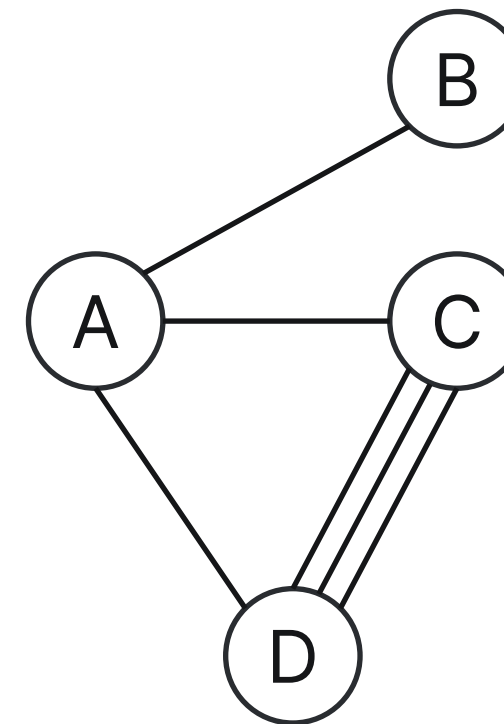


그래프의 특징

1) 자기 자신을 향하는 간선은 없다.



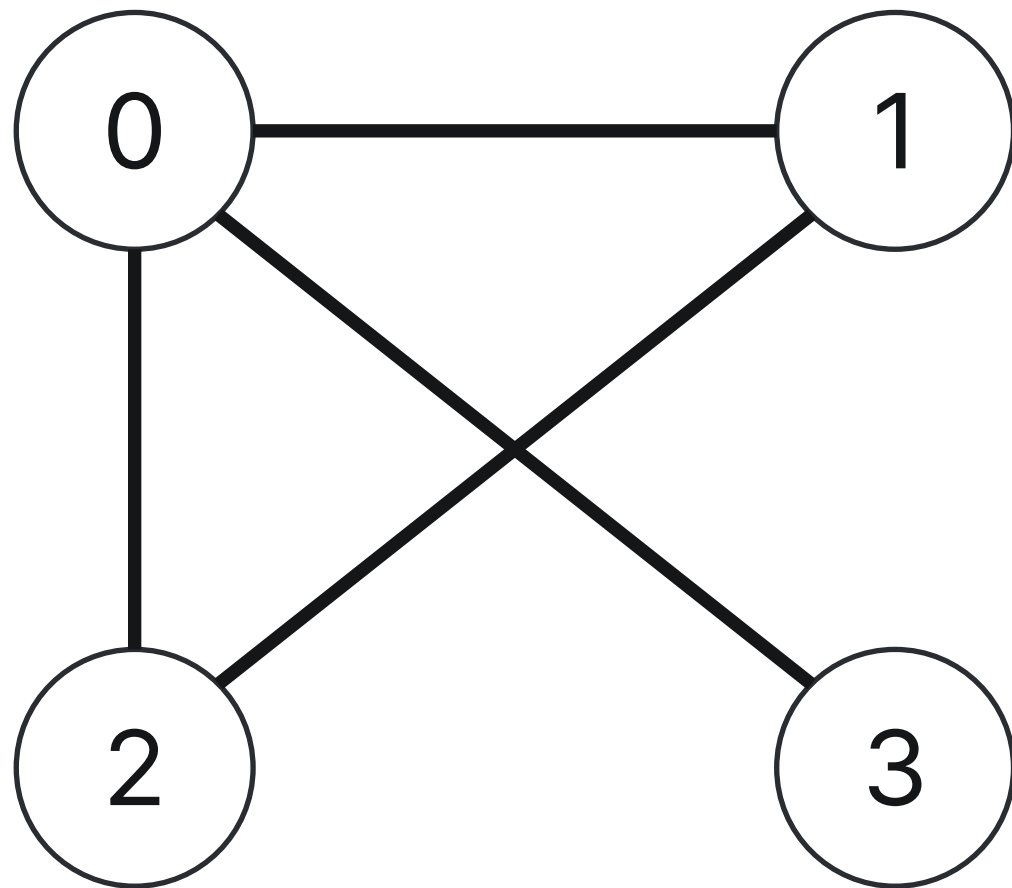
2) 중복된 간선을 허용하지 않는다.



그래프를 표현하는 방식



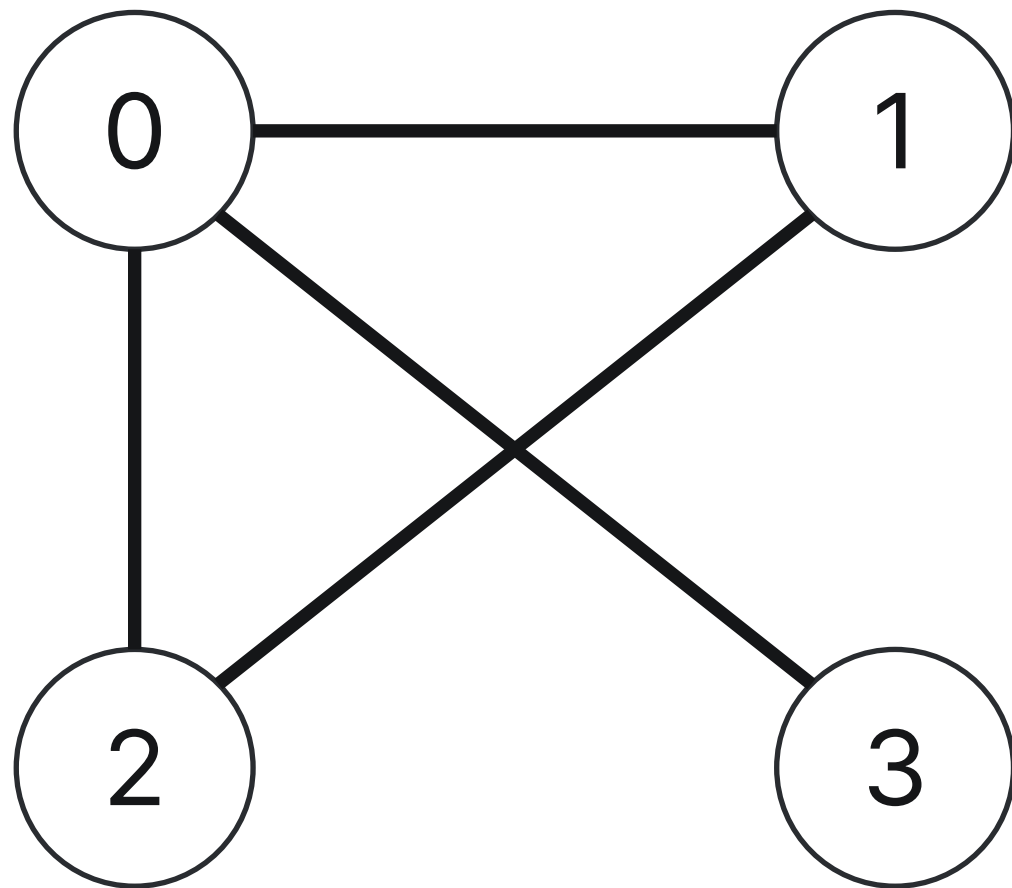
인접행렬 - 무방향 그래프



	0	1	2	3
0				
1				
2				
3				



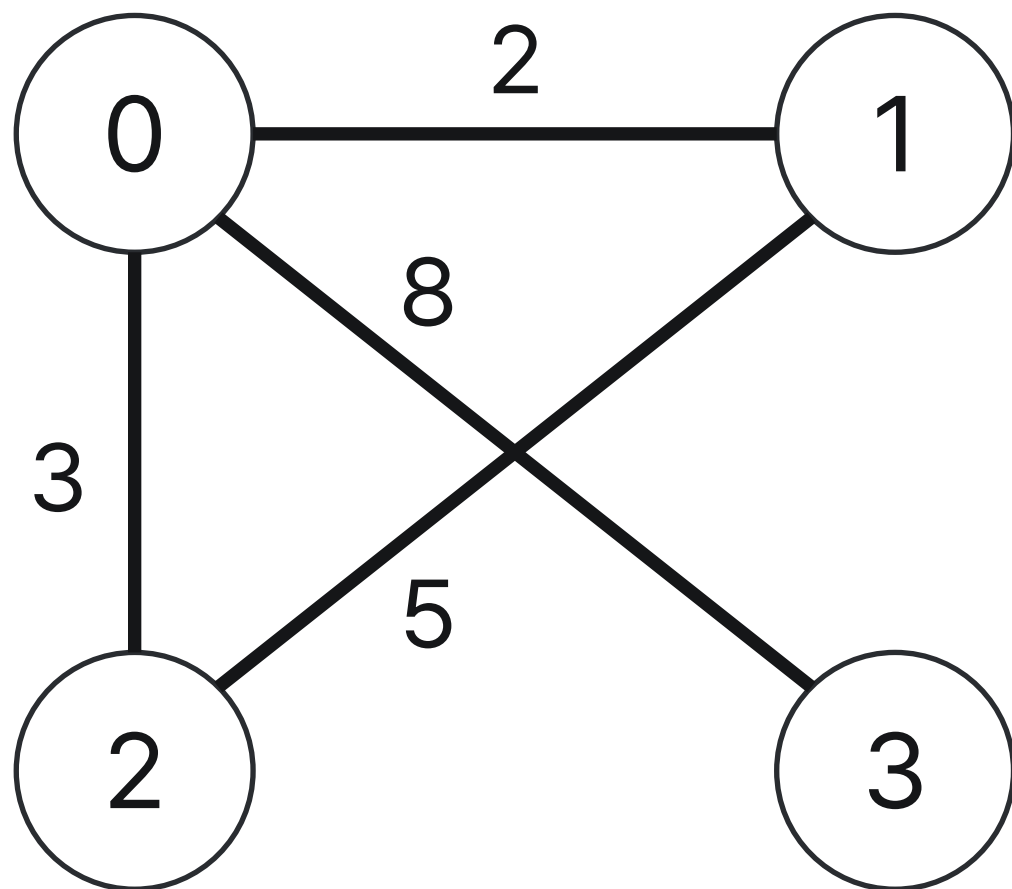
인접행렬 - 무방향 그래프



	0	1	2	3
0	0	1	1	1
1	1	0	1	0
2	1	1	0	0
3	1	0	0	0



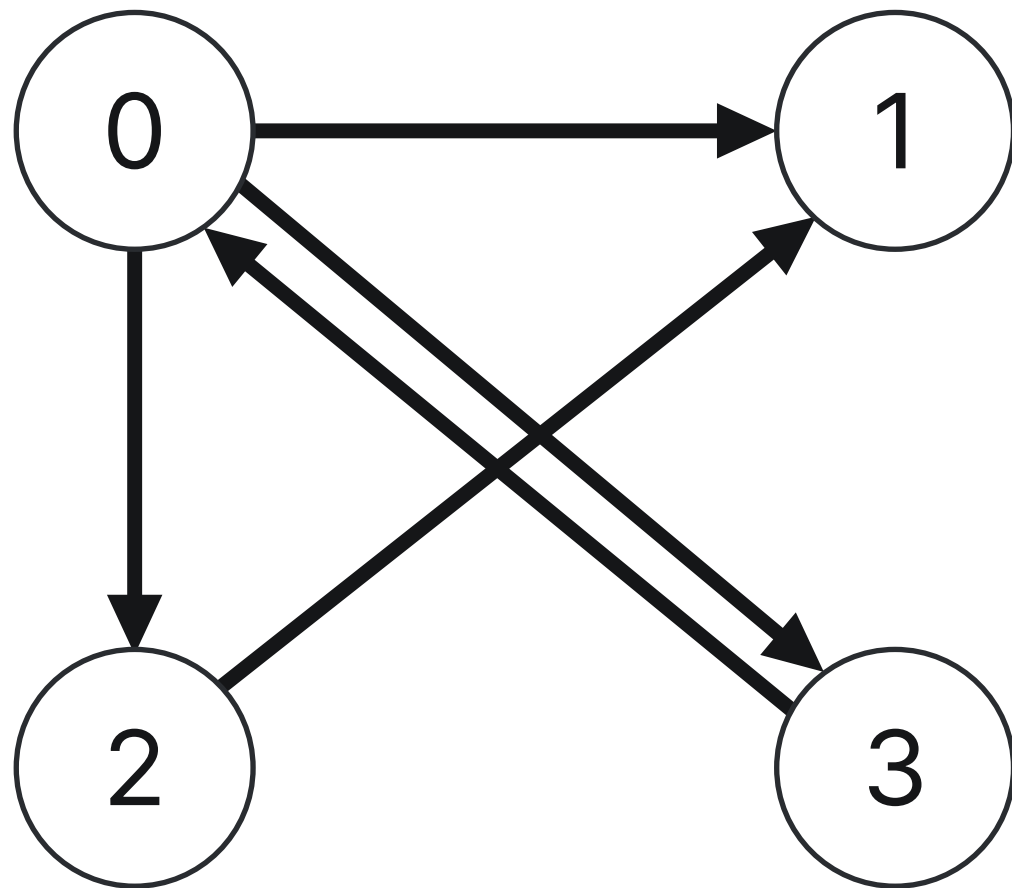
인접행렬 - 무방향 그래프 with 가중치



	0	1	2	3
0	0	2	3	8
1	2	0	5	0
2	3	5	0	0
3	8	0	0	0



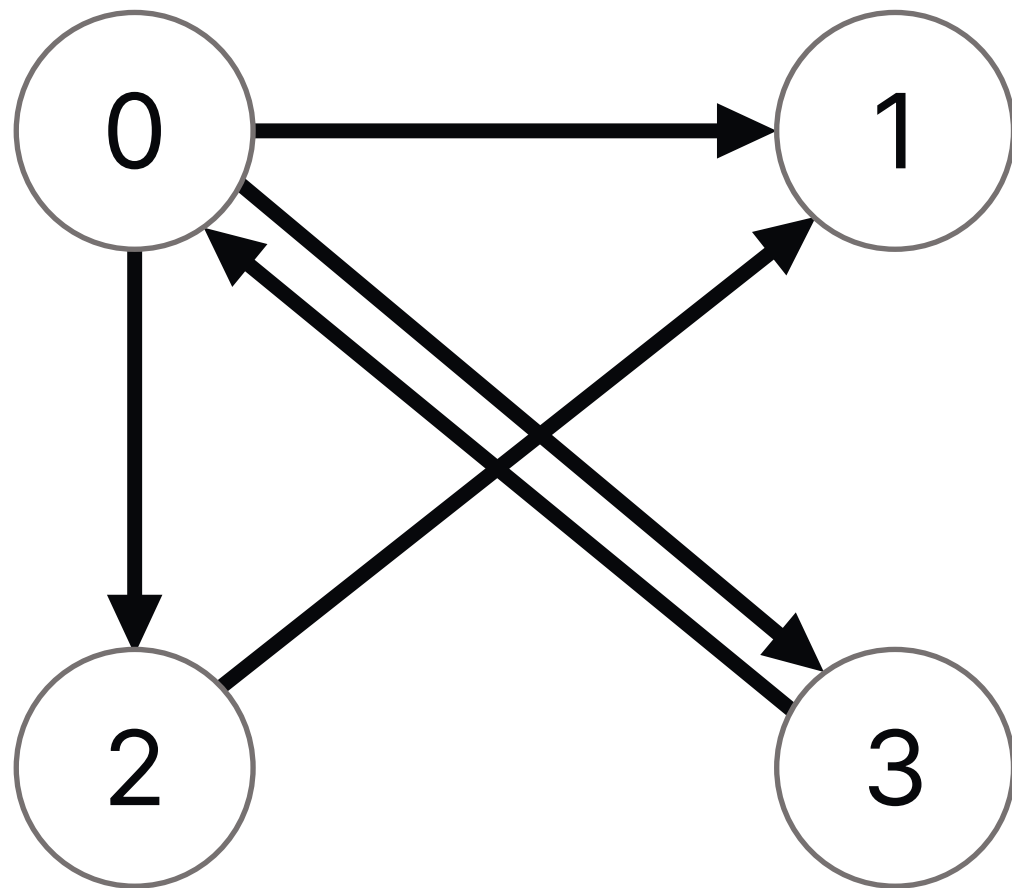
인접행렬 - 방향 그래프



	0	1	2	3
0				
1				
2				
3				



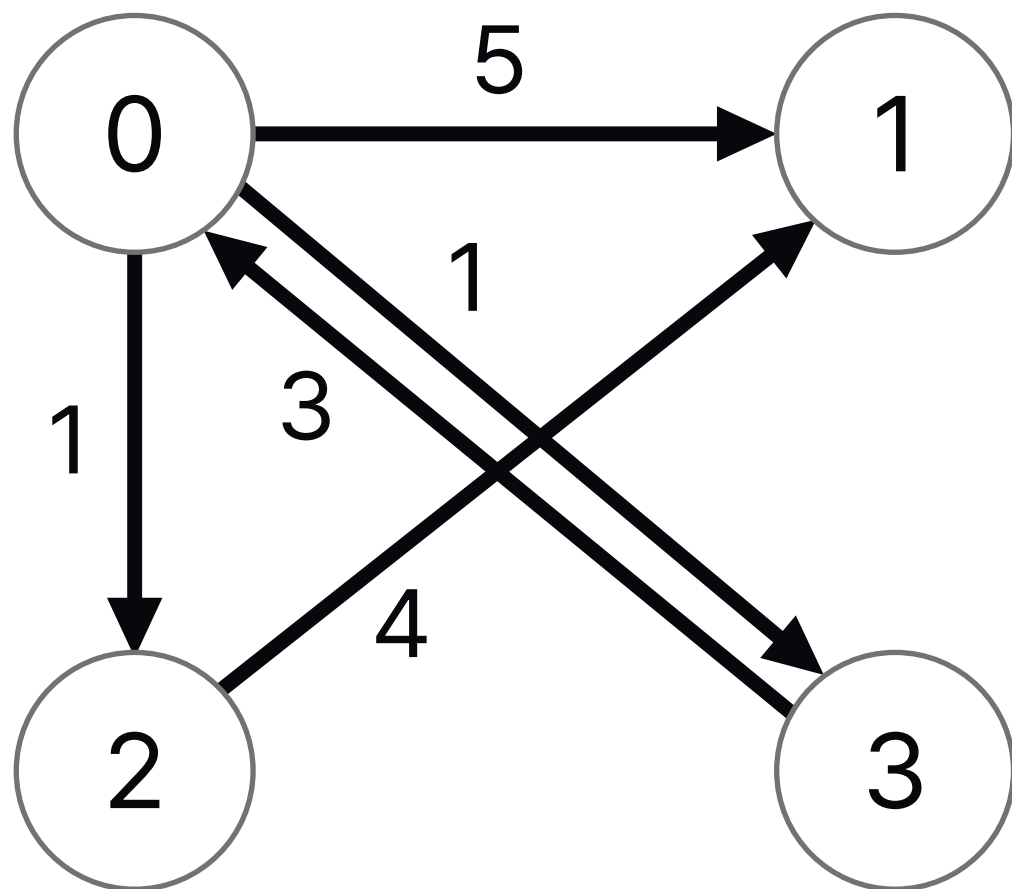
인접행렬 - 방향 그래프



	0	1	2	3
0	0	1	1	1
1	0	0	0	0
2	0	1	0	0
3	1	0	0	0



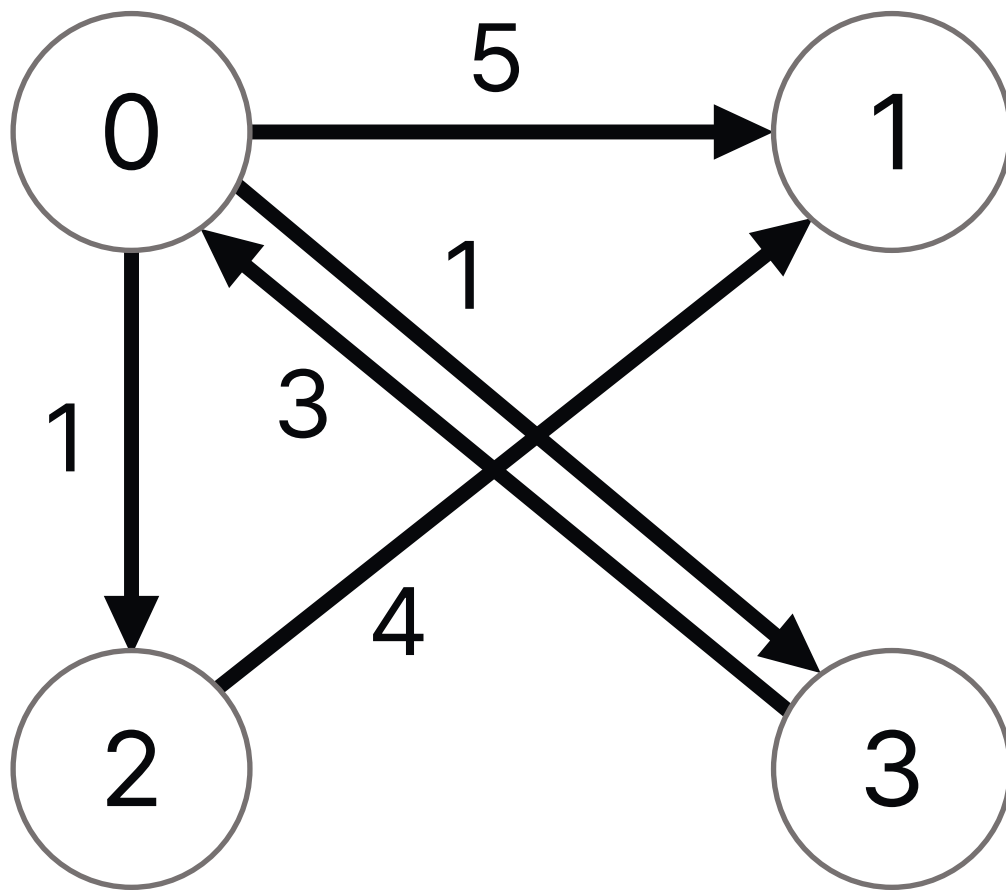
인접행렬 - 방향 그래프 with 가중치



	0	1	2	3
0	0	5	1	1
1	0	0	0	0
2	0	4	0	0
3	3	0	0	0



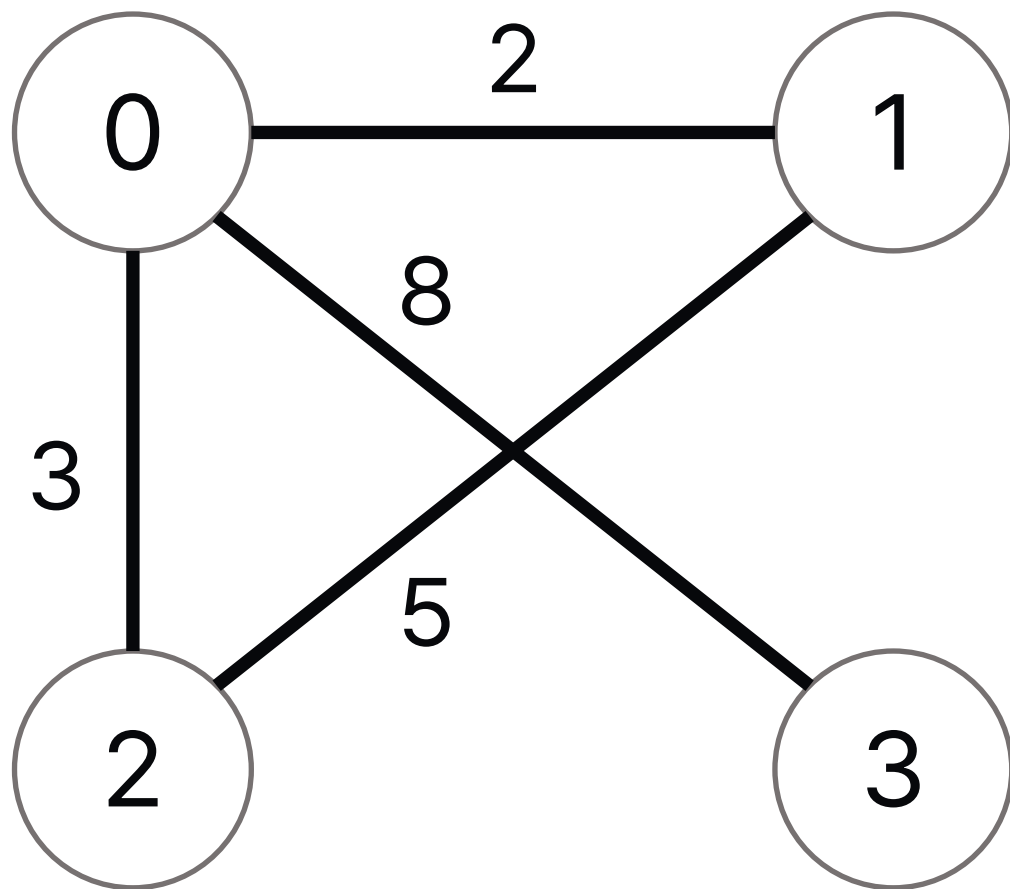
인접행렬



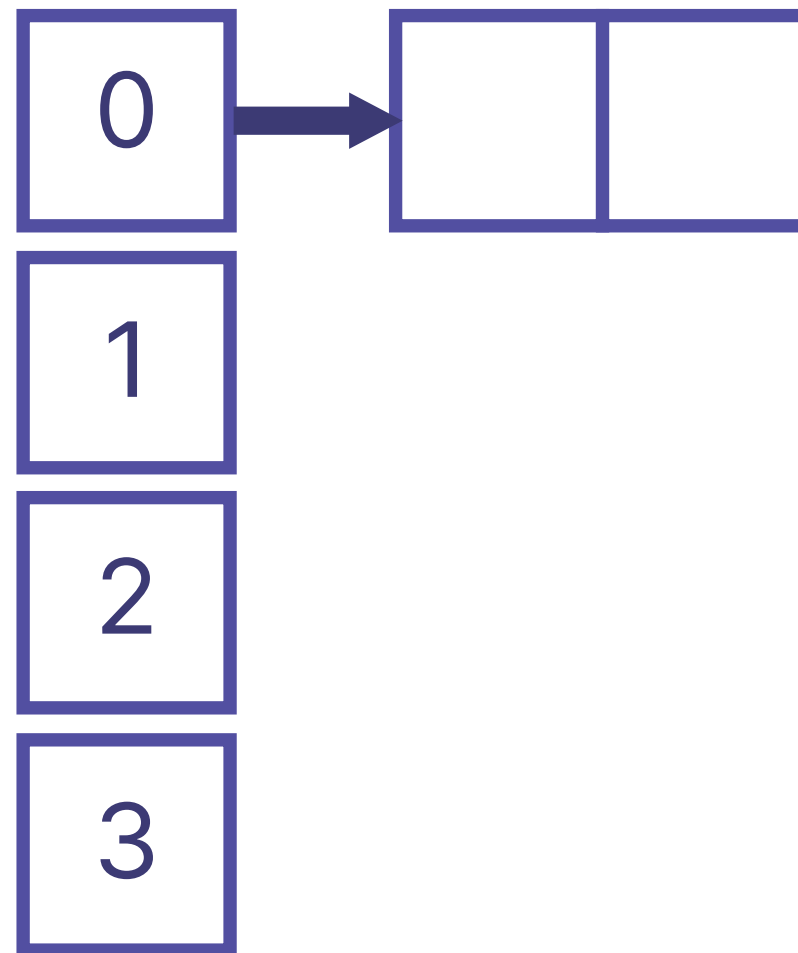
```
graph = [[0,5,1,1],[0,0,0,0],[0,4,0,0],[3,0,0,0]]
```



인접리스트

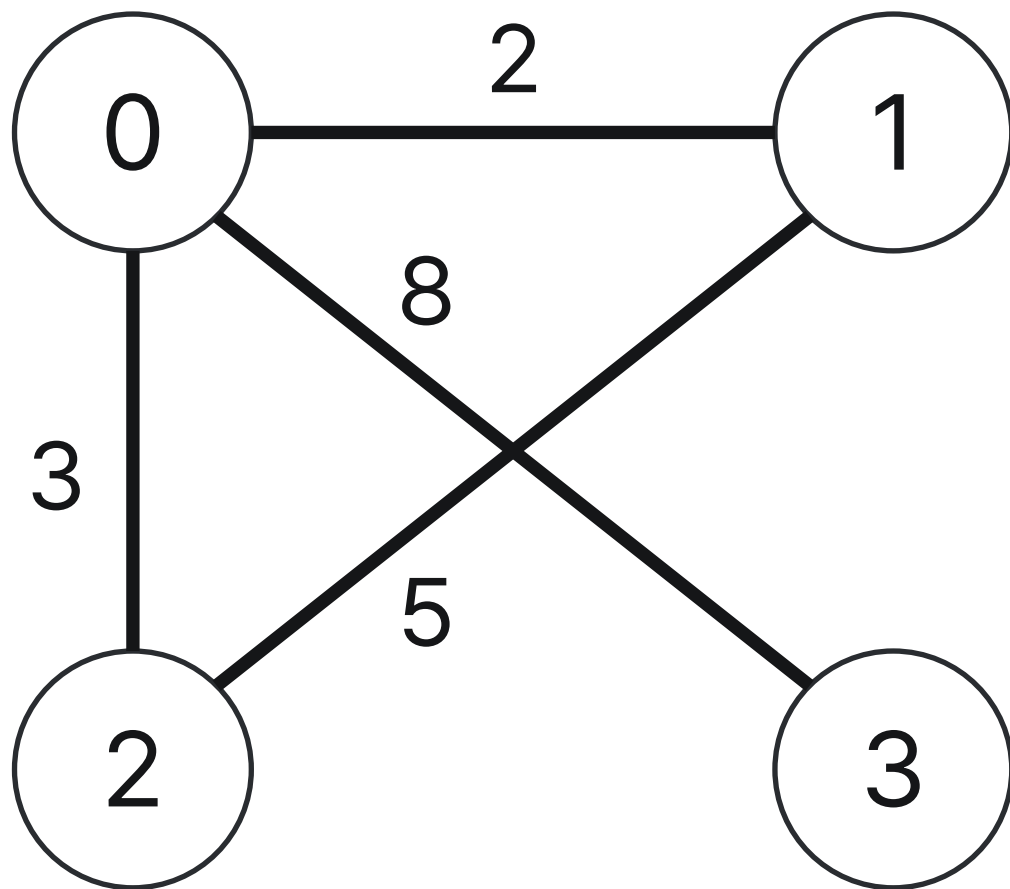


header vertex value

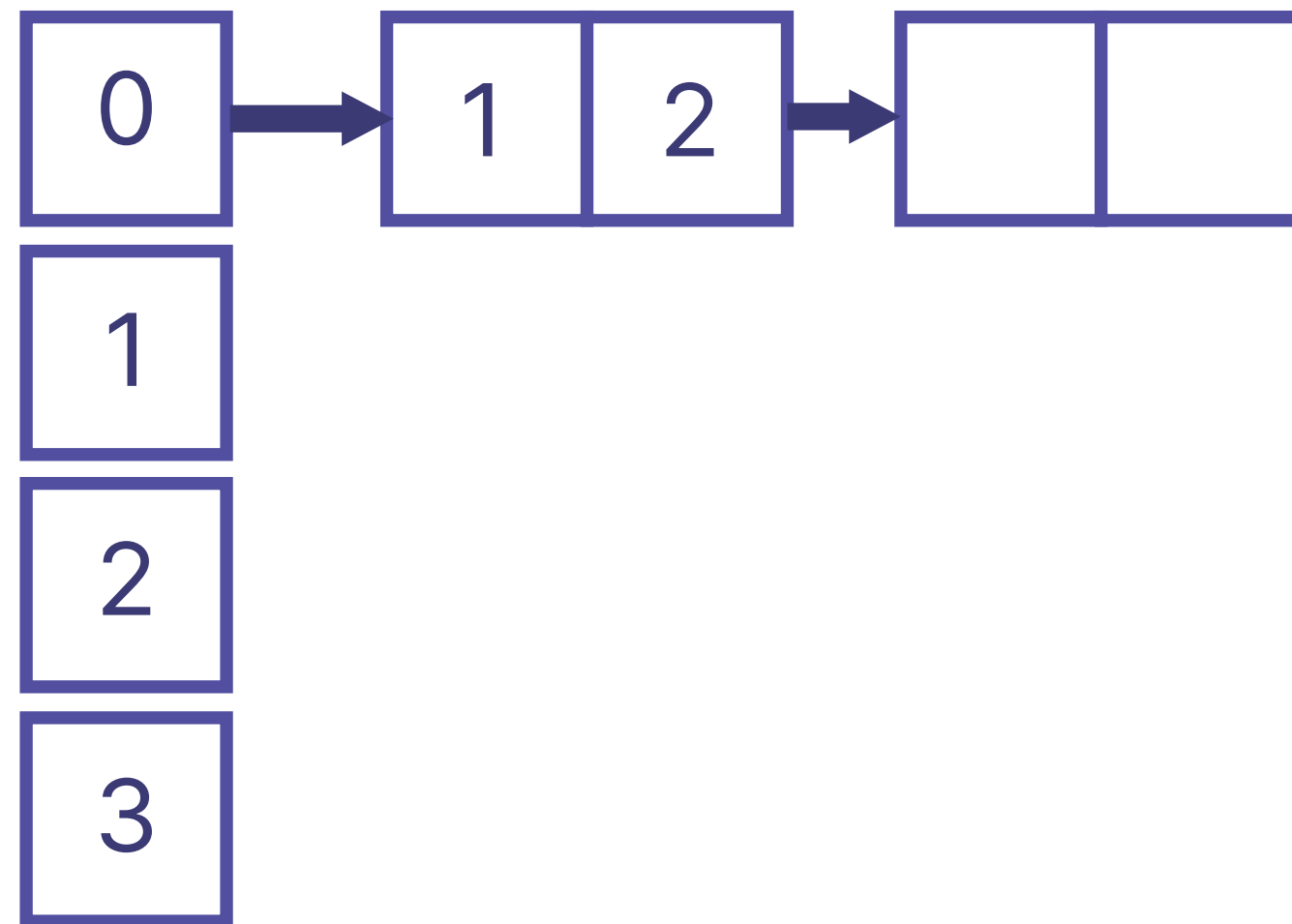




인접리스트

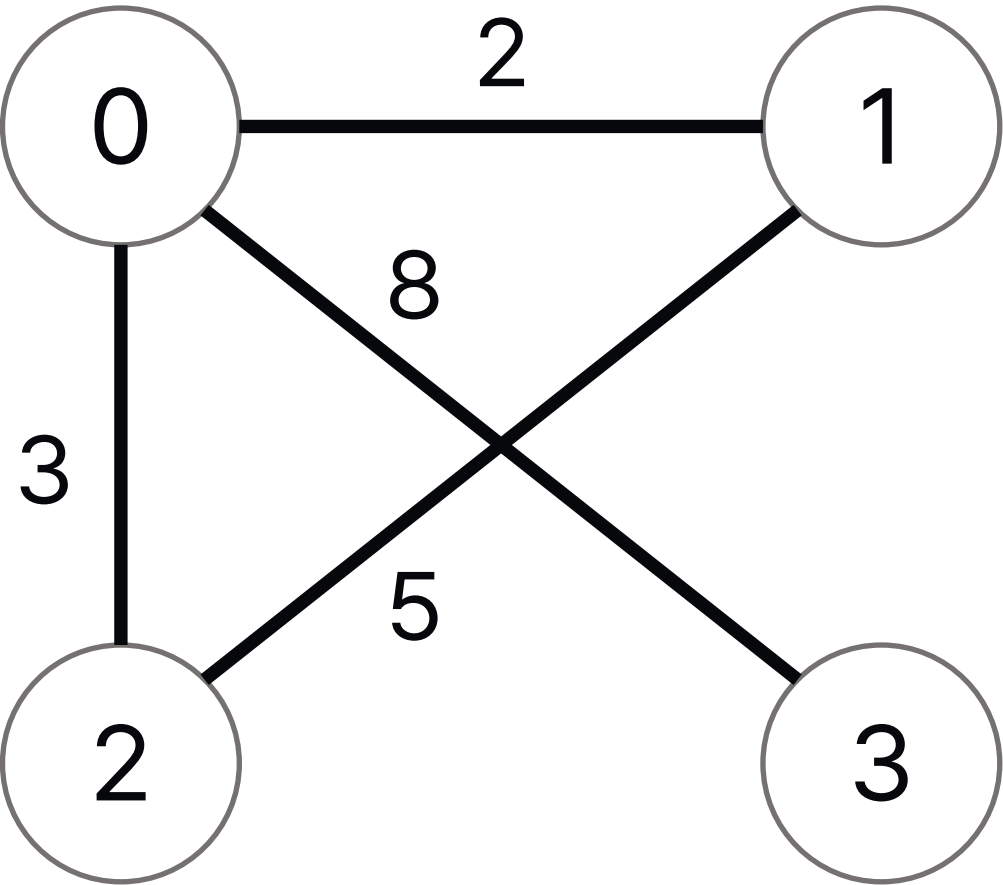


header vertex value

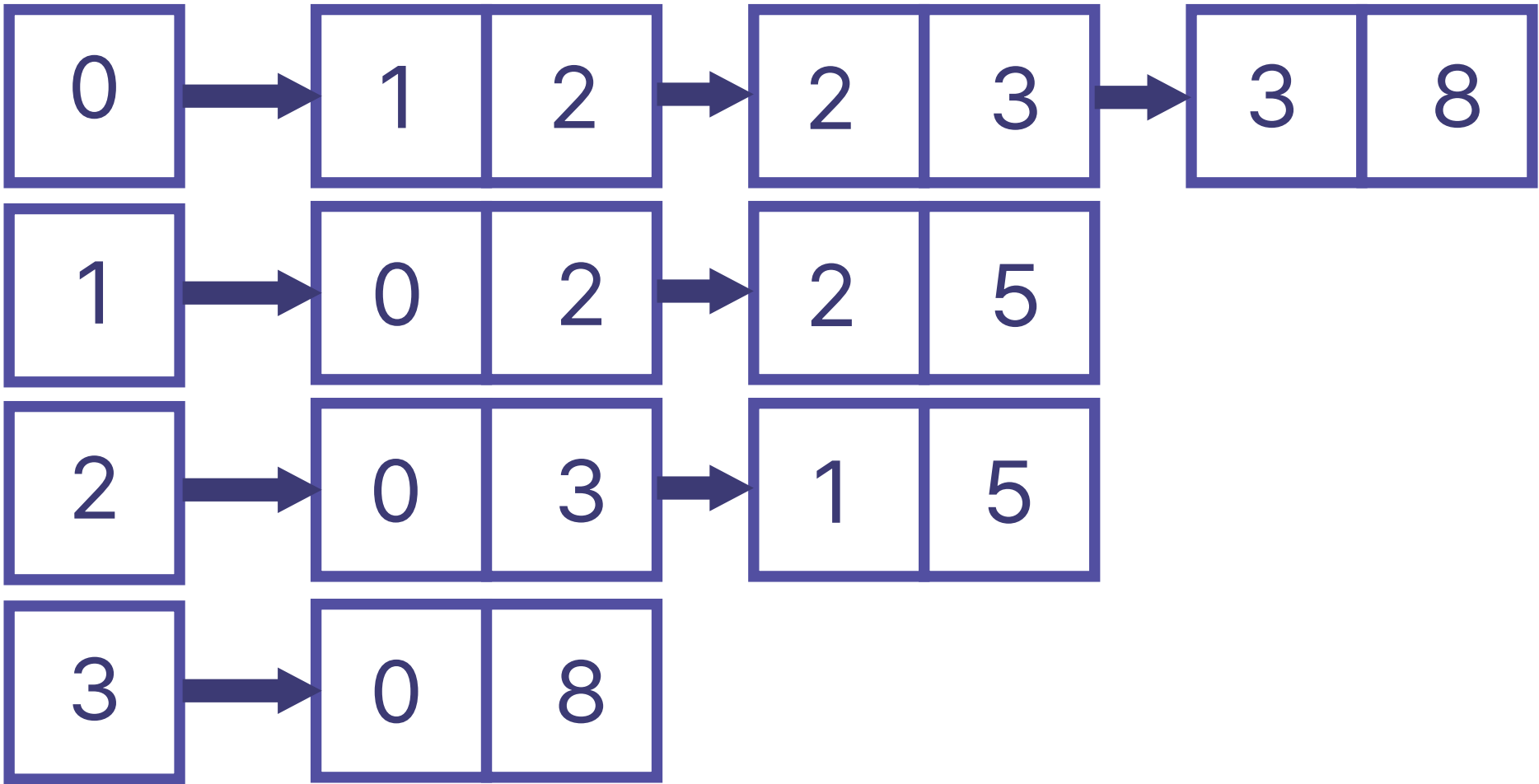




인접리스트

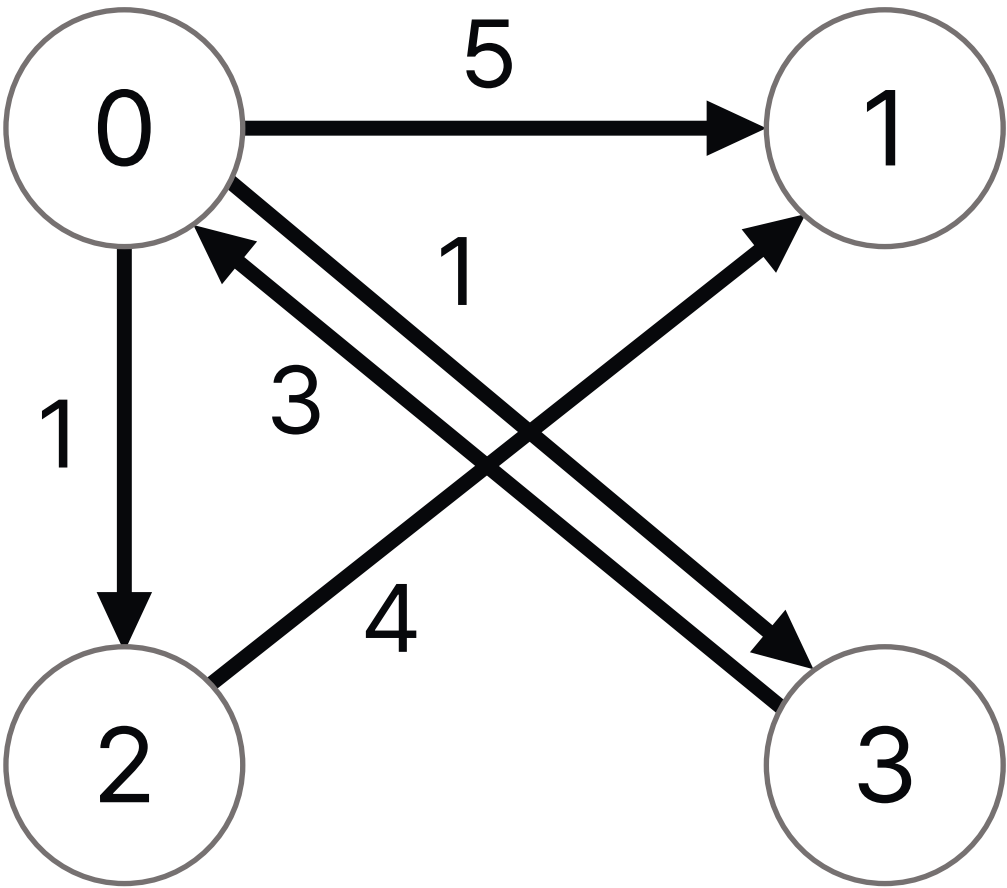


header vertex value

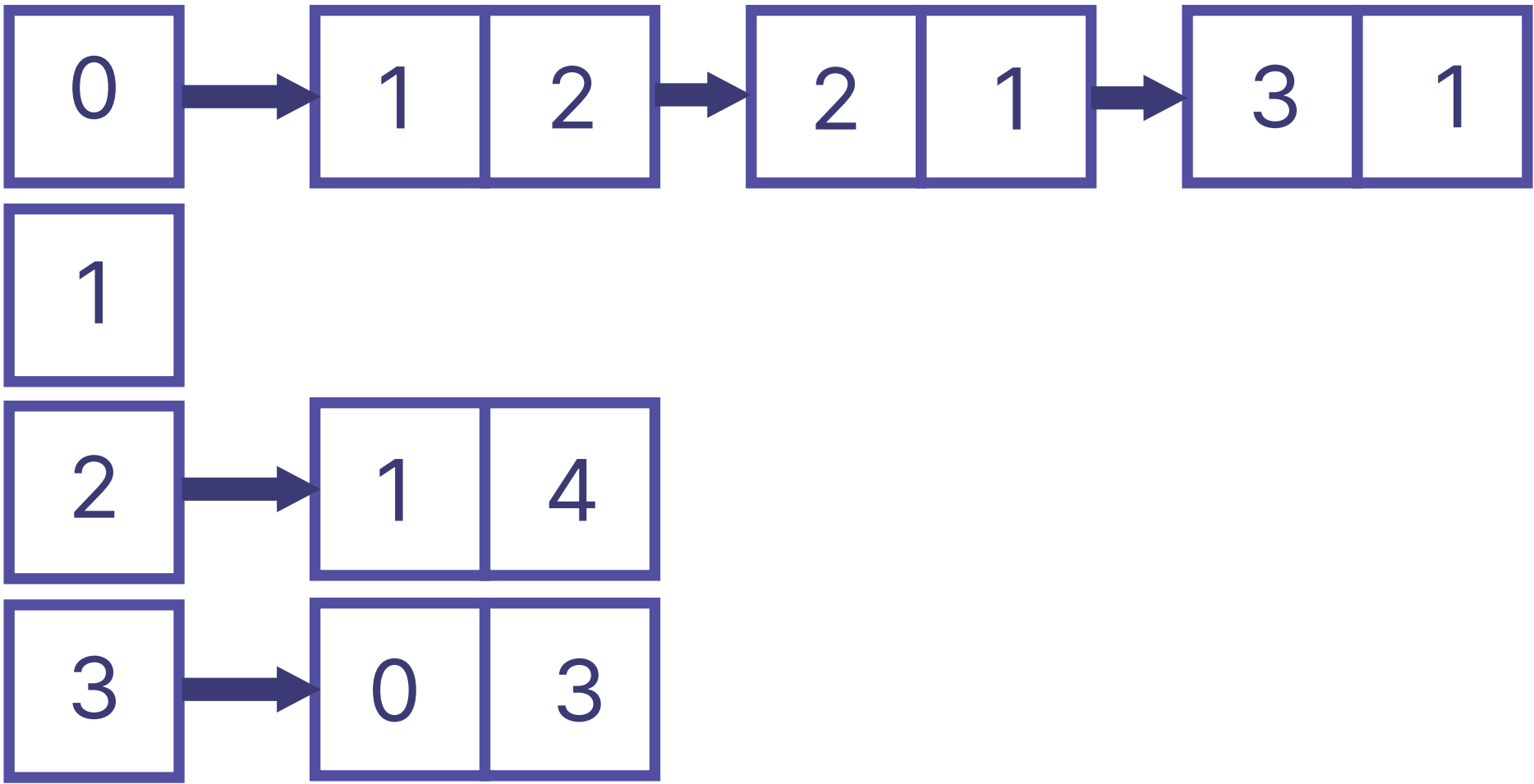




인접리스트



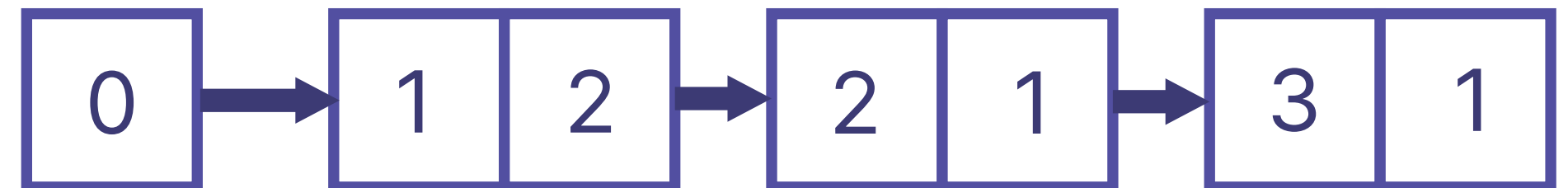
header vertex value





인접리스트

```
class AdjNode:  
    def __init__(self, vertex, value):  
        self.vertex = vertex  
        self.value = value  
        self.next = None
```





인접리스트

```
class AdjNode:  
    def __init__(self, vertex, value):  
        self.vertex = vertex  
        self.value = value  
        self.next = None
```





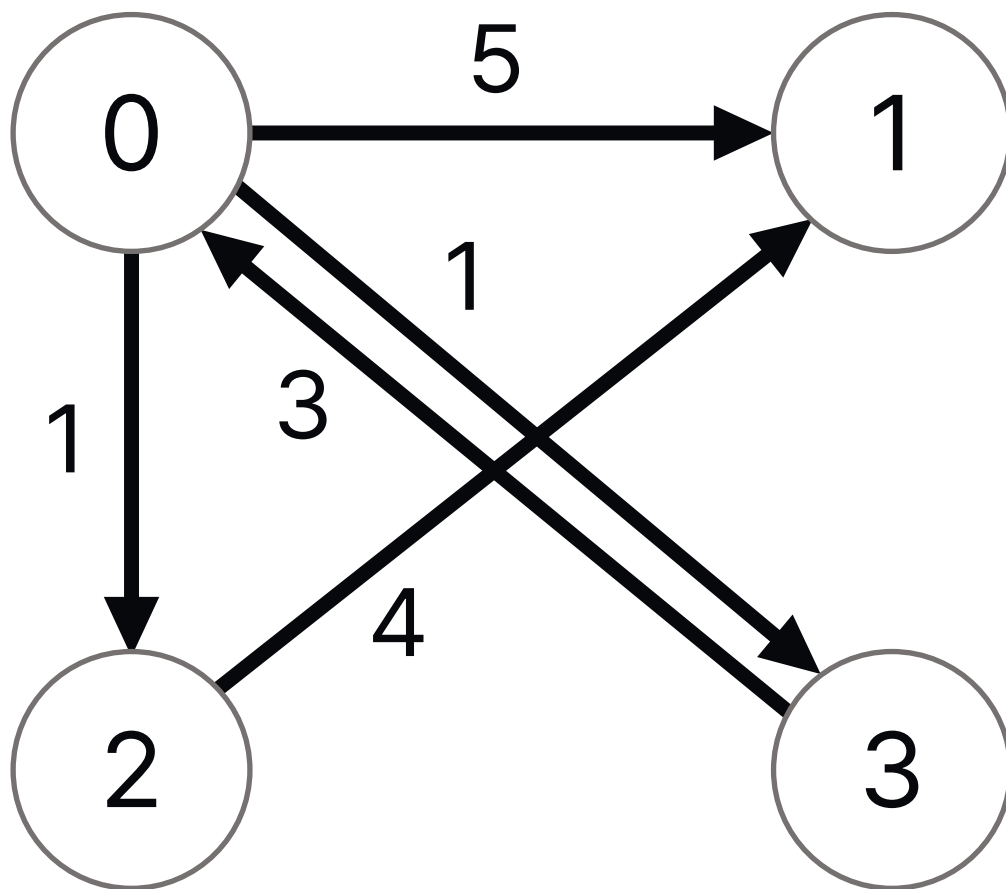
인접리스트

```
class Graph:
    def __init__(self, num):
        self.V = num
        self.graph = [None] * self.V
```

```
def add_edge(self, s, d, w):
    node = AdjNode(d,w) # vertex, value
    node.next = self.graph[s]
    self.graph[s] = node
```



인접리스트 (Python: dict)



```
graph = {0: {1:5, 2:1, 3:1}, 2: {1: 4}, 3: {0: 3}}
```

그래프 탐색



깊이 우선 탐색 (DFS)

DFS(Depth First Search)

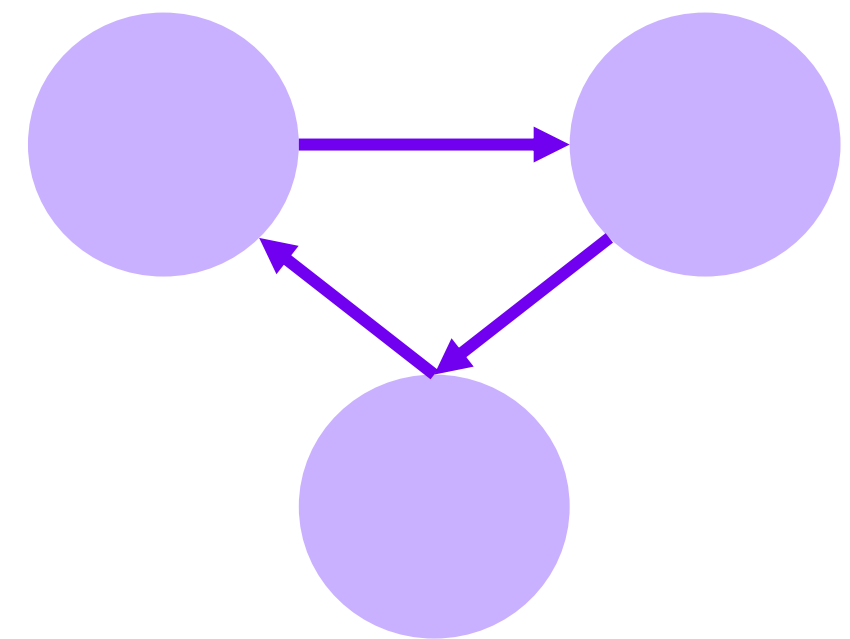
- 특정 정점(노드)에서 시작해서 트리나 그래프에서 한 가지 경로를 최대한 깊게 탐색하고, 해당 경로를 끝까지 탐색한 후 다른 경로로 이동
- 미로를 탐색할 때 한 방향으로 갈 수 있을 때까지 계속 가다가, 더 이상 갈 수 없게 되면 다시 가장 가까운 갈림길로 돌아와서 다른 방향으로 다시 탐색을 진행하는 방법과 유사
- 모든 정점을 방문하고자 하는 경우에 사용





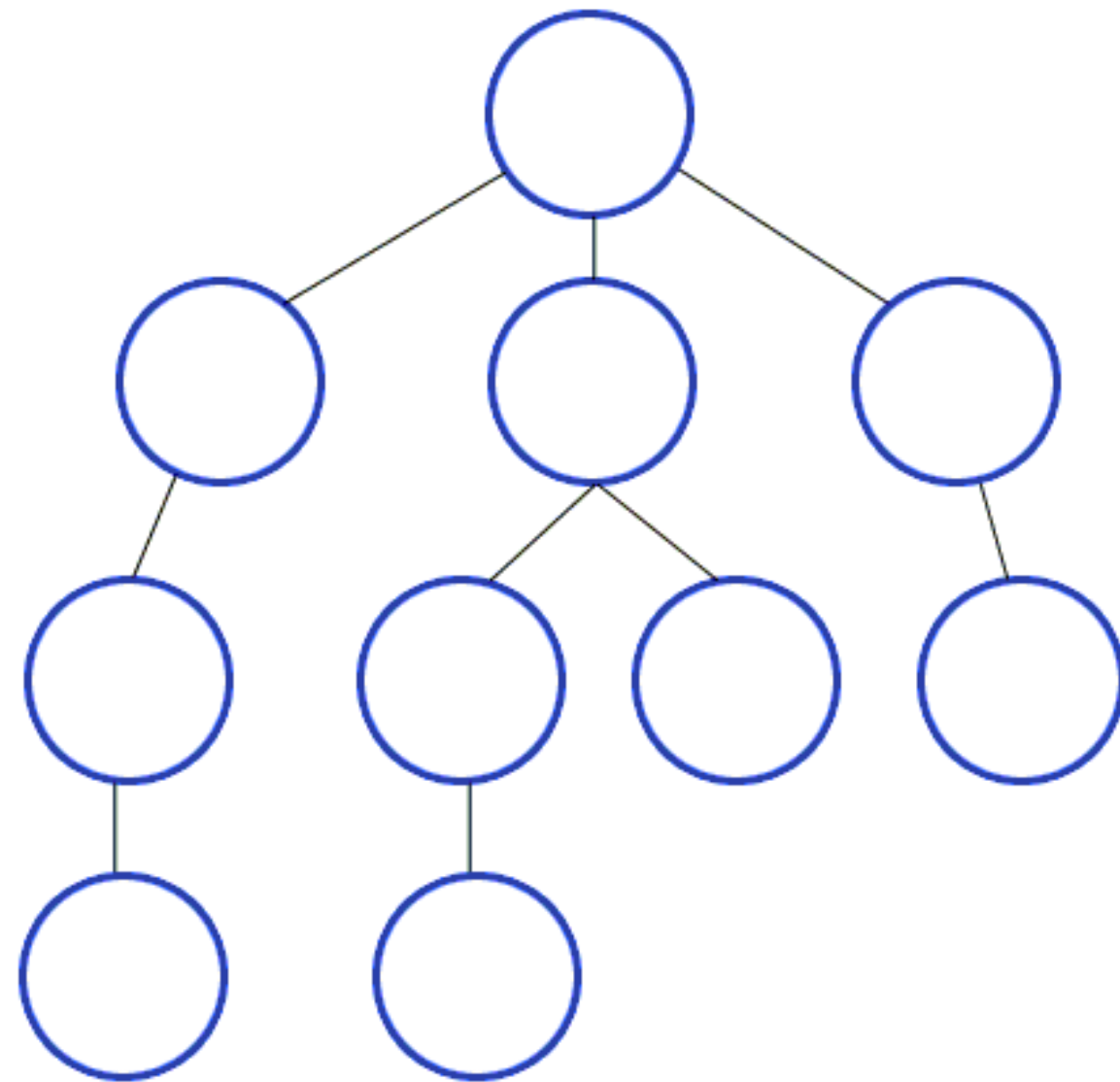
깊이 우선 탐색 (DFS)

- 일반적으로 재귀 함수를 사용하여 구현
 - 스택(Stack)으로도 구현 가능
- 모든 경우의 수에 대해 탐색을 진행
- 사이클이 있는 경우, 무한 루프에 빠지지 않도록 방문 체크를 해주어야 함
 - 사이클(Cycle) : 그래프에서 한 노드에서 시작해, 여러 간선을 지나 시작 노드로 다시 도착하는 경로
- BFS보다 깊은 경로를 빠르게 찾는데 용이





DFS의 진행 순서



한 방향으로 갈 수 있을 때까지 계속 가다가, 더 이상 갈 수 없게 되면 다시 가장 가까운 분기점으로 돌아와서 다른 방향으로 다시 탐색을 진행



DFS의 구현

함수 dfs(now):

현재 노드를 방문한 것으로 표시

현재 노드를 출력

모든 이웃 노드 'next'에 대해서 반복:

만약 'next'를 아직 방문하지 않았다면 :

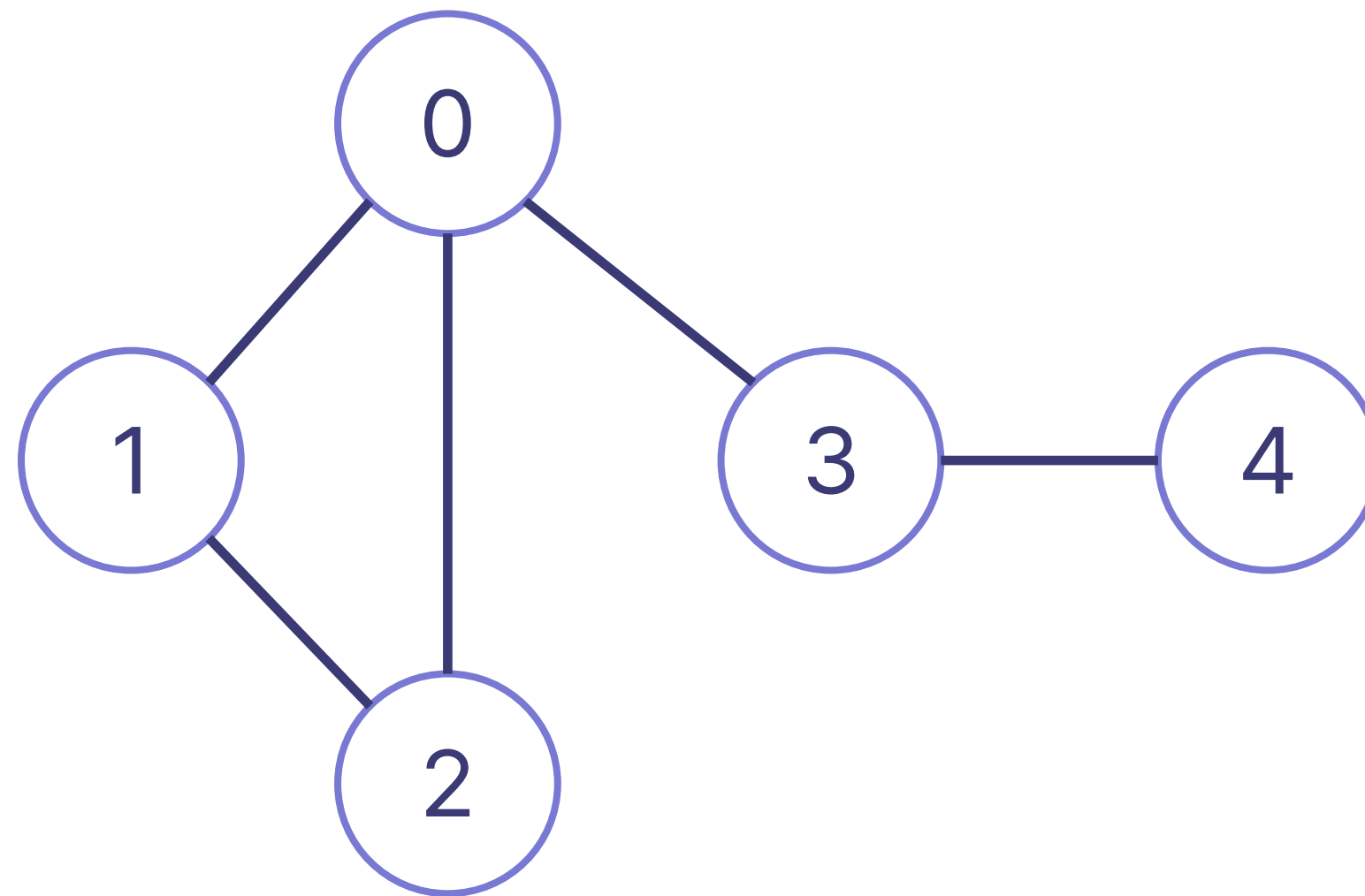
dfs(next)

```
def dfs(now):  
    visited[now] = 1  
    print(now, end=" ")  
  
    for next in v[now]:  
        if visited[next] == 0:  
            dfs(next)
```



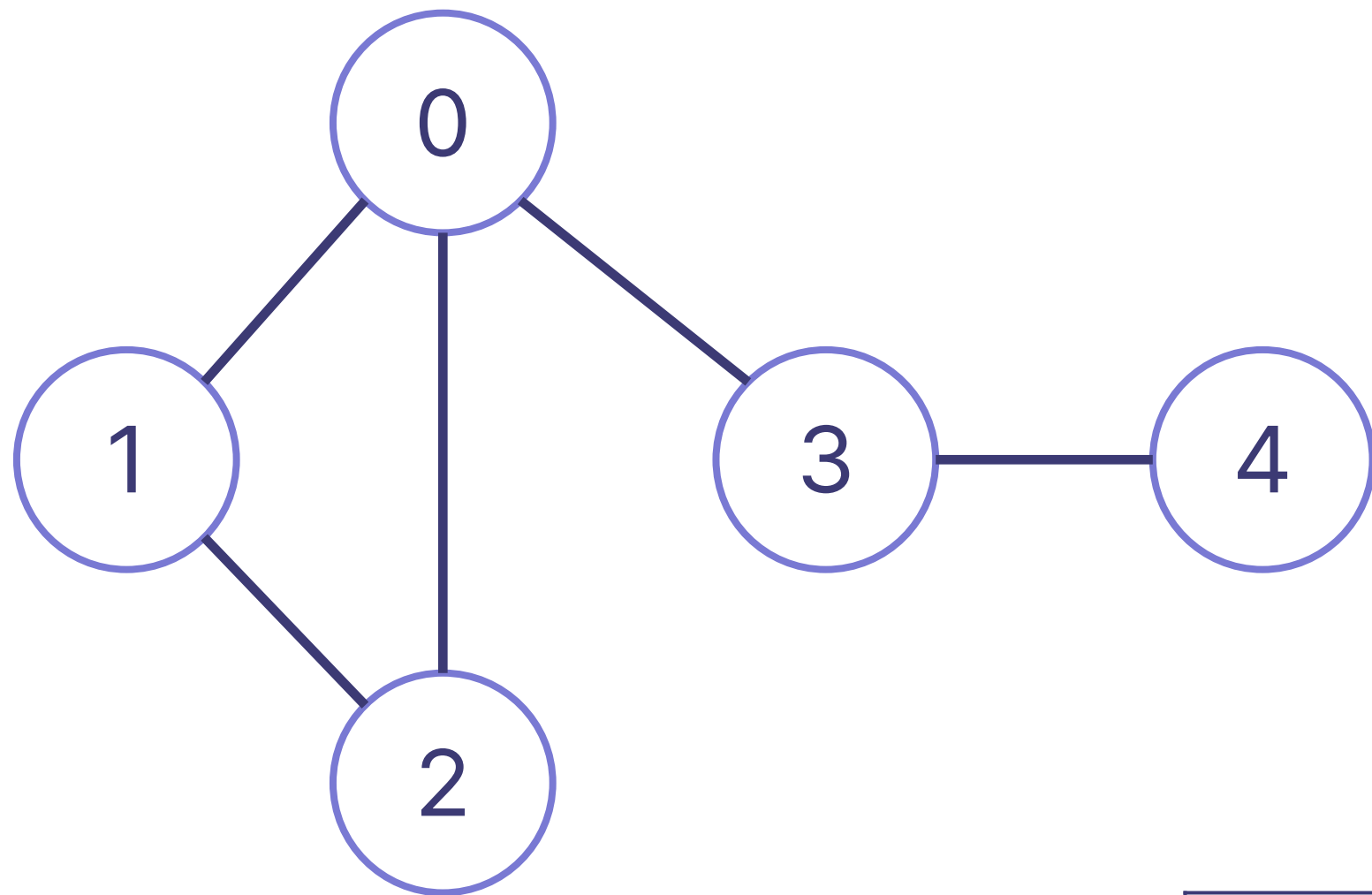
깊이우선탐색 (DFS, Depth-First Search)

그래프를 탐색하는 기법 중 하나로,
다음 분기로 넘어가기 전에 해당 분기를 우선적으로 탐색하는 방법



깊이우선탐색 (DFS, Depth-First Search)

그래프를 탐색하는 기법 중 하나로,
다음 분기로 넘어가기 전에 해당 분기를 우선적으로 탐색하는 방법

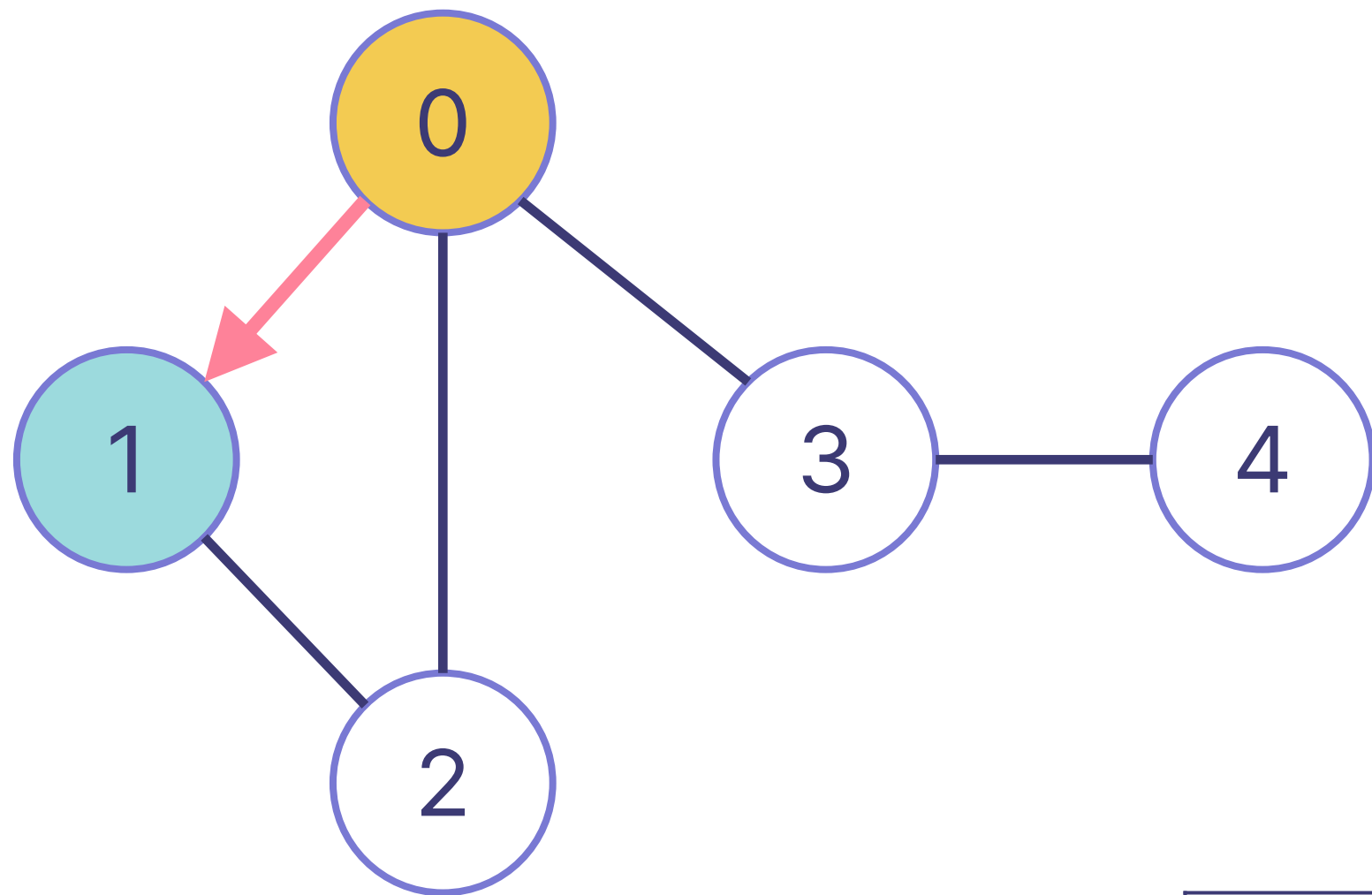


Stack



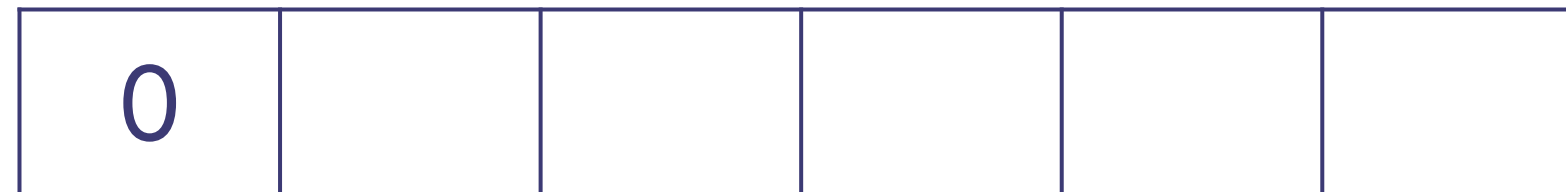
깊이우선탐색 (DFS, Depth-First Search)

그래프를 탐색하는 기법 중 하나로,
다음 분기로 넘어가기 전에 해당 분기를 우선적으로 탐색하는 방법



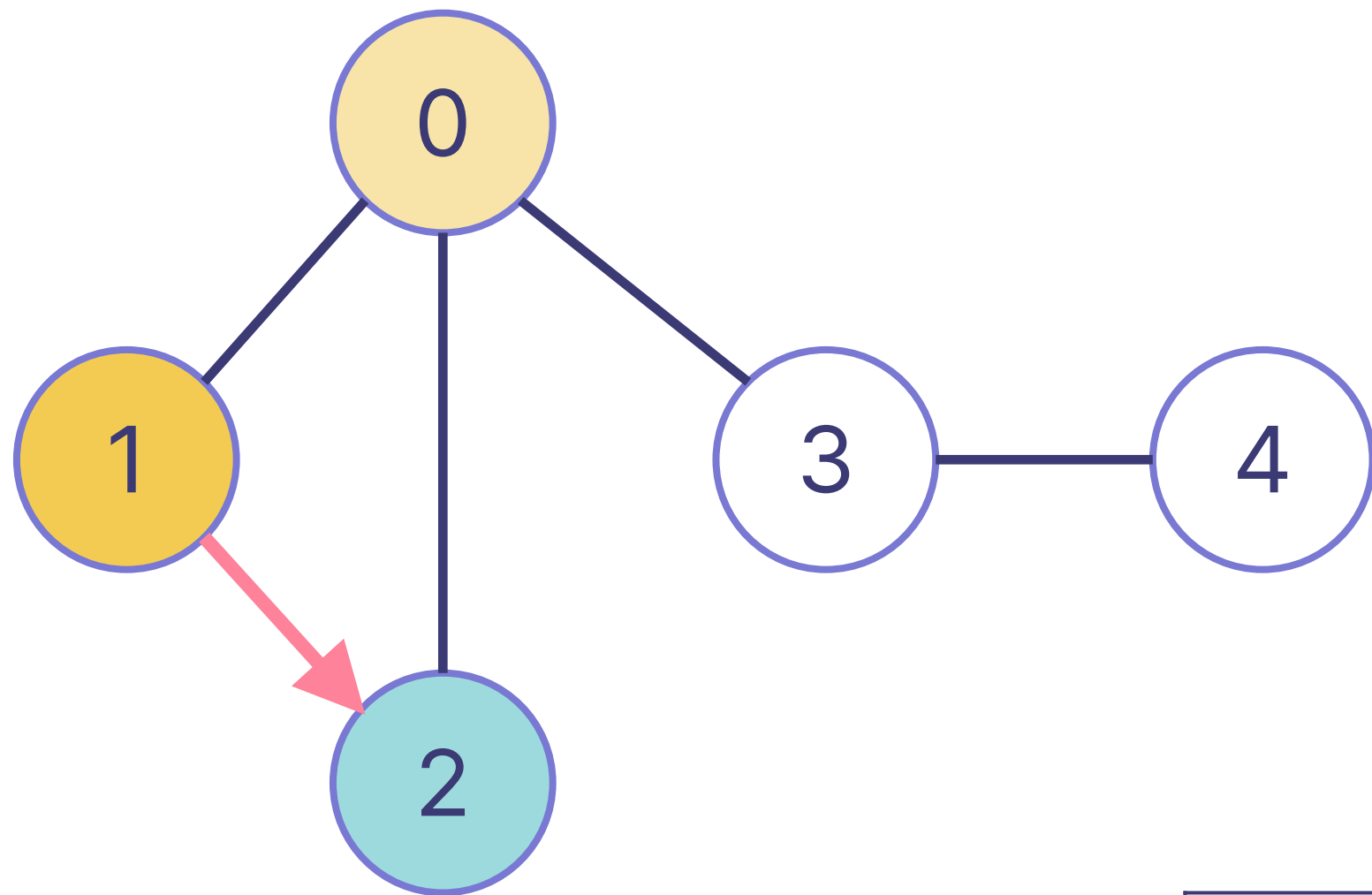
0

Stack



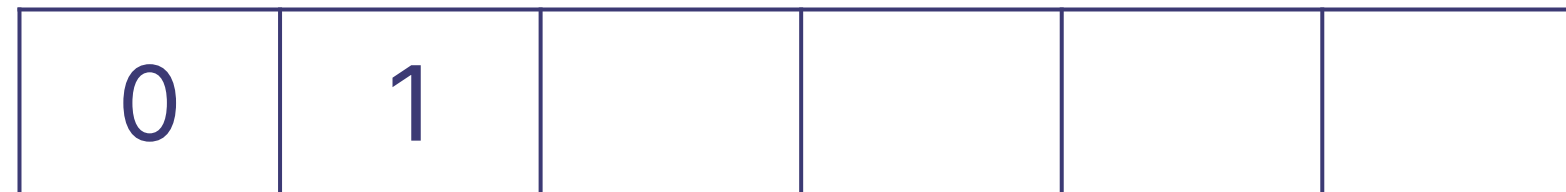
깊이우선탐색 (DFS, Depth-First Search)

그래프를 탐색하는 기법 중 하나로,
다음 분기로 넘어가기 전에 해당 분기를 우선적으로 탐색하는 방법



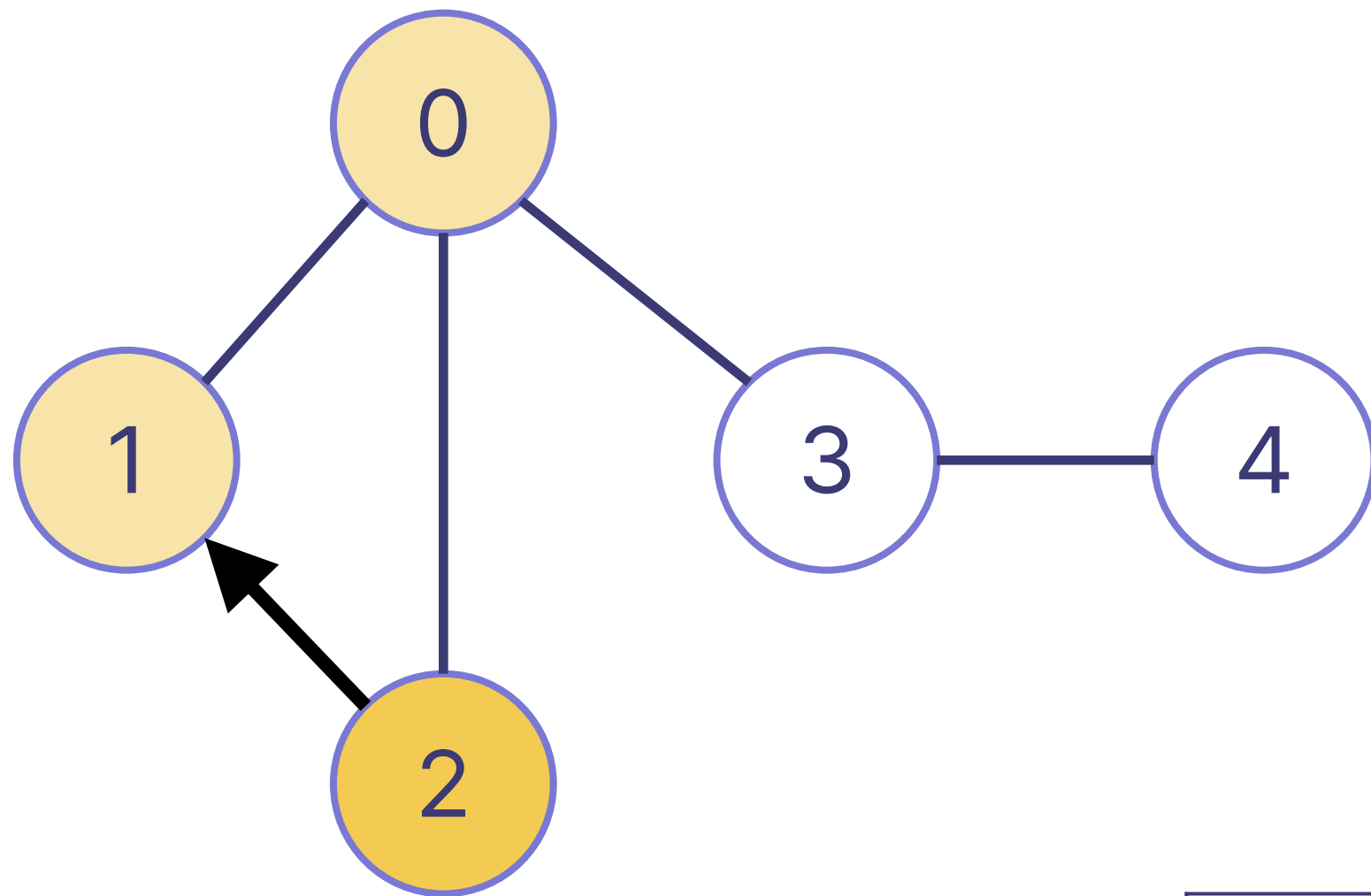
0 1

Stack



깊이우선탐색 (DFS, Depth-First Search)

그래프를 탐색하는 기법 중 하나로,
다음 분기로 넘어가기 전에 해당 분기를 우선적으로 탐색하는 방법



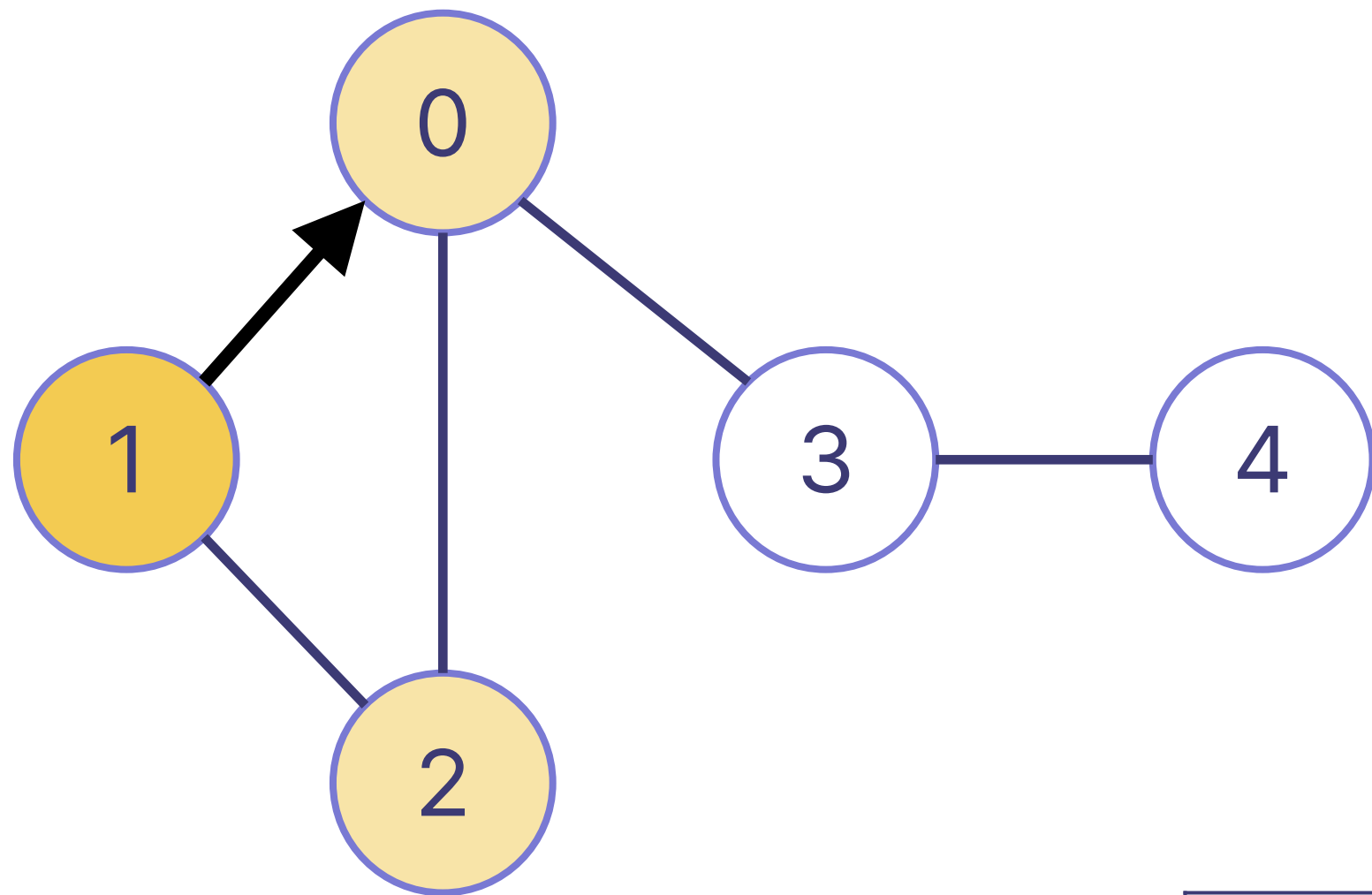
0 1 2

Stack

0	1	2			
---	---	---	--	--	--

깊이우선탐색 (DFS, Depth-First Search)

그래프를 탐색하는 기법 중 하나로,
다음 분기로 넘어가기 전에 해당 분기를 우선적으로 탐색하는 방법



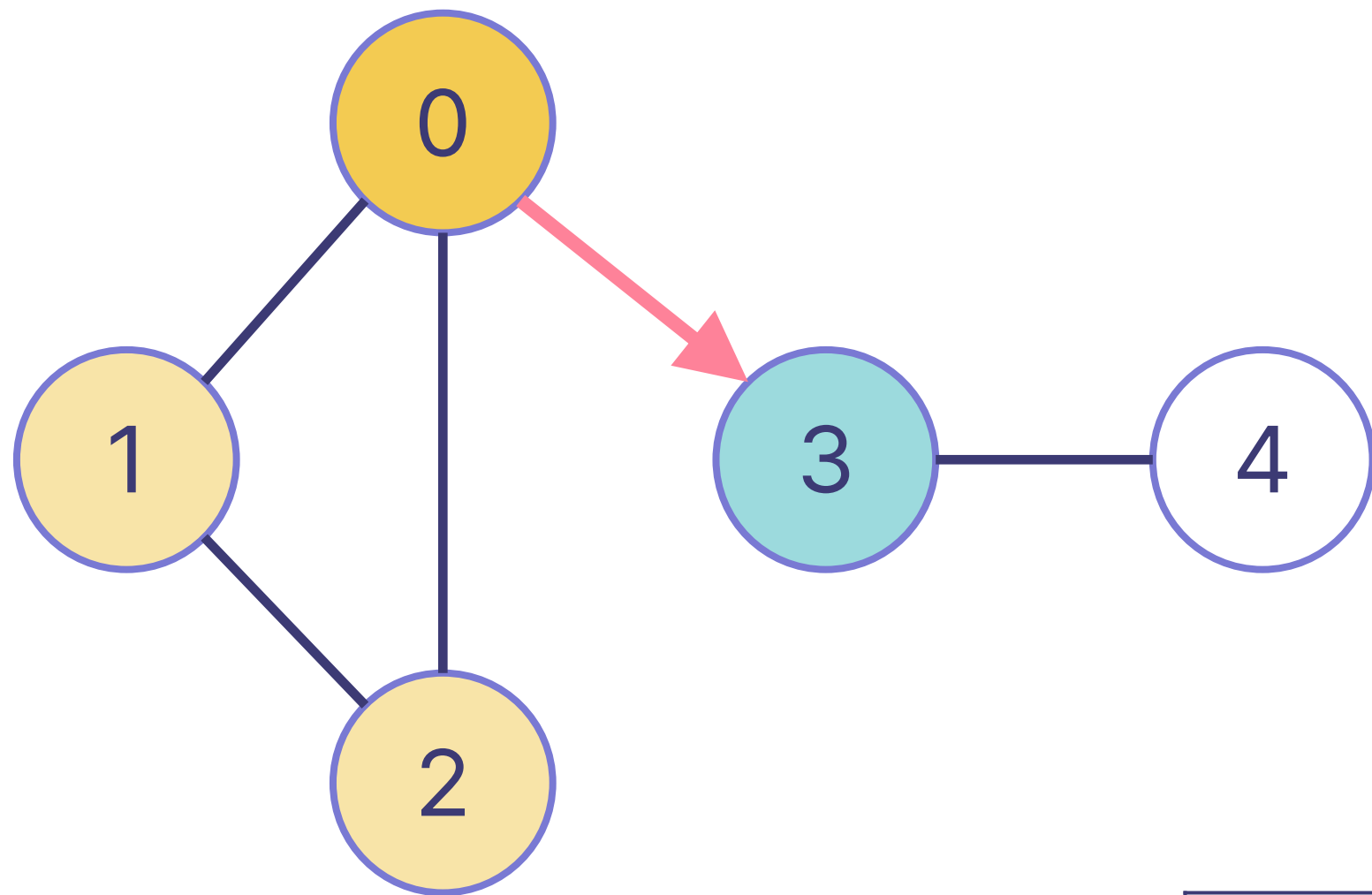
0 1 2

Stack



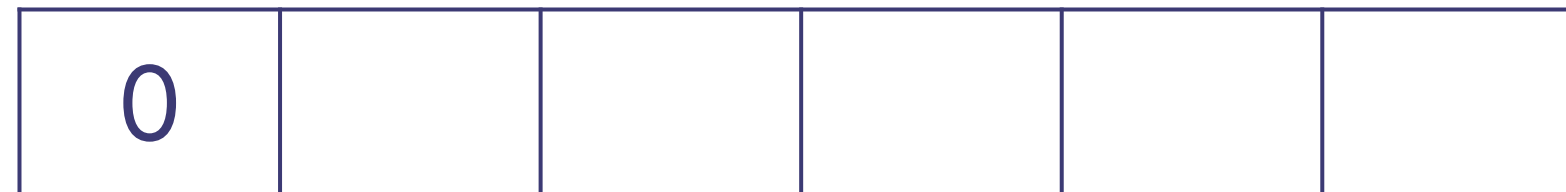
깊이우선탐색 (DFS, Depth-First Search)

그래프를 탐색하는 기법 중 하나로,
다음 분기로 넘어가기 전에 해당 분기를 우선적으로 탐색하는 방법



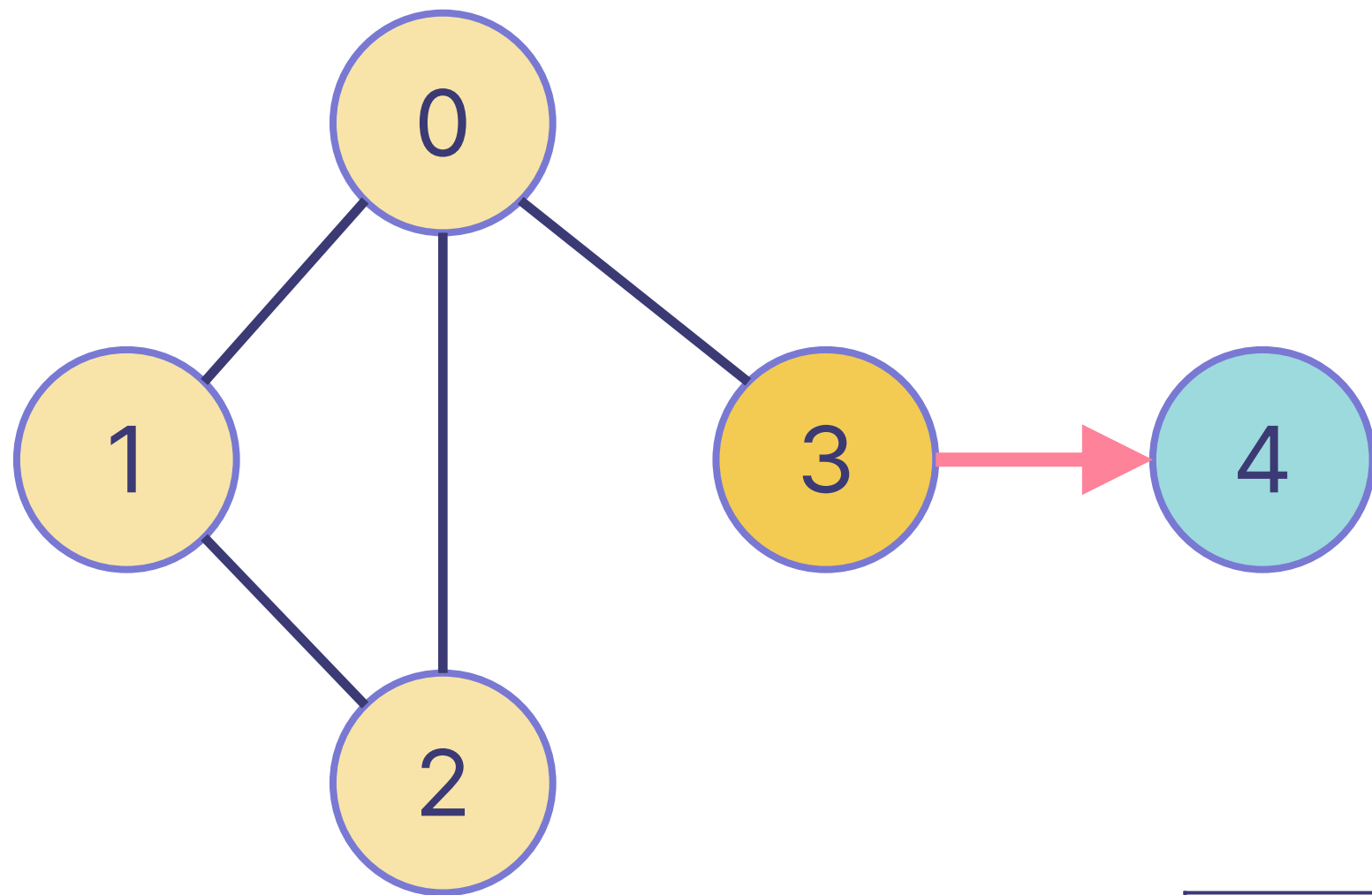
0 1 2

Stack



깊이우선탐색 (DFS, Depth-First Search)

그래프를 탐색하는 기법 중 하나로,
다음 분기로 넘어가기 전에 해당 분기를 우선적으로 탐색하는 방법



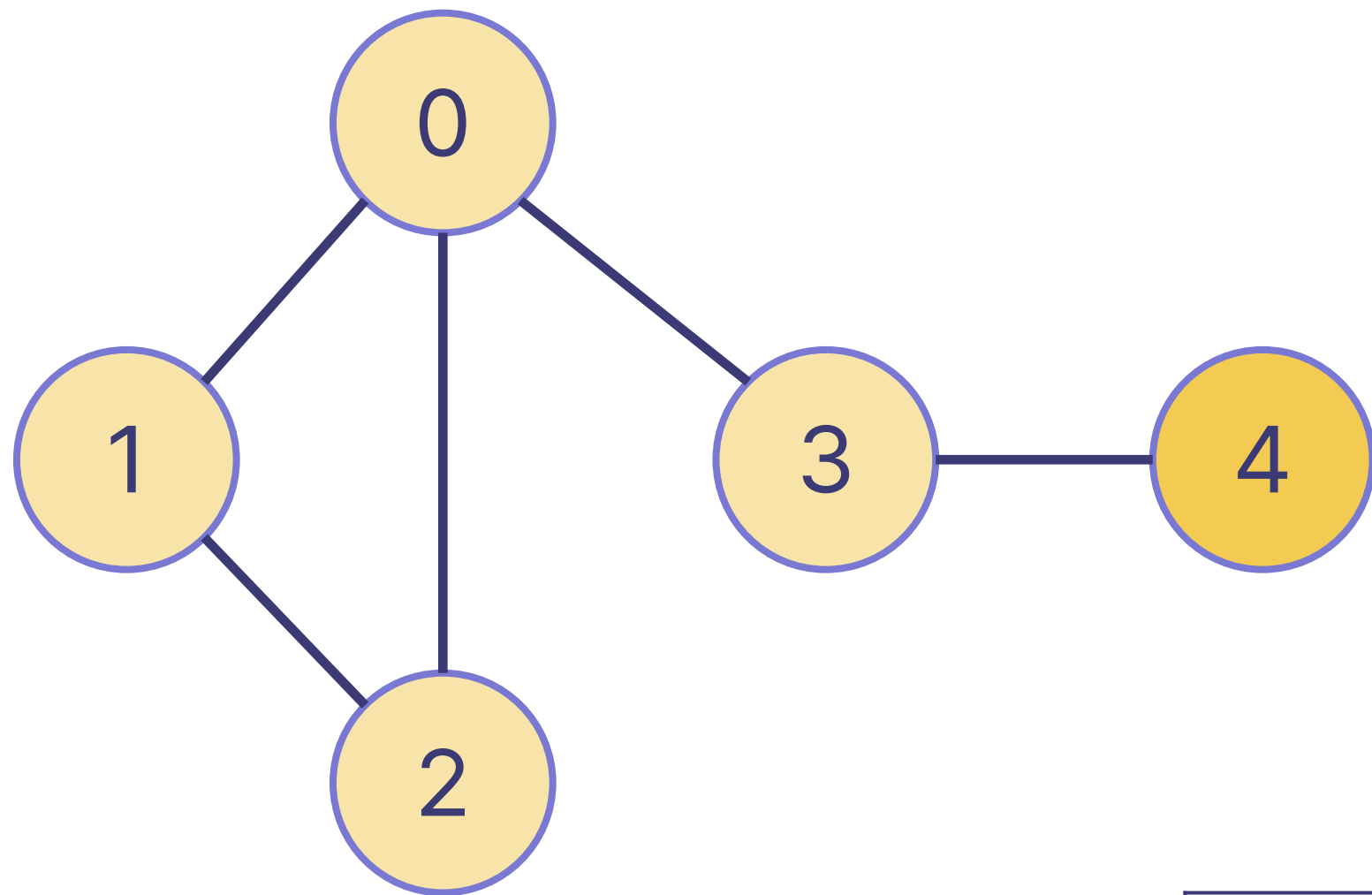
0 1 2 3

Stack

0	3				
---	---	--	--	--	--

깊이우선탐색 (DFS, Depth-First Search)

그래프를 탐색하는 기법 중 하나로,
다음 분기로 넘어가기 전에 해당 분기를 우선적으로 탐색하는 방법



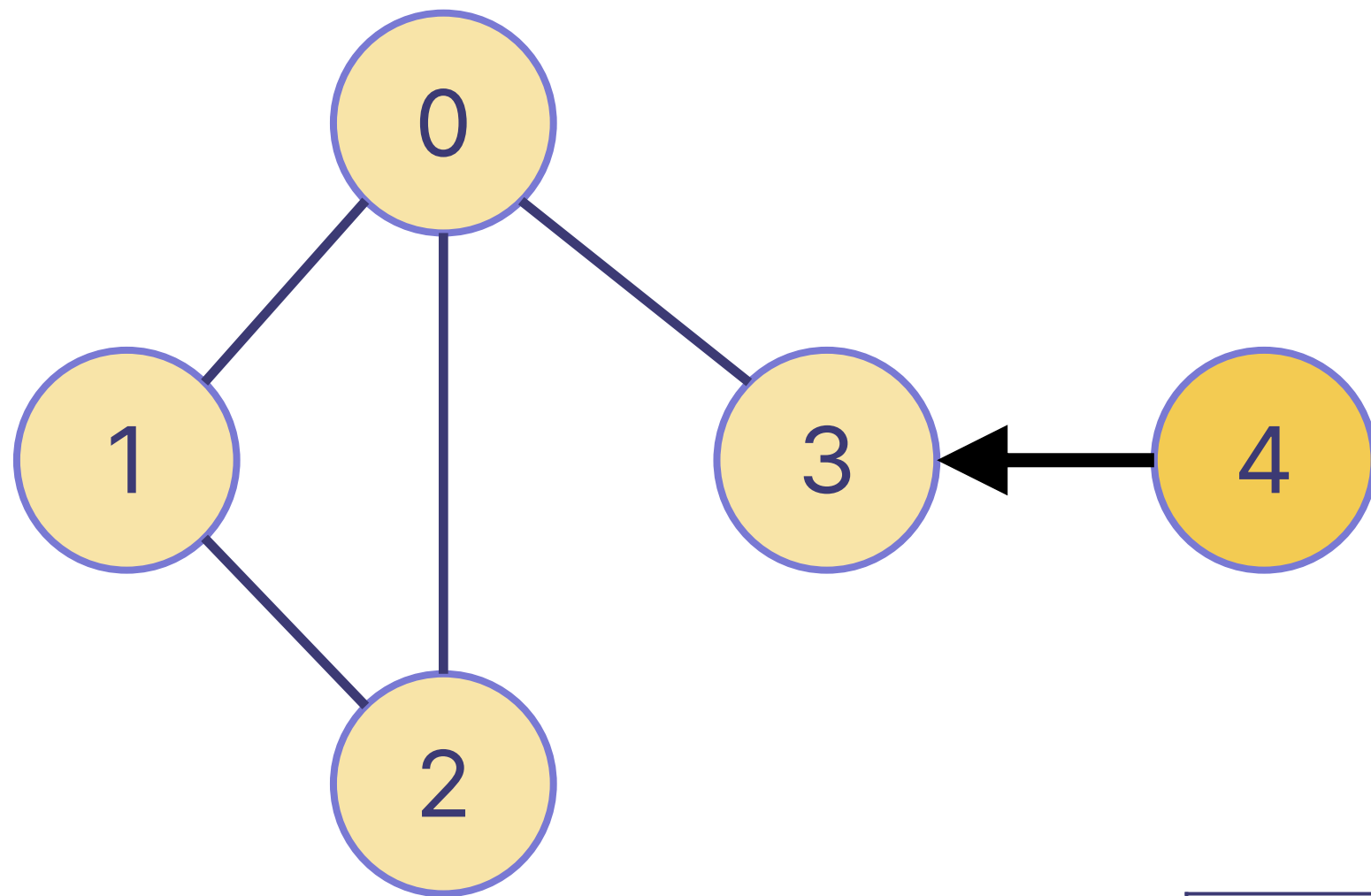
0 1 2 3 4

Stack

0	3	4			
---	---	---	--	--	--

깊이우선탐색 (DFS, Depth-First Search)

그래프를 탐색하는 기법 중 하나로,
다음 분기로 넘어가기 전에 해당 분기를 우선적으로 탐색하는 방법



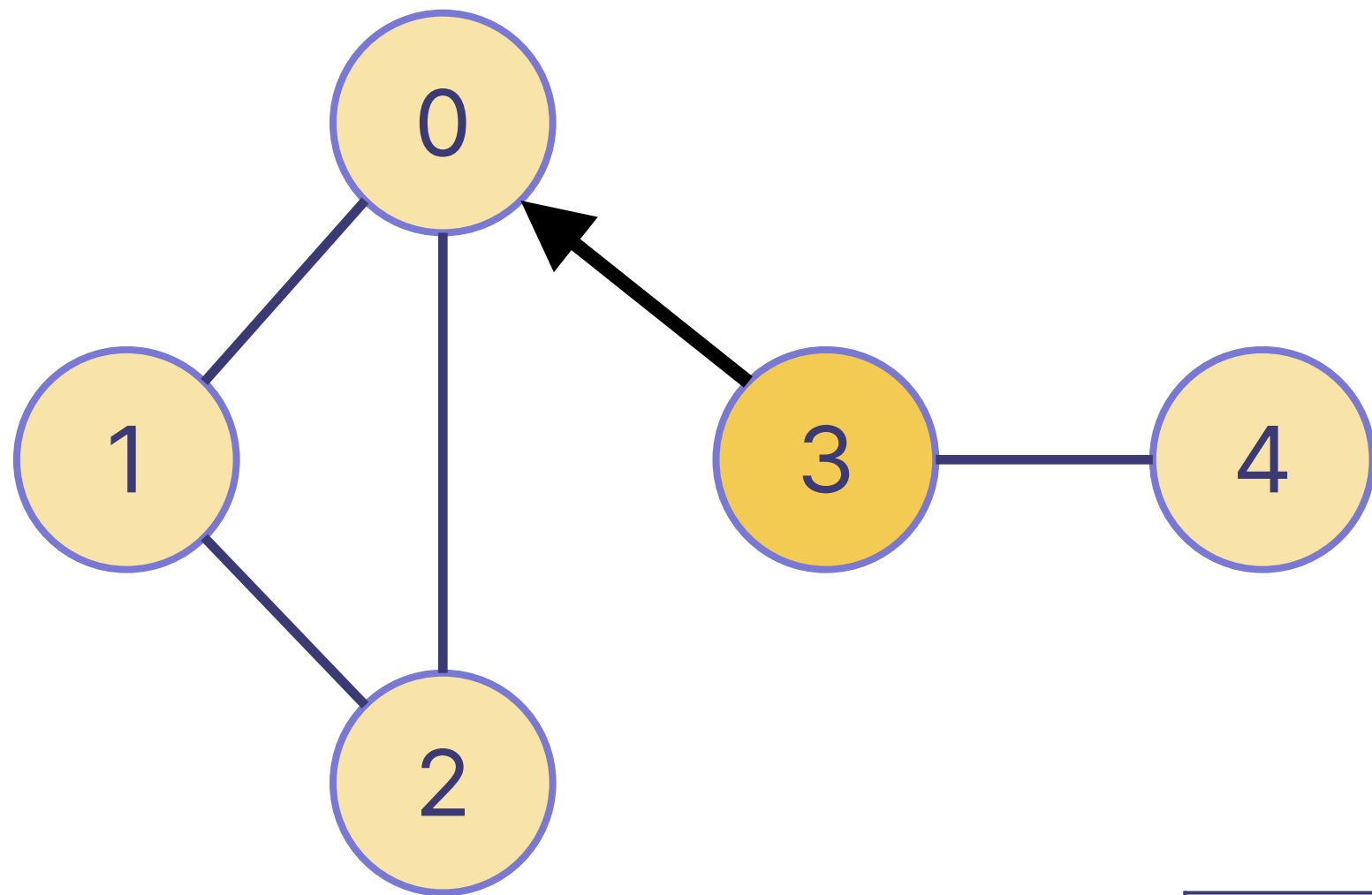
0 1 2 3 4

Stack

0	3	4			
---	---	---	--	--	--

깊이우선탐색 (DFS, Depth-First Search)

그래프를 탐색하는 기법 중 하나로,
다음 분기로 넘어가기 전에 해당 분기를 우선적으로 탐색하는 방법



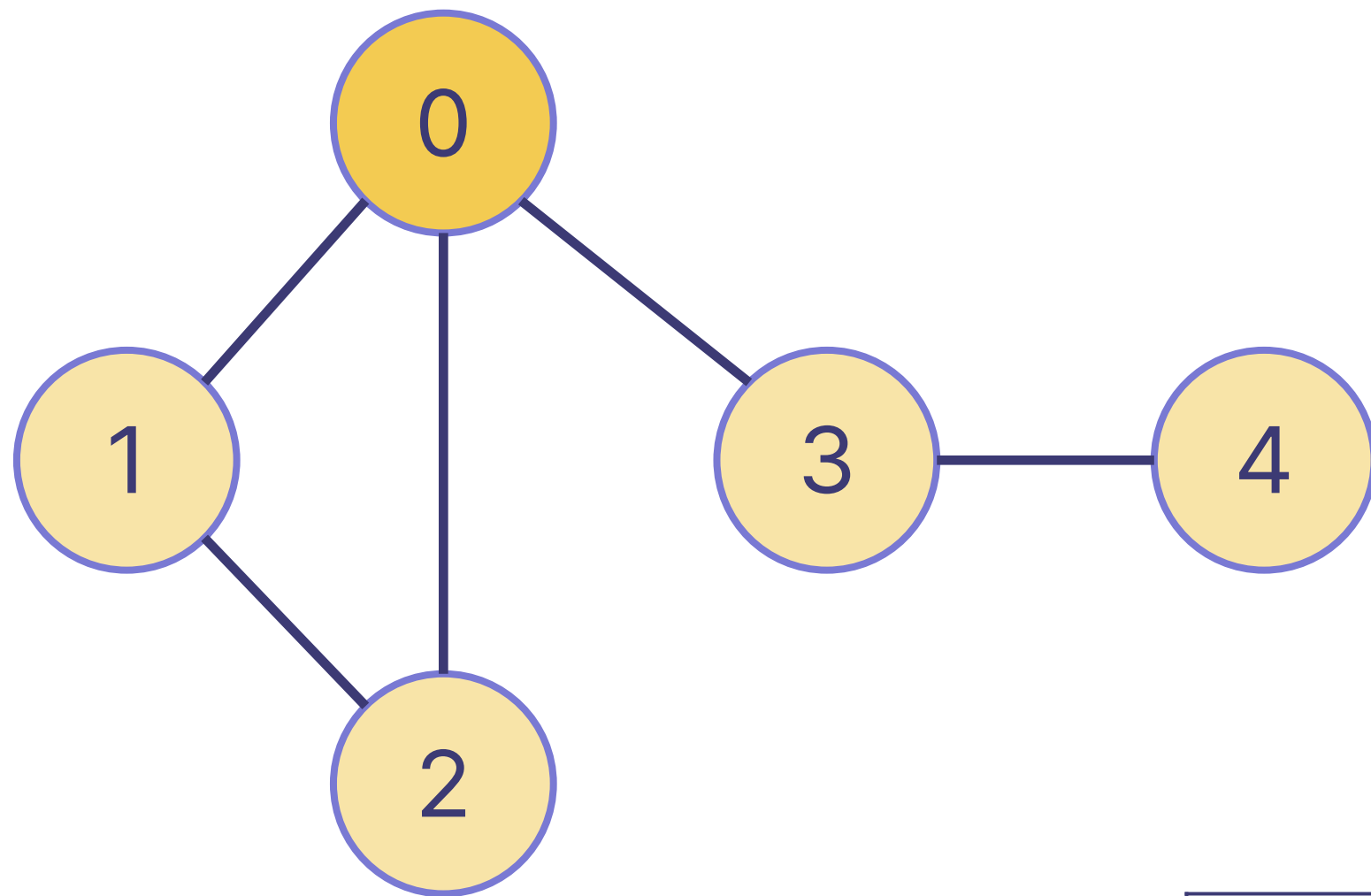
0 1 2 3 4

Stack

0	3				
---	---	--	--	--	--

깊이우선탐색 (DFS, Depth-First Search)

그래프를 탐색하는 기법 중 하나로,
다음 분기로 넘어가기 전에 해당 분기를 우선적으로 탐색하는 방법



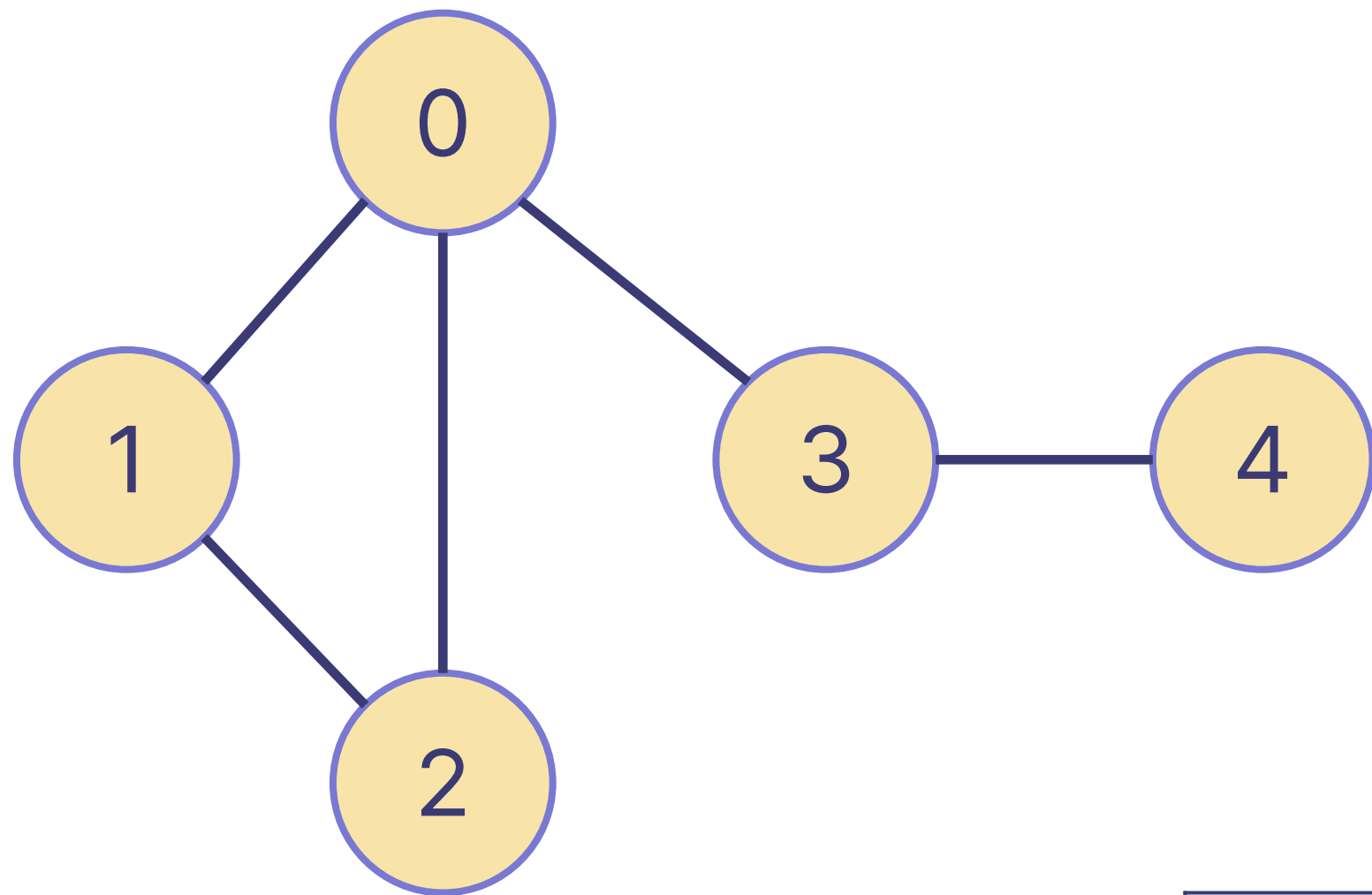
0 1 2 3 4

Stack



깊이우선탐색 (DFS, Depth-First Search)

그래프를 탐색하는 기법 중 하나로,
다음 분기로 넘어가기 전에 해당 분기를 우선적으로 탐색하는 방법



0 1 2 3 4

Stack



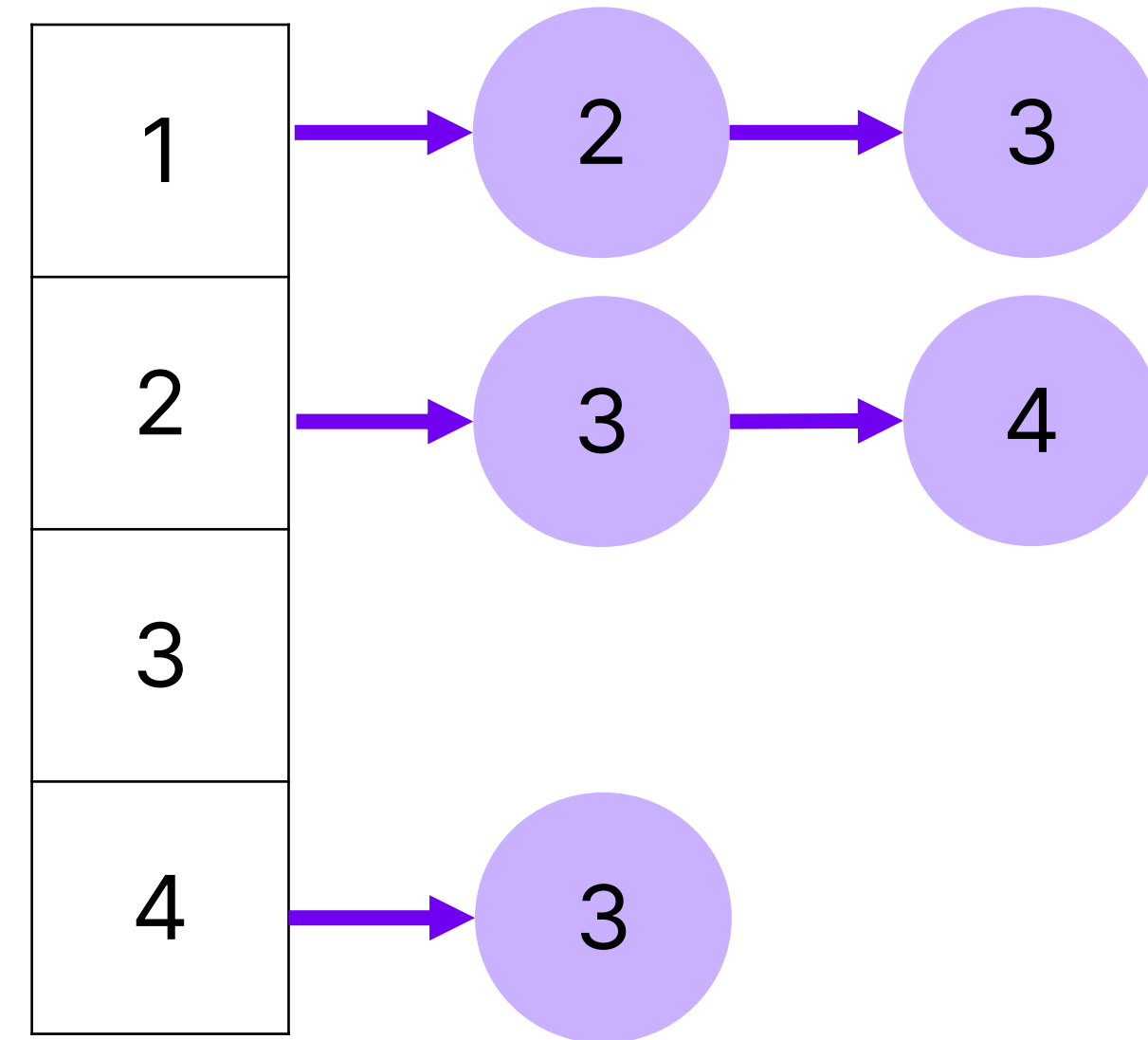
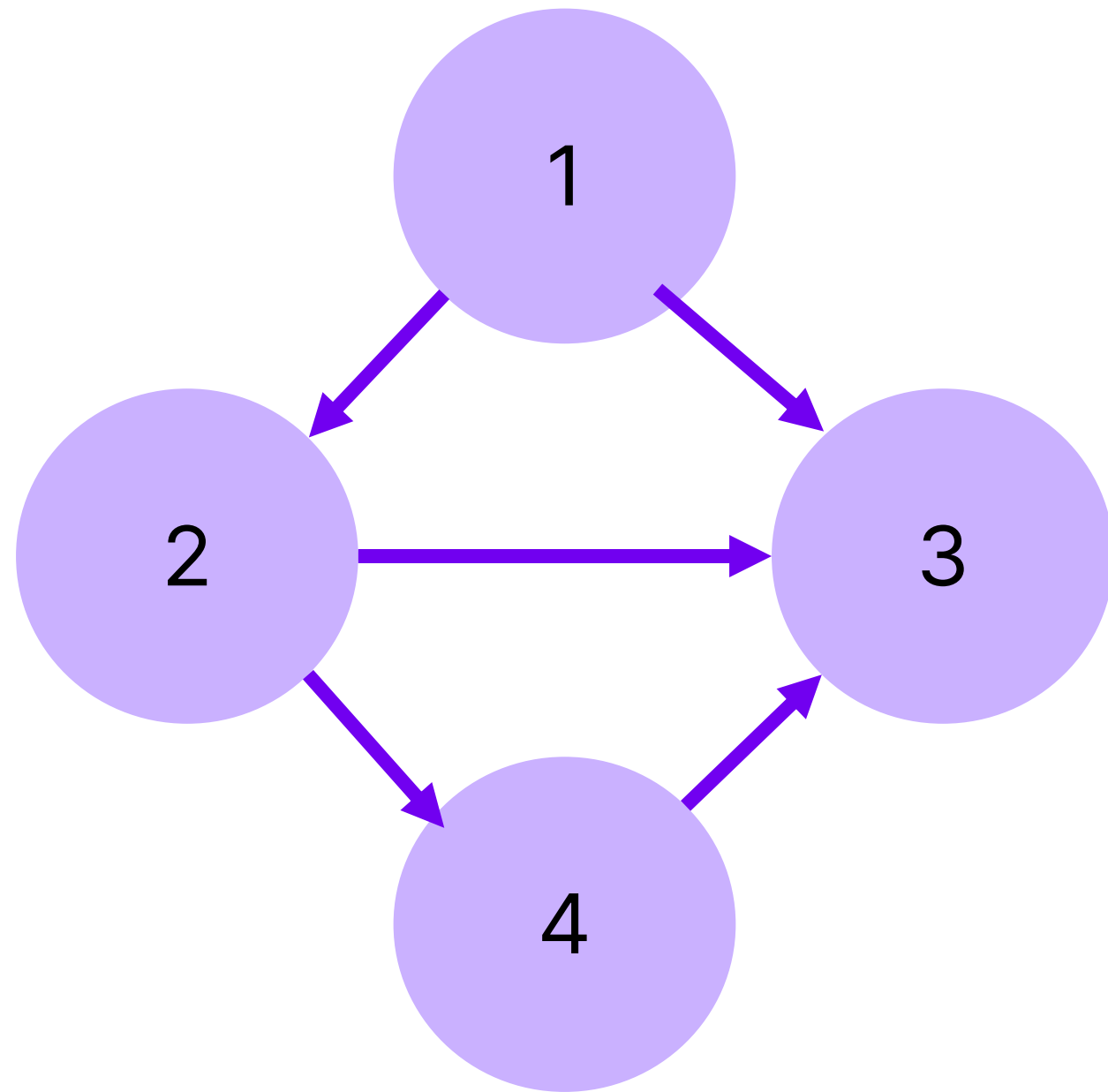


DFS의 시간 복잡도

- $V \rightarrow$ 정점(노드)의 수, $E \rightarrow$ 간선의 수
- 인접 리스트로 표현된 그래프 : $O(V + E)$
- 인접 행렬로 표현된 그래프 : $O(V^2)$
- 희소 그래프의 경우 인접 행렬보다 인접 리스트를 사용하는 것이 유리
 - 희소 그래프(Sparse Graph) $\rightarrow$ 그래프 내에 적은 숫자의 간선만을 가지는 그래프



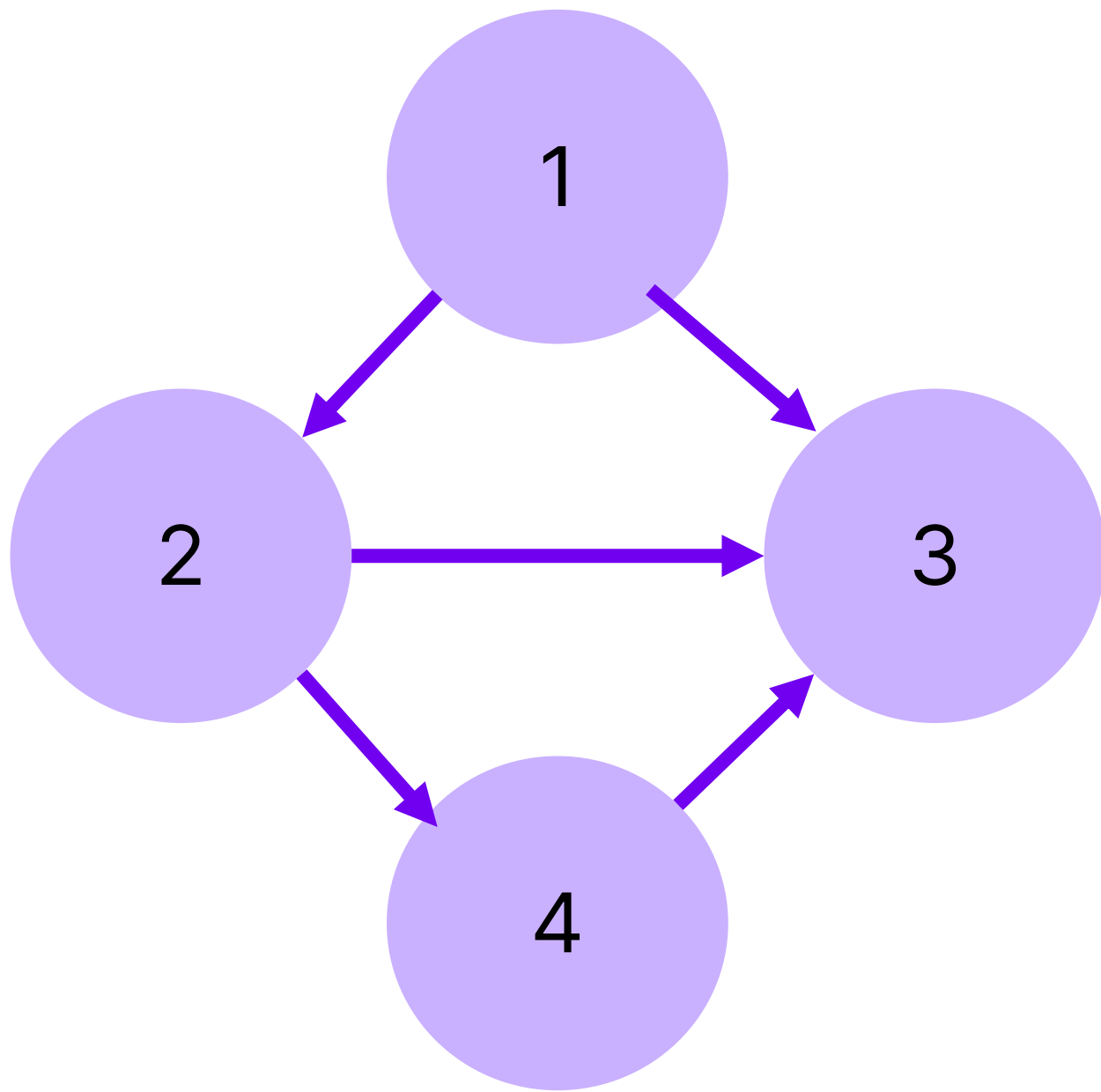
인접 리스트로 표현된 그래프: $O(V + E)$



정점 1에서 2, 3으로 이동할 수 있는 간선이 존재



행렬로 표현된 그래프: $O(V^2)$



0	1	1	0
0	0	1	1
0	0	0	0
0	0	1	0

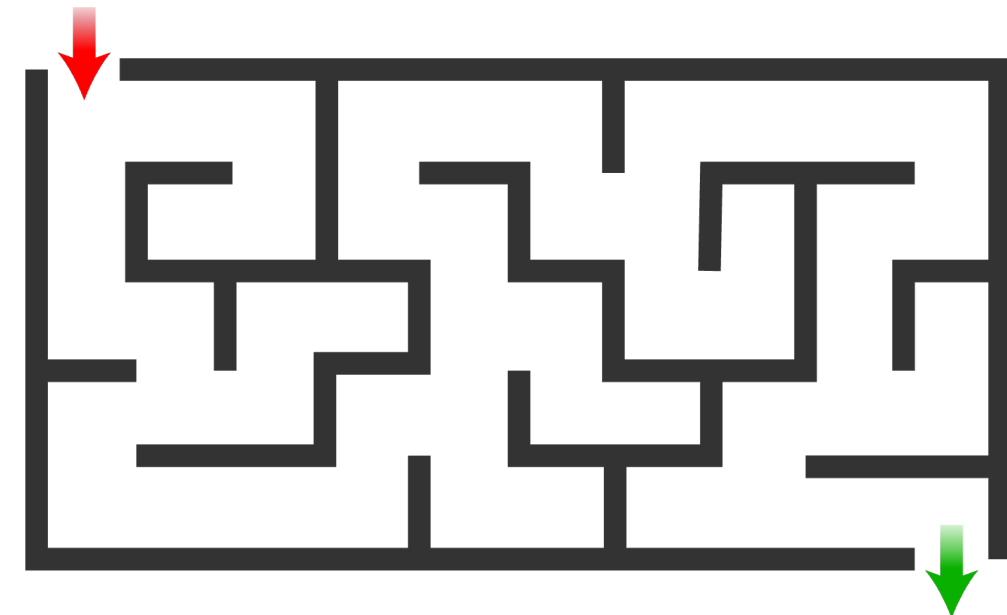
$(x, y) = 1 \rightarrow x$ 에서 y 로 가는 간선이 존재



너비 우선 탐색 (BFS)

BFS(Breadth First Search)

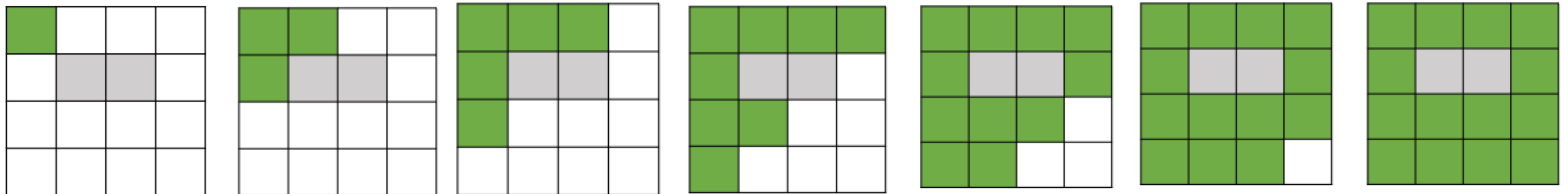
- 하나의 정점으로부터 시작하여 차례대로 모든 정점들을 한 번씩 방문하는 것
- 루트 노드(혹은 다른 임의의 노드)에서 시작해서 인접한 노드를 먼저 탐색하는 방법
- 두 노드 사이의 **최단 경로** 혹은 임의의 경로를 찾고 싶을 때 사용





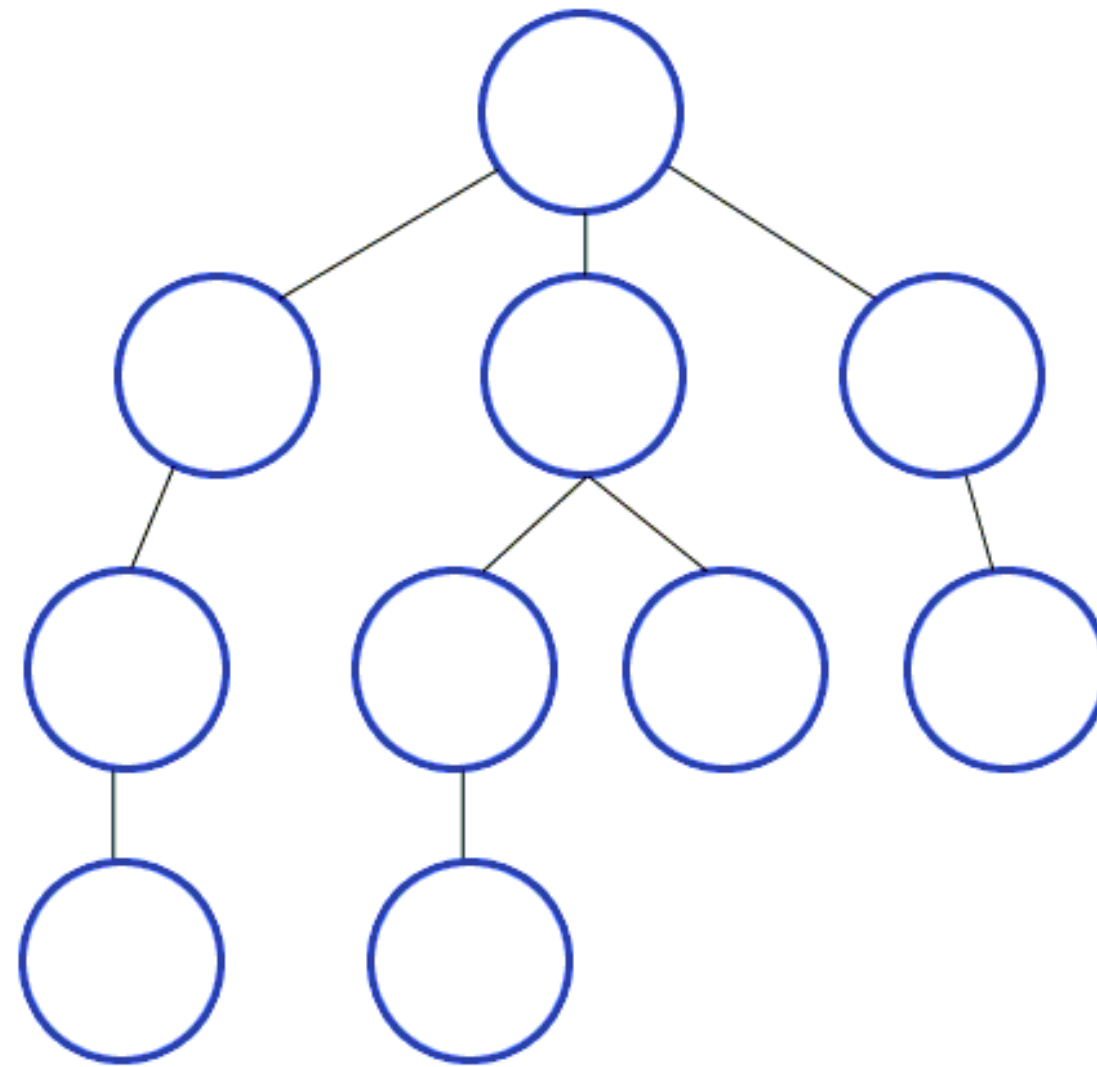
너비 우선 탐색 (BFS)

- 재귀적으로 동작하는 DFS와 달리 BFS는 주로 큐(Queue)를 사용
- 사이클이 있는 경우, 무한 루프에 빠지지 않도록 방문 체크를 해주어야 함
- 물웅덩이에 돌멩이를 하나 던지면, 파동이 전체 방향으로 퍼져 나가는 동심원의 형태로 탐색이 진행





BFS의 동작 순서



특정 노드에서 시작하여, 인접한 노드를 먼저 탐색



BFS의 구현

빈 큐 q 및 visited 배열 생성
시작 노드 'st'를 큐 q에 삽입
노드 'st'를 방문한 것으로 표시(1 or True)

큐 q가 비어 있지 않은 동안 다음을 반복:
- 큐의 맨 앞에서 요소를 꺼내 'now'에 저장.
- 큐의 맨 앞의 요소를 제거.

'now'의 값을 출력하고 뒤에 공백을 붙임.

노드 'now'의 인접 리스트 v에서 각 이웃 'next'에 대해:
만약 'next'가 아직 방문하지 않은 노드인 경우 :
- 노드 'next'를 방문한 것으로 표시.
- 'next'를 큐 q에 넣음.

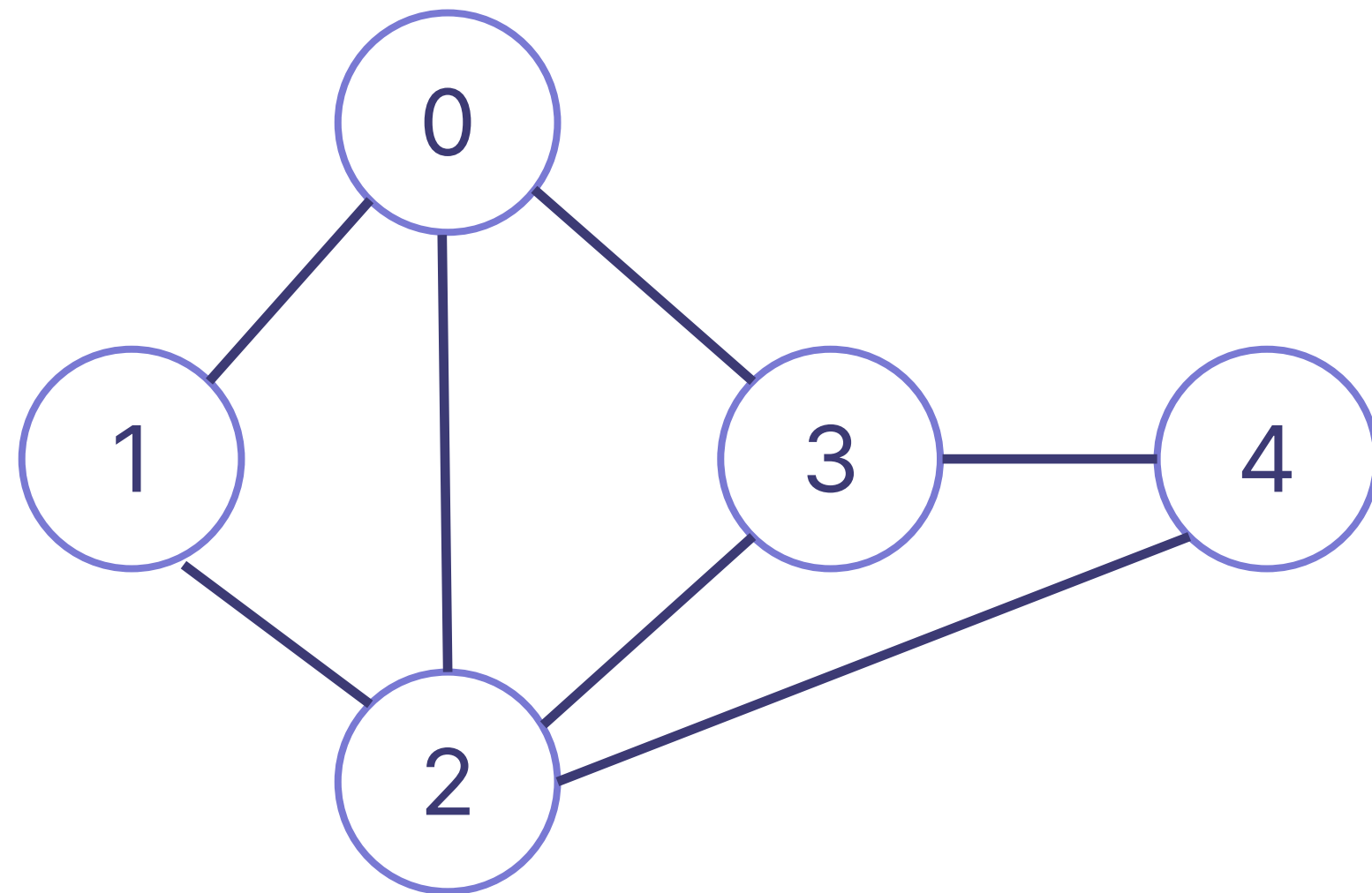
```
from collections import deque

q = deque()
q.append(st)
visited[st] = 1

while q:
    now = q.popleft()
    print(now, end=" ")
    for next in v[now]:
        if not visited[next]:
            visited[next] = 1
            q.append(next)
```

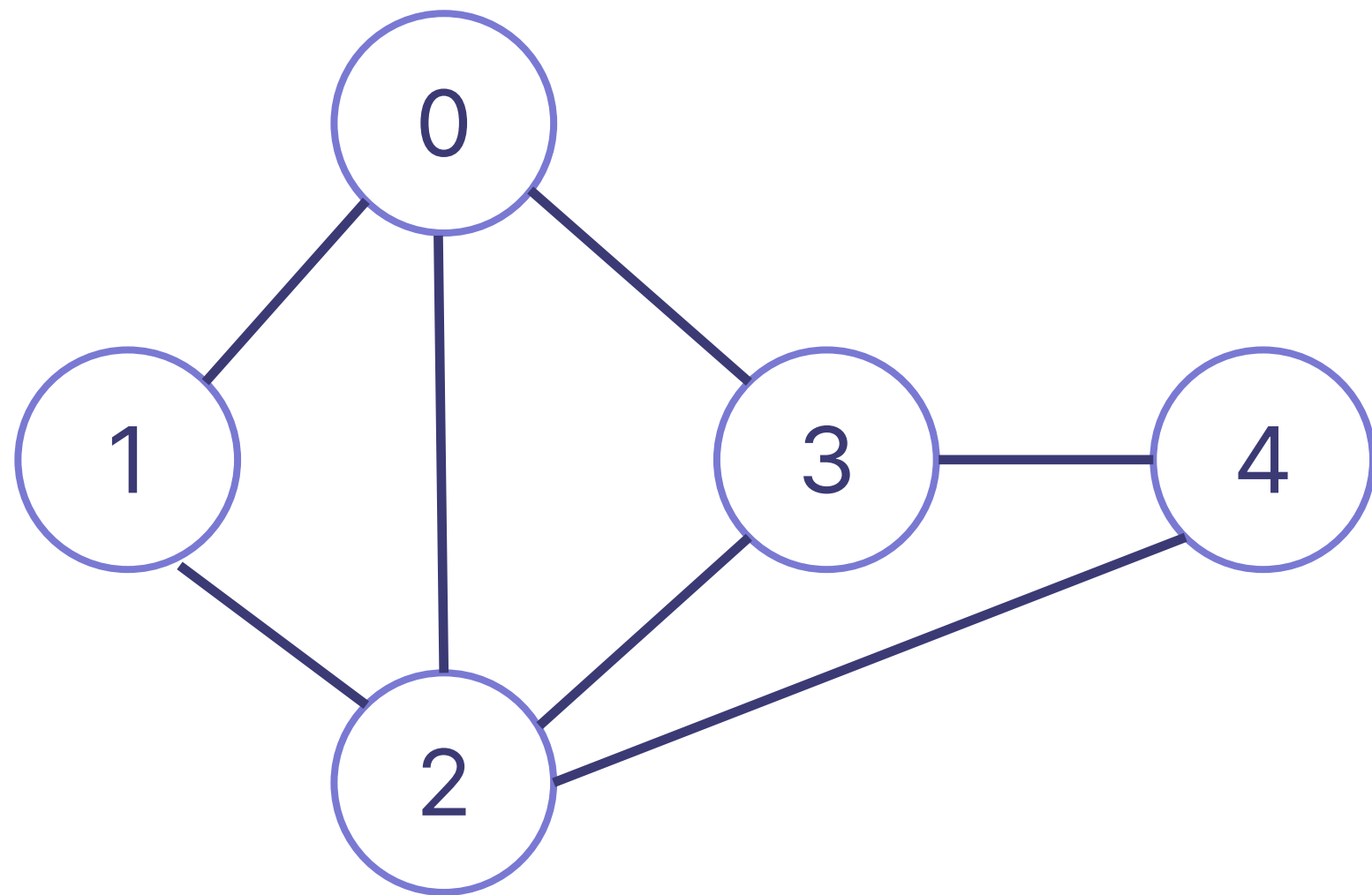
너비우선탐색 (BFS, Breadth-First Search)

그래프를 탐색하는 기법 중 하나로, **부모를 공유하는 인접 노드**들을 우선적으로 탐색합니다.



너비우선탐색 (BFS, Breadth-First Search)

그래프를 탐색하는 기법 중 하나로, **부모를 공유하는 인접 노드**들을 우선적으로 탐색합니다.

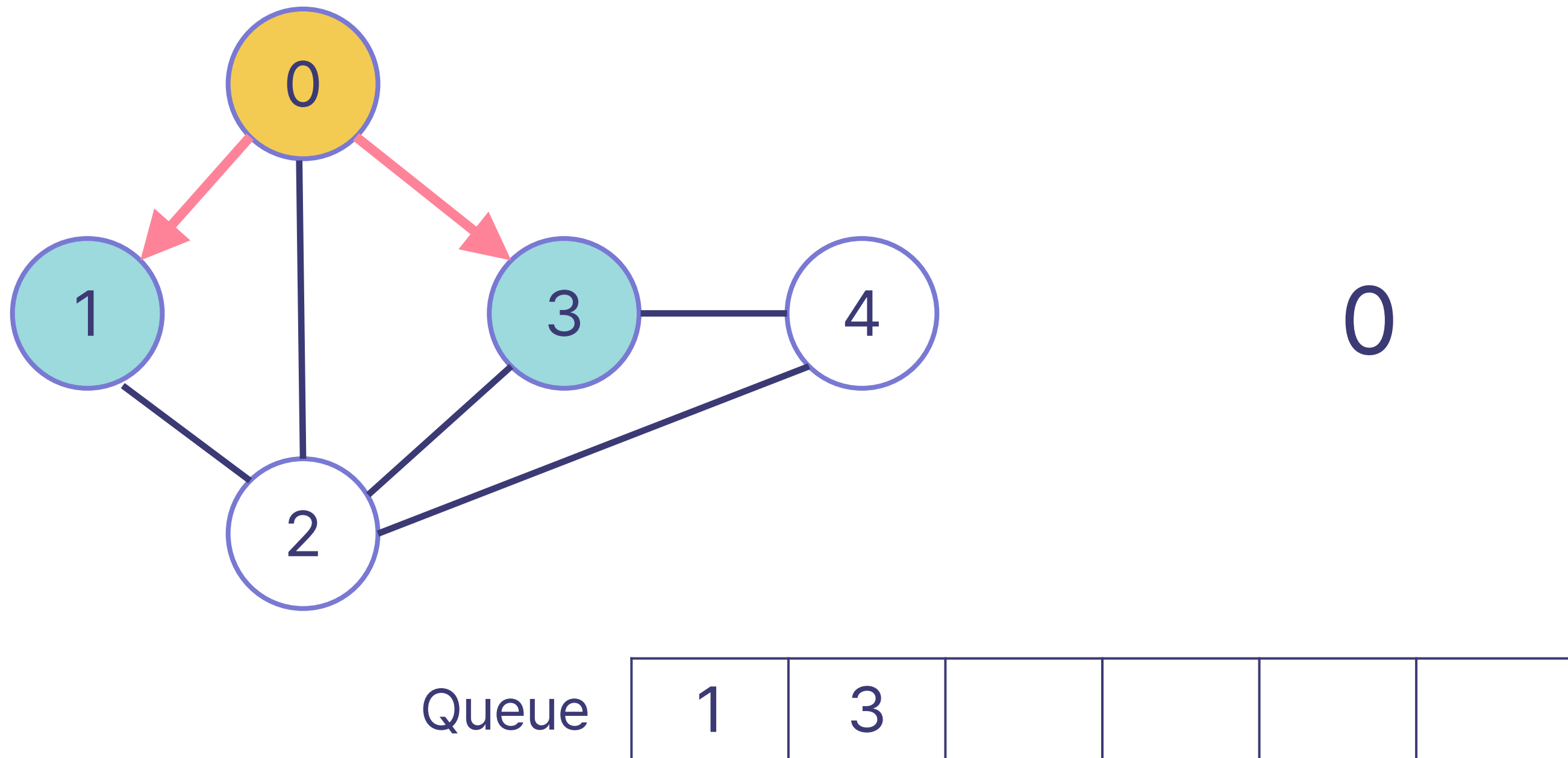


Queue



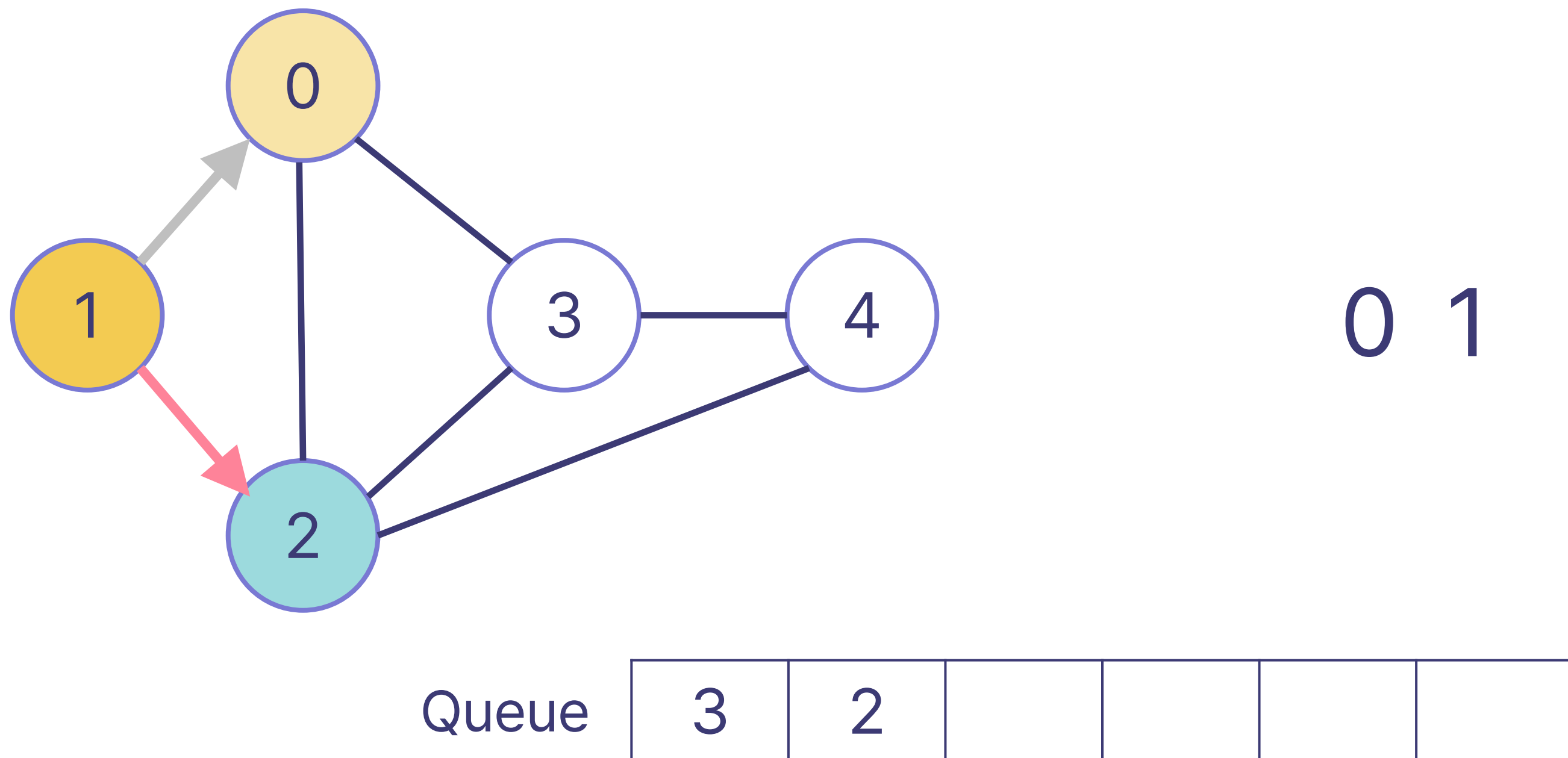
너비우선탐색 (BFS, Breadth-First Search)

그래프를 탐색하는 기법 중 하나로, **부모를 공유하는 인접 노드**들을 우선적으로 탐색합니다.



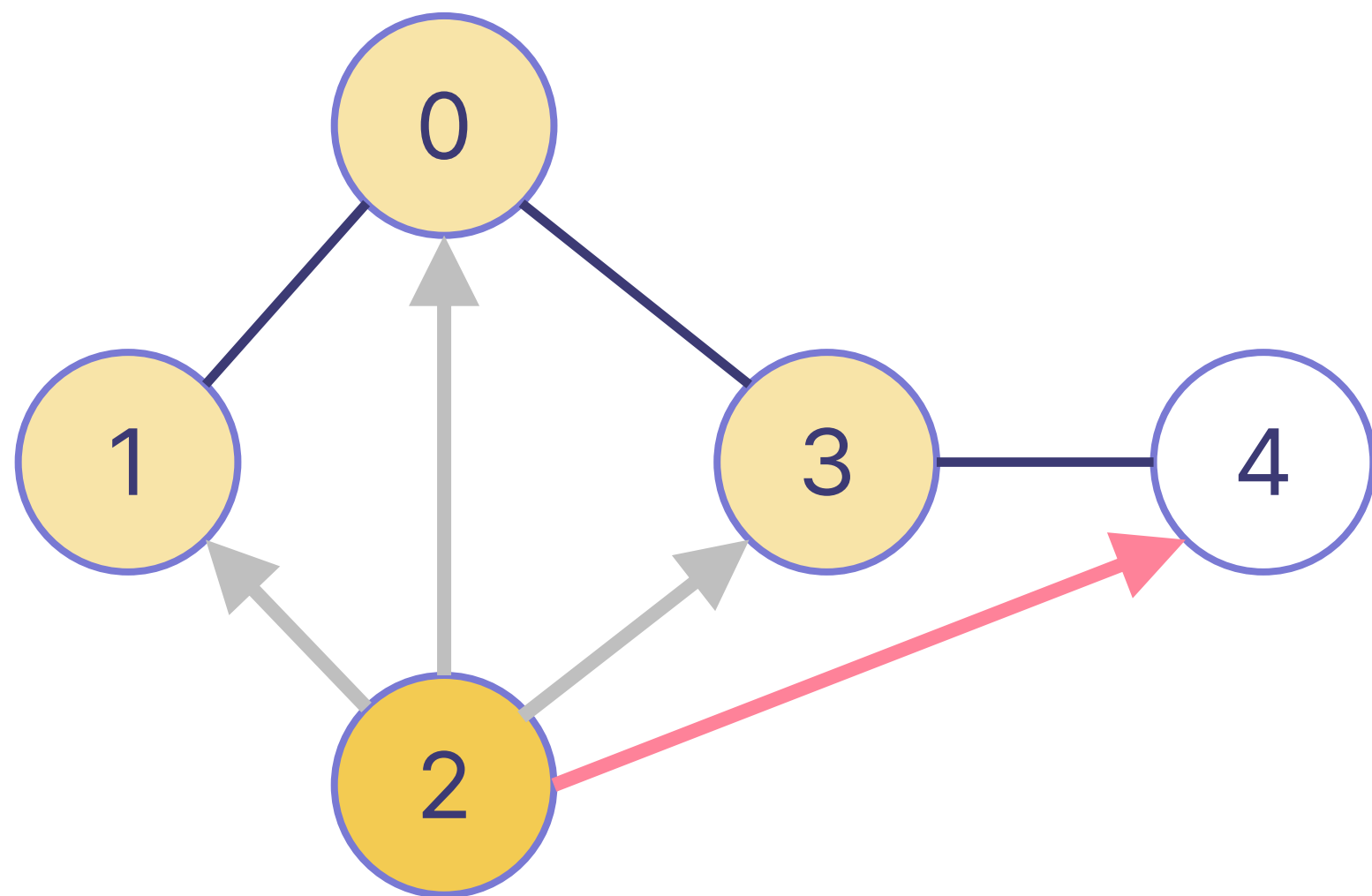
너비우선탐색 (BFS, Breadth-First Search)

그래프를 탐색하는 기법 중 하나로, **부모를 공유하는 인접 노드**들을 우선적으로 탐색합니다.



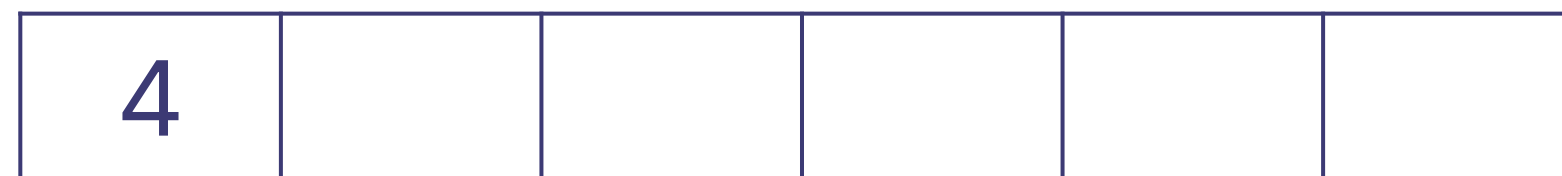
너비우선탐색 (BFS, Breadth-First Search)

그래프를 탐색하는 기법 중 하나로, **부모를 공유하는 인접 노드**들을 우선적으로 탐색합니다.



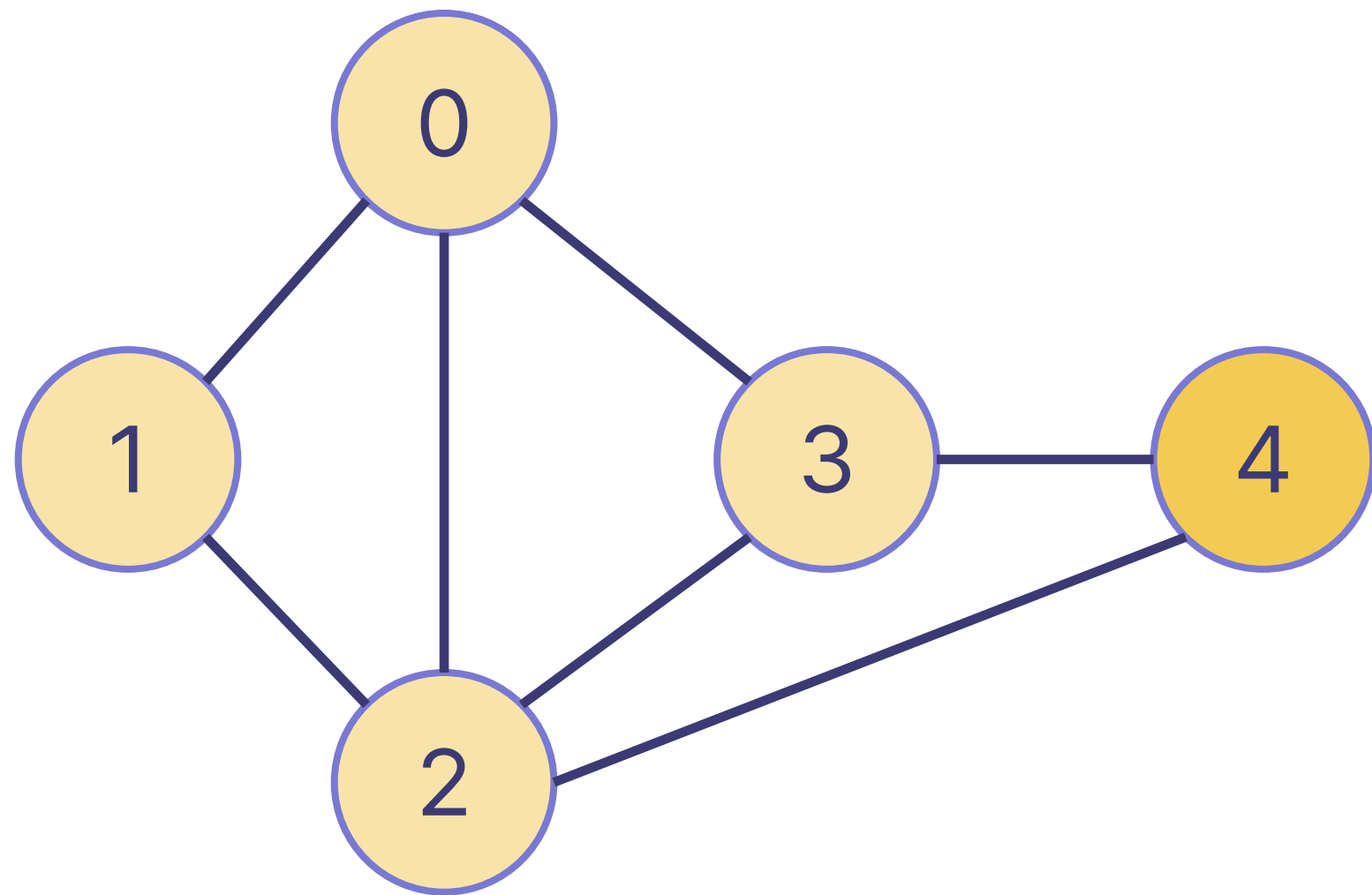
0 1 3 2

Queue



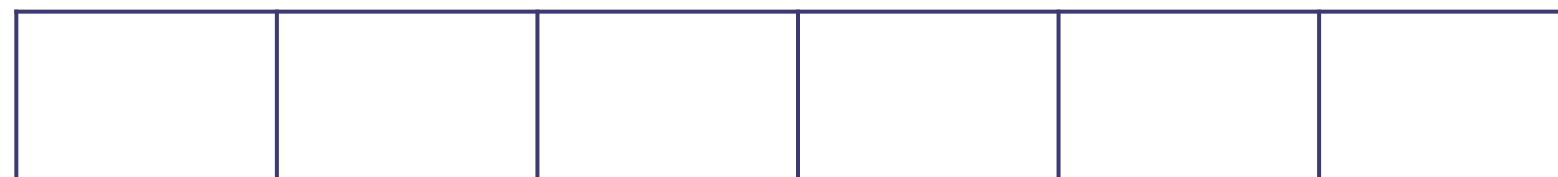
너비우선탐색 (BFS, Breadth-First Search)

그래프를 탐색하는 기법 중 하나로, **부모를 공유하는 인접 노드**들을 우선적으로 탐색합니다.



0 1 3 2 4

Queue





BFS의 시간 복잡도

- 인접 리스트로 표현된 그래프: $O(V + E)$
- 인접 행렬로 표현된 그래프: $O(V^2)$
- $V \rightarrow$ 정점(노드)의 수, $E \rightarrow$ 간선의 수

두 방식 모두 조건 내의 모든 노드를 검색한다는 점에서 시간 복잡도는 동일하지만 일반적으로 DFS를 재귀 함수로 구현한다는 점에서 **DFS보다 BFS가 조금 더 빠르게 동작**

주어진 그래프의 구조와 시작 노드에 따라서 실제 시간 복잡도가 다를 수 있으며,
어떤 알고리즘이 더 효율적인지는 그래프의 형태와 알고리즘의 목적에 따라 달라짐
일반적으로 어떤 알고리즘을 선택할지는 문제의 특성과 요구 사항에 따라 결정



DFS와 BFS의 공통점 및 차이점

• 공통점

- 그래프에서 시작 노드로부터 목적지 노드까지 도달하거나 특정 정보를 찾는 것이 목표
- 방문 기록을 체크함으로써, 이미 방문한 노드를 다시 방문하지 않게 하여 무한 루프 방지

• 차이점

- DFS는 주로 재귀로 구현하지만, BFS는 큐(queue) 자료구조를 활용하여 구현
- 동작 순서 상 DFS는 트리를 탐색할 때 자주 사용, BFS는 최단 경로 탐색에 자주 사용

그래프 예제



[예제] 디지털 세계 토지조사

네모 모양 행성에 **1로 되어있는 육지**와 **0으로 되어있는 바다**가 있습니다.
육지의 좌,우,상,하가 모두 바다라면 섬이라고 합니다. 섬의 개수를 구하세요.

0	1	1	0	0	1	0
1	1	0	0	1	1	1
0	1	0	1	0	1	0
0	0	0	1	0	0	0
1	1	0	1	0	0	0
0	0	0	0	0	1	1
0	0	0	0	0	1	1

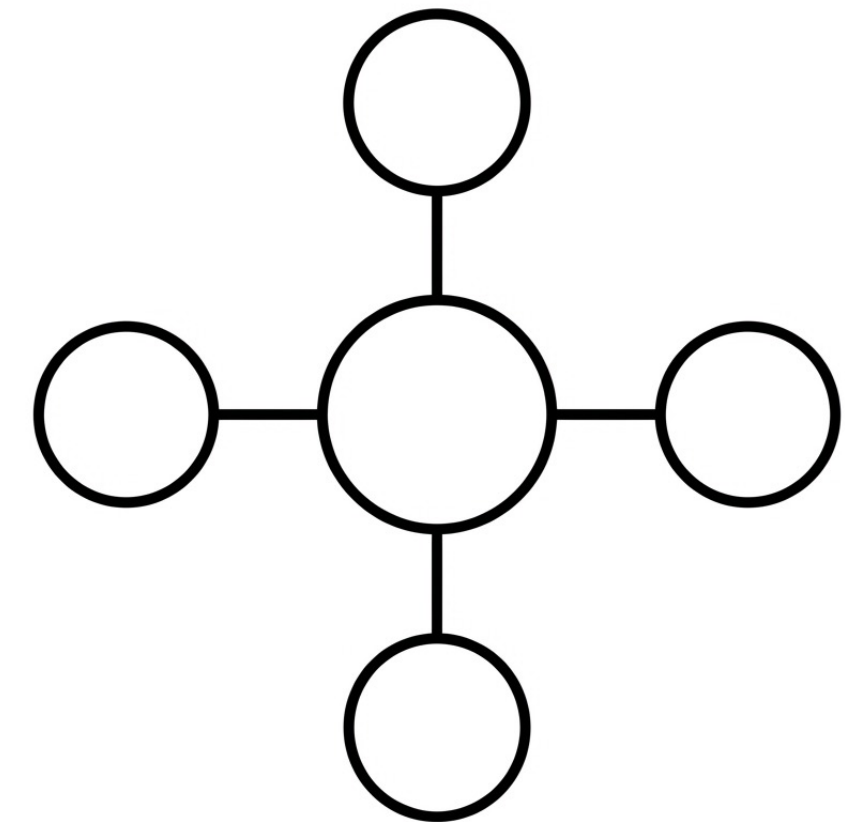


[예제] 디지털 세계 토지조사

네모 모양 행성에 **1로 되어있는 육지**와 **0으로 되어있는 바다**가 있습니다.
육지의 좌,우,상,하가 모두 바다라면 섬이라고 합니다. 섬의 개수를 구하세요.

0	1	1	0	0	1	0
1	1	0	0	1	1	1
0	1	0	1	0	1	0
0	0	0	1	0	0	0
1	1	0	1	0	0	0
0	0	0	0	0	1	1
0	0	0	0	0	1	1

0	1	0
1	1	1
0	1	0

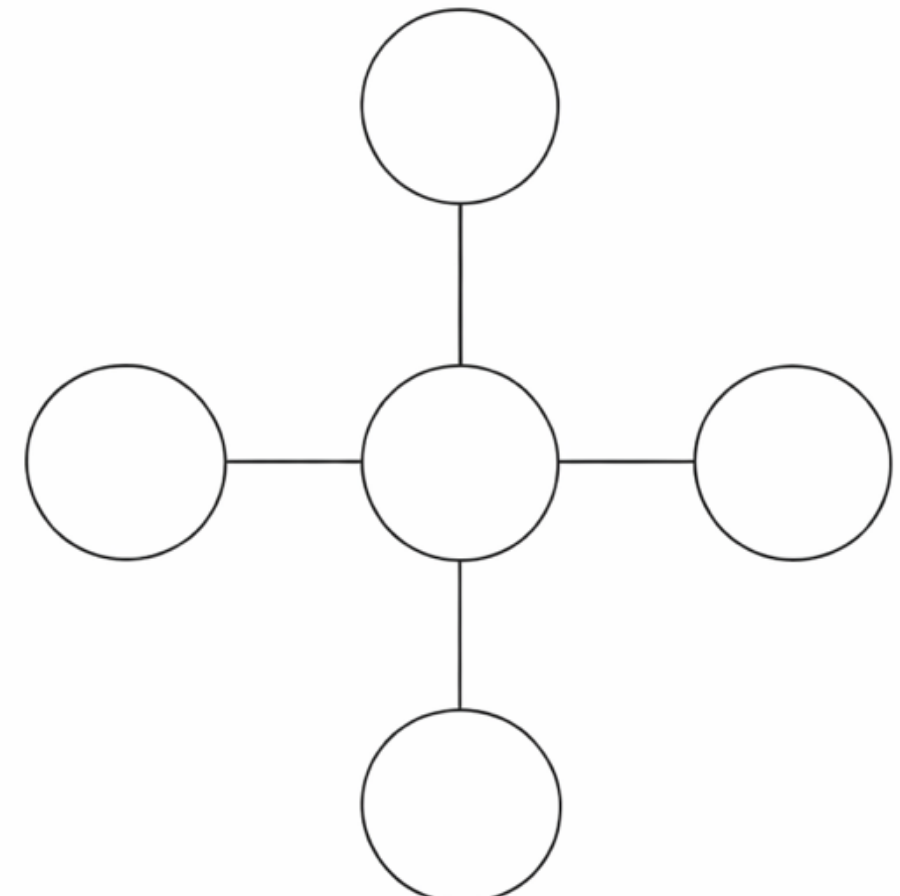
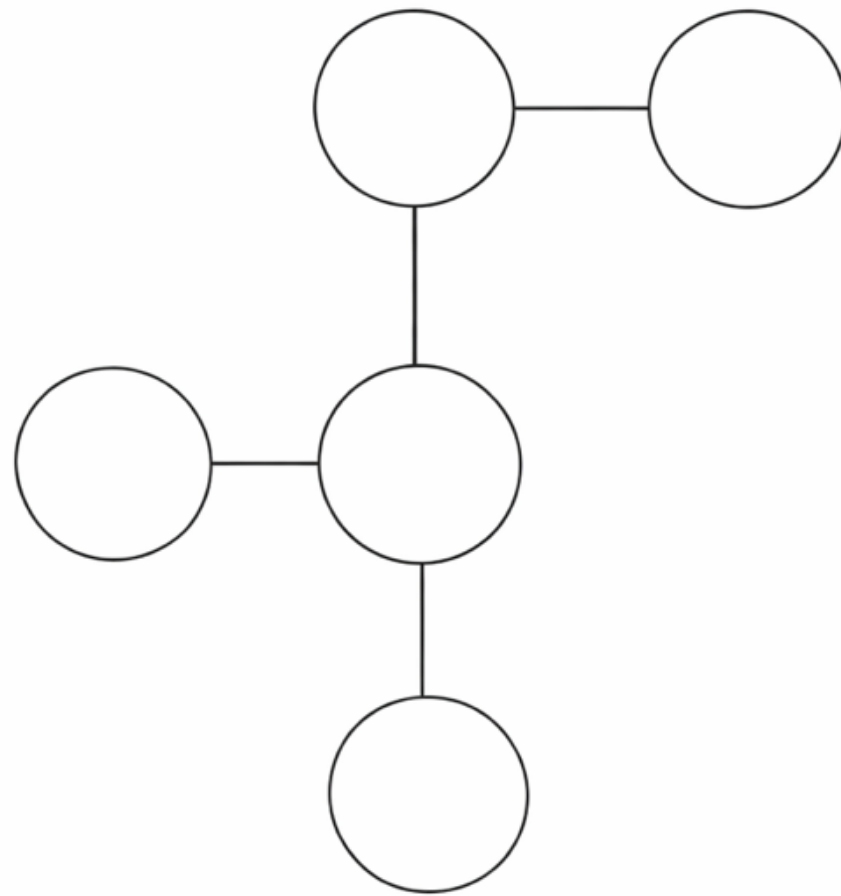




[예제] 디지털 세계 토지조사

네모 모양 행성에 **1로 되어있는 육지**와 **0으로 되어있는 바다**가 있습니다.
육지의 좌,우,상,하가 모두 바다라면 섬이라고 합니다. 섬의 개수를 구하세요.

0	1	1	0	0	1	0
1	1	0	0	1	1	1
0	1	0	1	0	1	0
0	0	0	1	0	0	0
1	1	0	1	0	0	0
0	0	0	0	0	1	1
0	0	0	0	0	1	1





[예제] 디지털 세계 토지조사

네모 모양 행성에 **1로 되어있는 육지**와 **0으로 되어있는 바다**가 있습니다.
육지의 좌,우,상,하가 모두 바다라면 섬이라고 합니다. 섬의 개수를 구하세요.

0	1	1	0	0	1	0
1	1	0	0	1	1	1
0	1	0	1	0	1	0
0	0	0	1	0	0	0
1	1	0	1	0	0	0
0	0	0	0	0	1	1
0	0	0	0	0	1	1



[예제] 디지털 세계 토지조사

네모 모양 행성에 **1로 되어있는 육지**와 **0으로 되어있는 바다**가 있습니다.
육지의 좌,우,상,하가 모두 바다라면 섬이라고 합니다. 섬의 개수를 구하세요.

0	1	1	0	0	1	0
1	1	0	0	1	1	1
0	1	0	1	0	1	0
0	0	0	1	0	0	0
1	1	0	1	0	0	0
0	0	0	0	0	1	1
0	0	0	0	0	1	1



[예제] 디지털 세계 토지조사

네모 모양 행성에 **1로 되어있는 육지**와 **0으로 되어있는 바다**가 있습니다.
육지의 좌,우,상,하가 모두 바다라면 섬이라고 합니다. 섬의 개수를 구하세요.

0	1	1	0	0	1	0
1	1	0	0	1	1	1
0	1	0	1	0	1	0
0	0	0	1	0	0	0
1	1	0	1	0	0	0
0	0	0	0	0	1	1
0	0	0	0	0	1	1



[예제] 디지털 세계 토지조사

네모 모양 행성에 **1로 되어있는 육지**와 **0으로 되어있는 바다**가 있습니다.
육지의 좌,우,상,하가 모두 바다라면 섬이라고 합니다. 섬의 개수를 구하세요.

0	1	1	0	0	1	0
1	1	0	0	1	1	1
0	1	0	1	0	1	0
0	0	0	1	0	0	0
1	1	0	1	0	0	0
0	0	0	0	0	1	1
0	0	0	0	0	1	1



[예제] 디지털 세계 토지조사

네모 모양 행성에 **1로 되어있는 육지**와 **0으로 되어있는 바다**가 있습니다.
육지의 좌,우,상,하가 모두 바다라면 섬이라고 합니다. 섬의 개수를 구하세요.

0	1	1	0	0	1	0
1	1	0	0	1	1	1
0	1	0	1	0	1	0
0	0	0	1	0	0	0
1	1	0	1	0	0	0
0	0	0	0	0	1	1
0	0	0	0	0	1	1



[예제] 디지털 세계 토지조사

네모 모양 행성에 **1로 되어있는 육지**와 **0으로 되어있는 바다**가 있습니다.
육지의 좌,우,상,하가 모두 바다라면 섬이라고 합니다. 섬의 개수를 구하세요.

0	1	1	0	0	1	0
1	1	0	0	1	1	1
0	1	0	1	0	1	0
0	0	0	1	0	0	0
1	1	0	1	0	0	0
0	0	0	0	0	1	1
0	0	0	0	0	1	1



[예제] 디지털 세계 토지조사

네모 모양 행성에 **1로 되어있는 육지**와 **0으로 되어있는 바다**가 있습니다.
육지의 좌,우,상,하가 모두 바다라면 섬이라고 합니다. 섬의 개수를 구하세요.

0	1	1	0	0	1	0
1	1	0	0	1	1	1
0	1	0	1	0	1	0
0	0	0	1	0	0	0
1	1	0	1	0	0	0
0	0	0	0	0	1	1
0	0	0	0	0	1	1



[예제] 디지털 세계 토지조사

네모 모양 행성에 **1로 되어있는 육지**와 **0으로 되어있는 바다**가 있습니다.
육지의 좌,우,상,하가 모두 바다라면 섬이라고 합니다. 섬의 개수를 구하세요.

0	1	1	0	0	1	0
1	1	0	0	1	1	1
0	1	0	1	0	1	0
0	0	0	1	0	0	0
1	1	0	1	0	0	0
0	0	0	0	0	1	1
0	0	0	0	0	1	1



[예제] 디지털 세계 토지조사

네모 모양 행성에 **1로 되어있는 육지**와 **0으로 되어있는 바다**가 있습니다.
육지의 좌,우,상,하가 모두 바다라면 섬이라고 합니다. 섬의 개수를 구하세요.

0	1	1	0	0	1	0
1	1	0	0	1	1	1
0	1	0	1	0	1	0
0	0	0	1	0	0	0
1	1	0	1	0	0	0
0	0	0	0	0	1	1
0	0	0	0	0	1	1



[예제] 디지털 세계 토지조사

네모 모양 행성에 **1로 되어있는 육지**와 **0으로 되어있는 바다**가 있습니다.
육지의 좌,우,상,하가 모두 바다라면 섬이라고 합니다. 섬의 개수를 구하세요.

0	1	1	0	0	1	0
1	1	0	0	1	1	1
0	1	0	1	0	1	0
0	0	0	1	0	0	0
1	1	0	1	0	0	0
0	0	0	0	0	1	1
0	0	0	0	0	1	1



[예제] 디지털 세계 토지조사

네모 모양 행성에 **1로 되어있는 육지**와 **0으로 되어있는 바다**가 있습니다.
육지의 좌,우,상,하가 모두 바다라면 섬이라고 합니다. 섬의 개수를 구하세요.

0	1	1	0	0	1	0
1	1	0	0	1	1	1
0	1	0	1	0	1	0
0	0	0	1	0	0	0
1	1	0	1	0	0	0
0	0	0	0	0	1	1
0	0	0	0	0	1	1



[예제] 디지털 세계 토지조사

네모 모양 행성에 **1로 되어있는 육지**와 **0으로 되어있는 바다**가 있습니다.
육지의 좌,우,상,하가 모두 바다라면 섬이라고 합니다. 섬의 개수를 구하세요.

0	1	1	0	0	1	0
1	1	0	0	1	1	1
0	1	0	1	0	1	0
0	0	0	1	0	0	0
1	1	0	1	0	0	0
0	0	0	0	0	1	1
0	0	0	0	0	1	1



[예제] 디지털 세계 토지조사

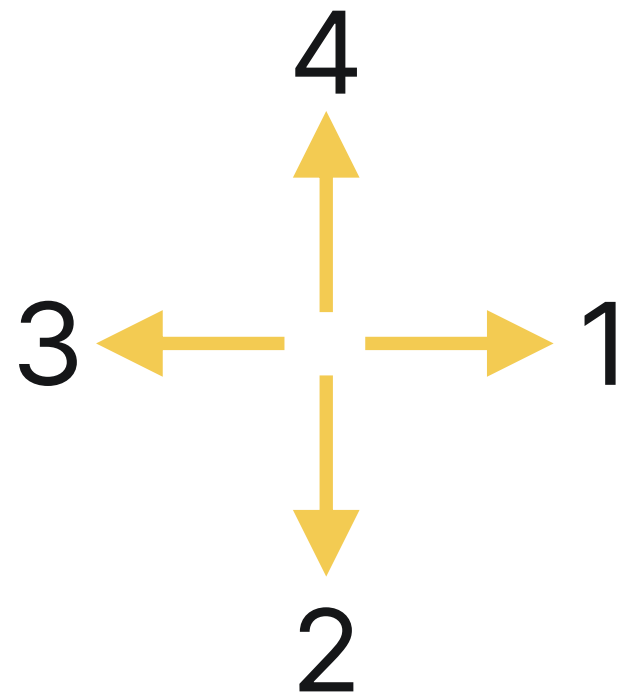
네모 모양 행성에 **1로 되어있는 육지**와 **0으로 되어있는 바다**가 있습니다.
육지의 좌,우,상,하가 모두 바다라면 섬이라고 합니다. 섬의 개수를 구하세요.

0	1	1	0	0	1	0
1	1	0	0	1	1	1
0	1	0	1	0	1	0
0	0	0	1	0	0	0
1	1	0	1	0	0	0
0	0	0	0	0	1	1
0	0	0	0	0	1	1



[예제] 디지털 세계 토지조사

네모 모양 행성에 **1로 되어있는 육지**와 **0으로 되어있는 바다**가 있습니다.
육지의 좌,우,상,하가 모두 바다라면 섬이라고 합니다. 섬의 개수를 구하세요.

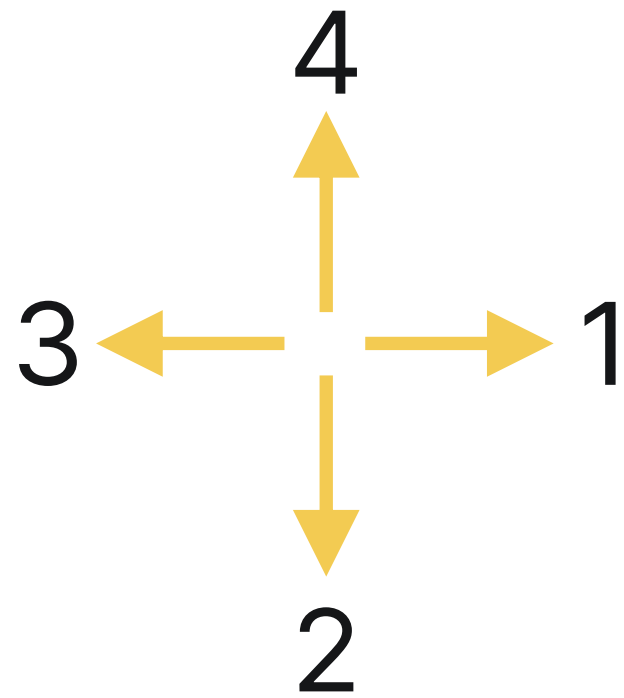


0	1	1	0	0	1	0
1	1	0	0	1	1	1
0	1	0	1	0	1	0
0	0	0	1	0	0	0
1	1	0	1	0	0	0
0	0	0	0	0	1	1
0	0	0	0	0	1	1



[예제] 디지털 세계 토지조사

네모 모양 행성에 **1로 되어있는 육지**와 **0으로 되어있는 바다**가 있습니다.
육지의 좌,우,상,하가 모두 바다라면 섬이라고 합니다. 섬의 개수를 구하세요.

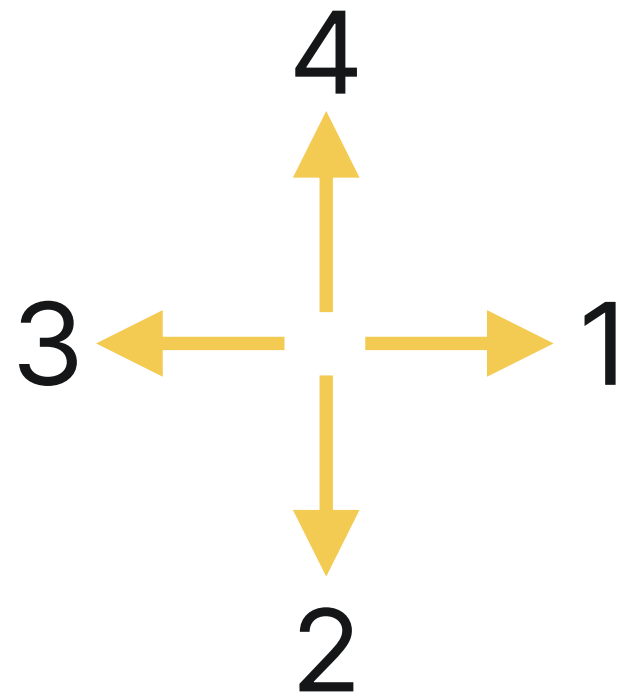


0	1	1	0	0	1	0
1	1	0	0	1	1	1
0	1	0	1	0	1	0
0	0	0	1	0	0	0
1	1	0	1	0	0	0
0	0	0	0	0	1	1
0	0	0	0	0	1	1



[예제] 디지털 세계 토지조사

네모 모양 행성에 **1로 되어있는 육지**와 **0으로 되어있는 바다**가 있습니다.
육지의 좌,우,상,하가 모두 바다라면 섬이라고 합니다. 섬의 개수를 구하세요.

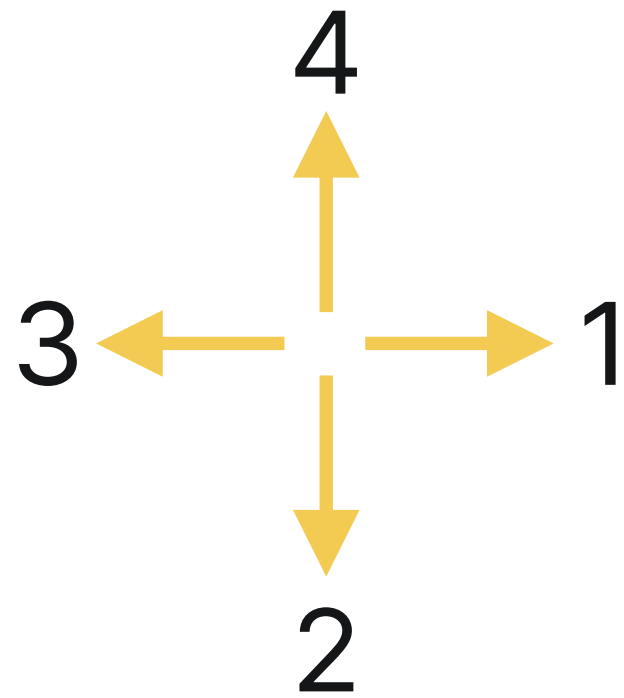


0	1	1	0	0	1	0
1	1	0	0	1	1	1
0	1	0	1	0	1	0
0	0	0	1	0	0	0
1	1	0	1	0	0	0
0	0	0	0	0	1	1
0	0	0	0	0	1	1



[예제] 디지털 세계 토지조사

네모 모양 행성에 **1로 되어있는 육지**와 **0으로 되어있는 바다**가 있습니다.
육지의 좌,우,상,하가 모두 바다라면 섬이라고 합니다. 섬의 개수를 구하세요.

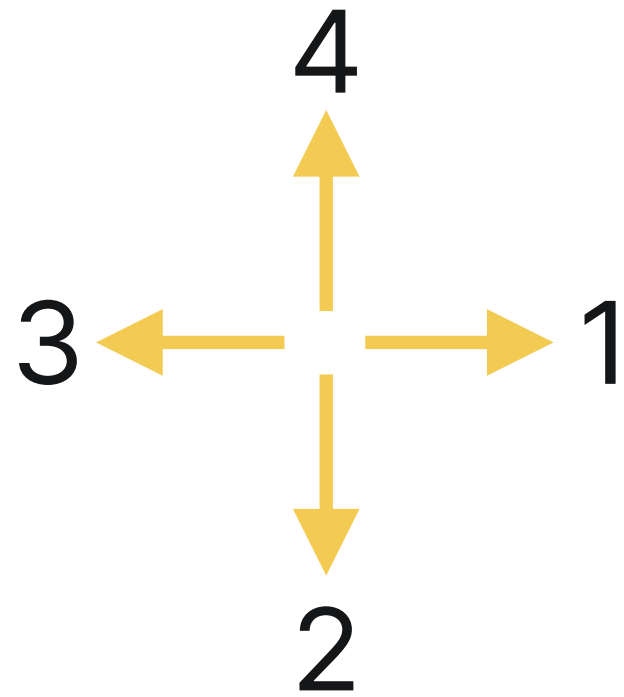


0	1	1	0	0	1	0
1	1	0	0	1	1	1
0	1	0	1	0	1	0
0	0	0	1	0	0	0
1	1	0	1	0	0	0
0	0	0	0	0	1	1
0	0	0	0	0	1	1



[예제] 디지털 세계 토지조사

네모 모양 행성에 **1로 되어있는 육지**와 **0으로 되어있는 바다**가 있습니다.
육지의 좌,우,상,하가 모두 바다라면 섬이라고 합니다. 섬의 개수를 구하세요.

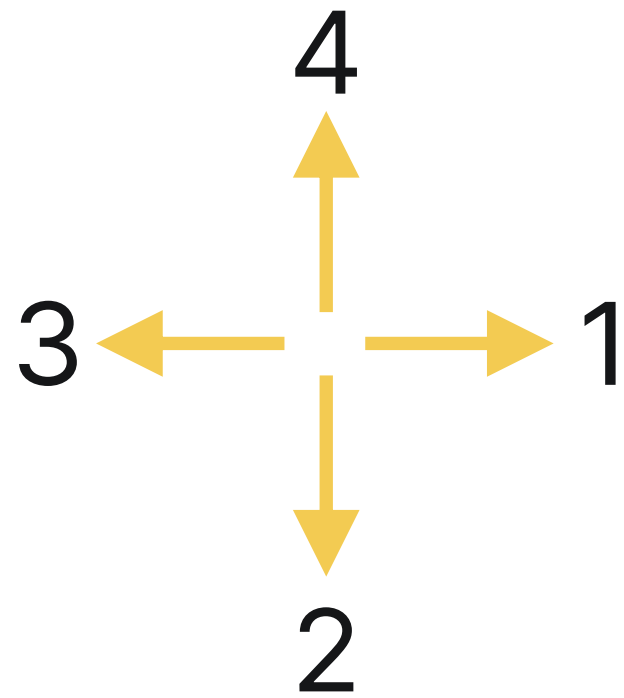


0	1	1	0	0	1	0
1	1	0	0	1	1	1
0	1	0	1	0	1	0
0	0	0	1	0	0	0
1	1	0	1	0	0	0
0	0	0	0	0	1	1
0	0	0	0	0	1	1



[예제] 디지털 세계 토지조사

네모 모양 행성에 **1로 되어있는 육지**와 **0으로 되어있는 바다**가 있습니다.
육지의 좌,우,상,하가 모두 바다라면 섬이라고 합니다. 섬의 개수를 구하세요.

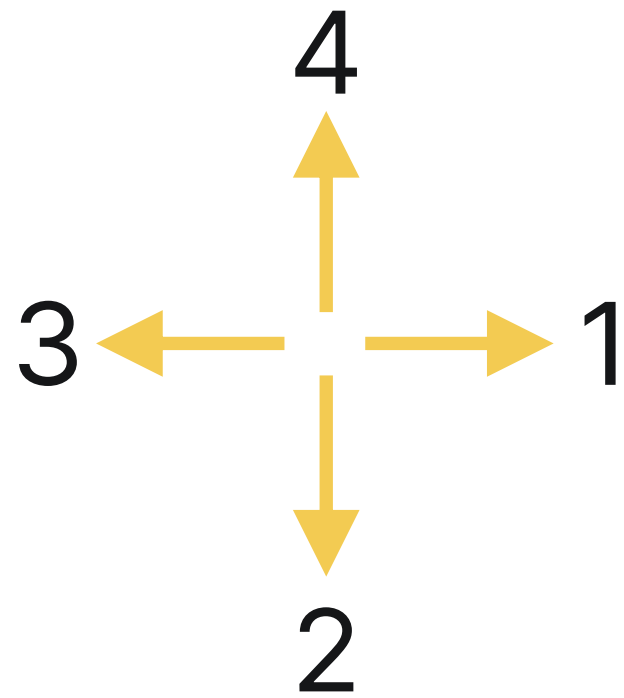


0	1	1	0	0	1	0
1	1	0	0	1	1	1
0	1	0	1	0	1	0
0	0	0	1	0	0	0
1	1	0	1	0	0	0
0	0	0	0	0	1	1
0	0	0	0	0	1	1



[예제] 디지털 세계 토지조사

네모 모양 행성에 **1로 되어있는 육지**와 **0으로 되어있는 바다**가 있습니다.
육지의 좌,우,상,하가 모두 바다라면 섬이라고 합니다. 섬의 개수를 구하세요.



0	1	1	0	0	1	0
1	1	0	0	1	1	1
0	1	0	1	0	1	0
0	0	0	1	0	0	0
1	1	0	1	0	0	0
0	0	0	0	0	1	1
0	0	0	0	0	1	1



[예제] 디지털 세계 토지조사

```
def count_island(h, w, maps):  
    cnt = 0  
    isize = []  
    for i in range(h):  
        for j in range(w):  
            if maps[i][j] != 0:  
                cnt += 1  
                size = 1  
                size = dfs(i, j)  
                isize.append(size)  
    return cnt, isize
```

0	1	1	0	0	1	0
1	1	0	0	1	1	1
0	1	0	1	0	1	0
0	0	0	1	0	0	0
1	1	0	1	0	0	0
0	0	0	0	0	1	1
0	0	0	0	0	1	1



[예제] 디지털 세계 토지조사

```
dir = [(0,1),(1,0),(0,-1),(-1,0)]

def dfs(x,y):
    global maps
    global size
    maps[x][y] = 0
```

```
for i in range(4):
    nx = x + dir[i][0]
    ny = y + dir[i][1]
    if 0 <= nx < h and 0 <= ny < w and maps[nx][ny] == 1:
        size += 1
        maps[nx][ny] = 0
        dfs(nx,ny)
return size
```



[예제] 촌수 계산하기

새로운 SNS 서비스 **엘리스친구**는 두 명의 사용자 간의 거리를 촌수로 표시합니다.
직접연결이 되어있는 친구라면 **1촌**, **N명의 친구를 거쳐** 연결되어있다면 **N+1촌**입니다.
두 사용자 간의 촌수를 구하세요.

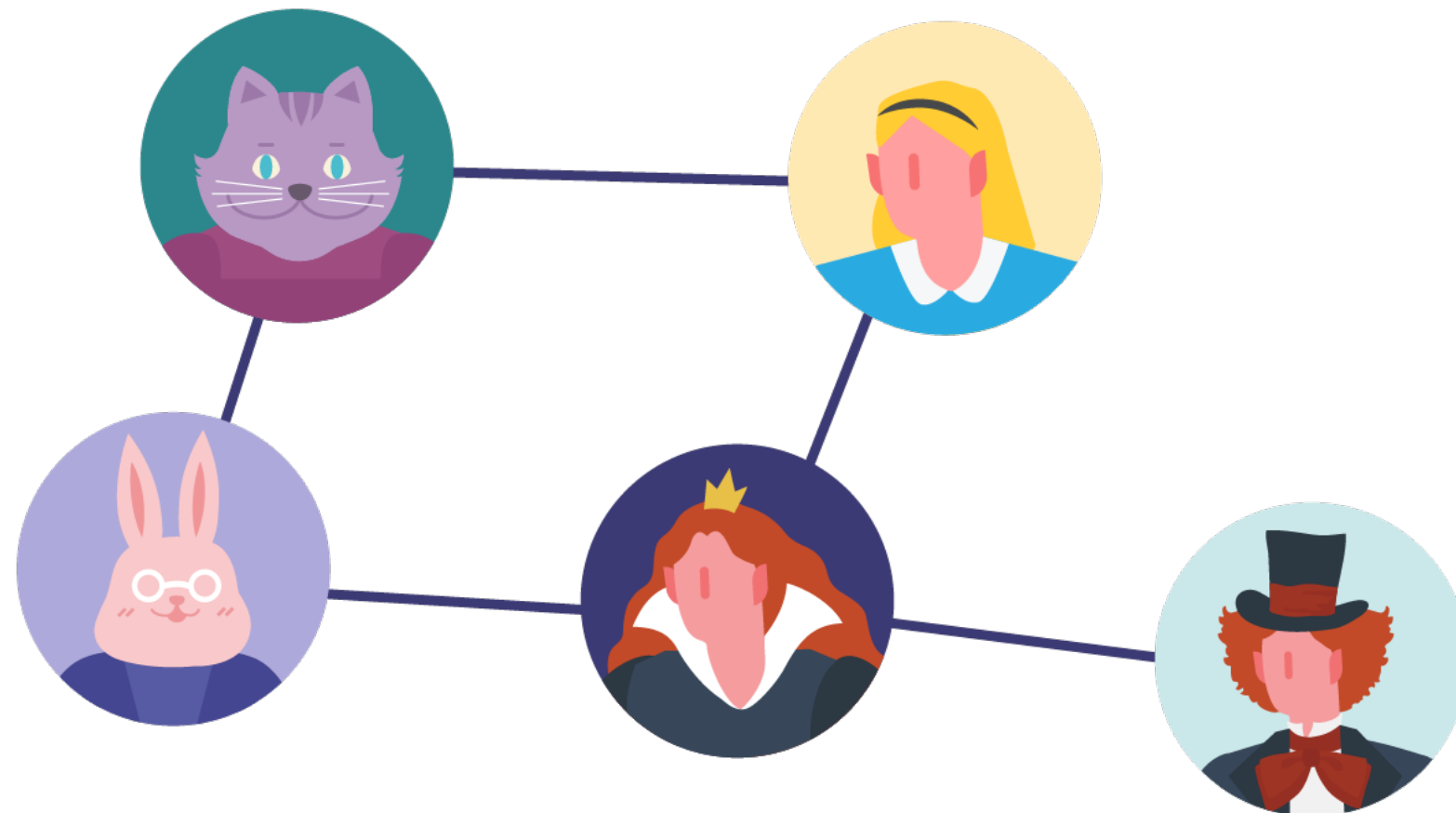
엘리스 - 체셔

체셔 - 토끼

엘리스 - 하트여왕

모자장수 - 하트여왕

토끼 - 하트여왕





[예제] 촌수 계산하기

새로운 SNS 서비스 **엘리스친구**는 두 명의 사용자 간의 거리를 촌수로 표시합니다.
직접연결이 되어있는 친구라면 **1촌**, **N명의 친구를 거쳐** 연결되어있다면 **N+1촌**입니다.
두 사용자 간의 촌수를 구하세요.

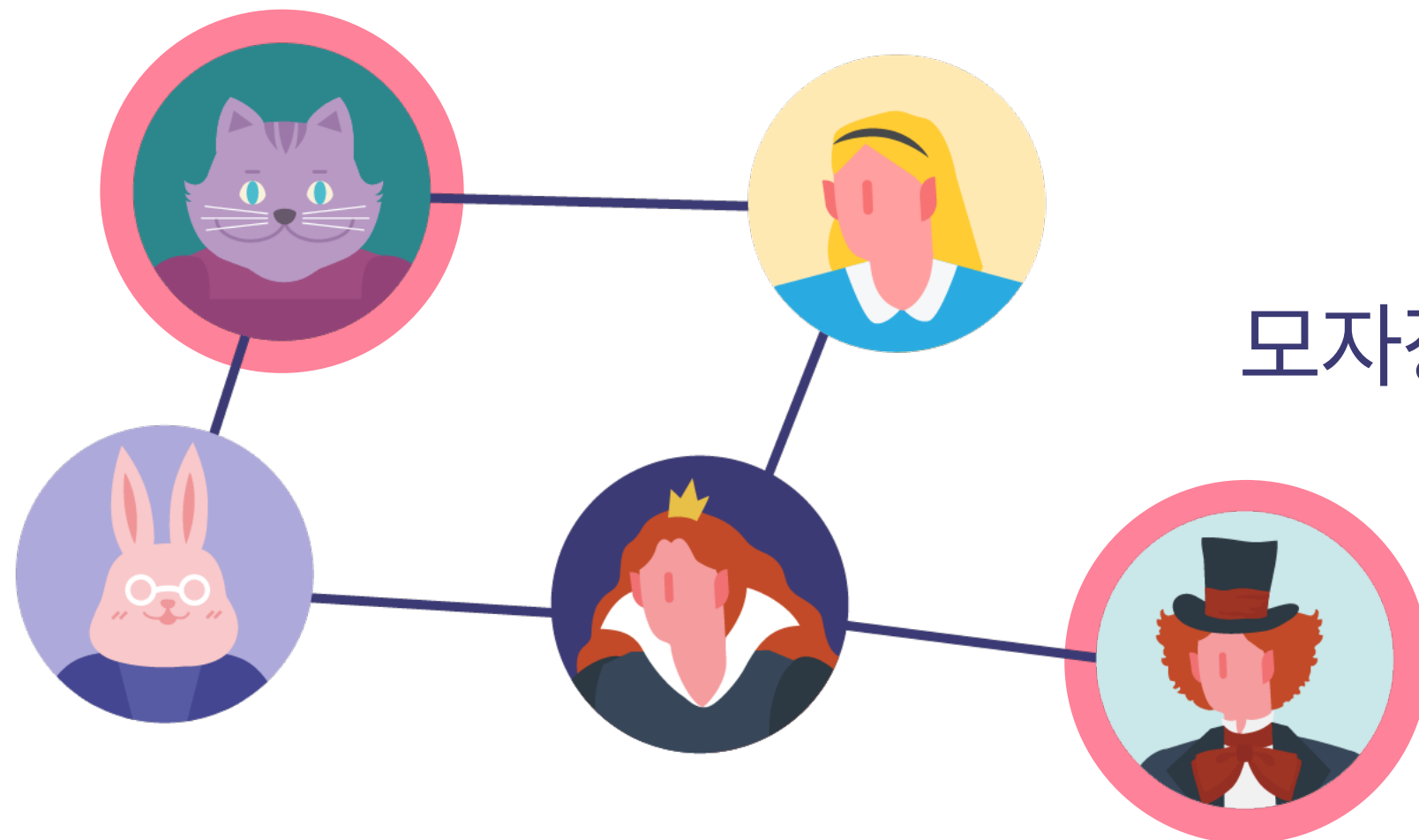
엘리스 - 체셔

체셔 - 토끼

엘리스 - 하트여왕

모자장수 - 하트여왕

토끼 - 하트여왕



모자장수 - 체셔?



[예제] 촌수 계산하기

새로운 SNS 서비스 **엘리스친구**는 두 명의 사용자 간의 거리를 촌수로 표시합니다.
직접연결이 되어있는 친구라면 **1촌**, **N명의 친구를 거쳐** 연결되어있다면 **N+1촌**입니다.
두 사용자 간의 촌수를 구하세요.

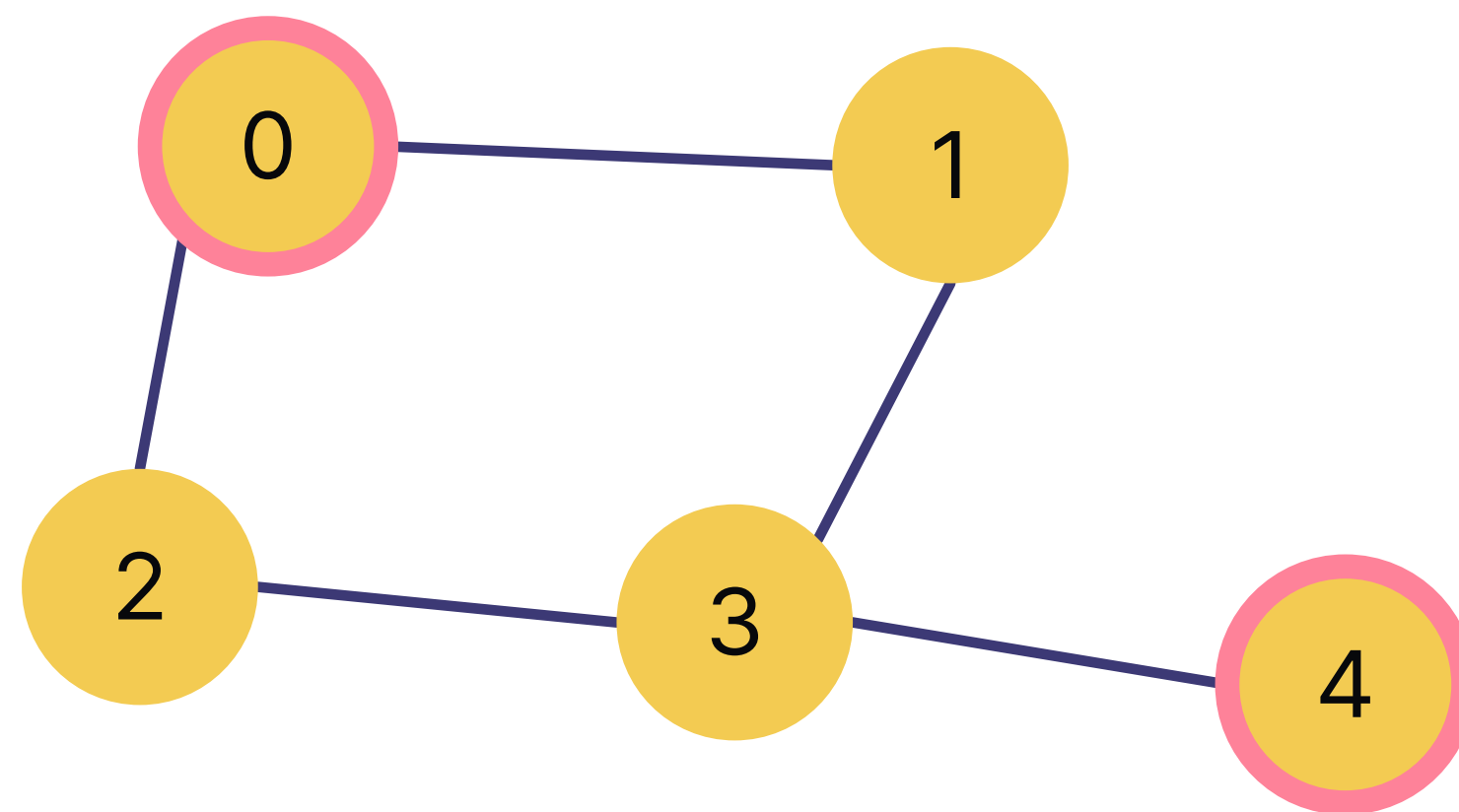
엘리스 - 체셔

체셔 - 토끼

엘리스 - 하트여왕

모자장수 - 하트여왕

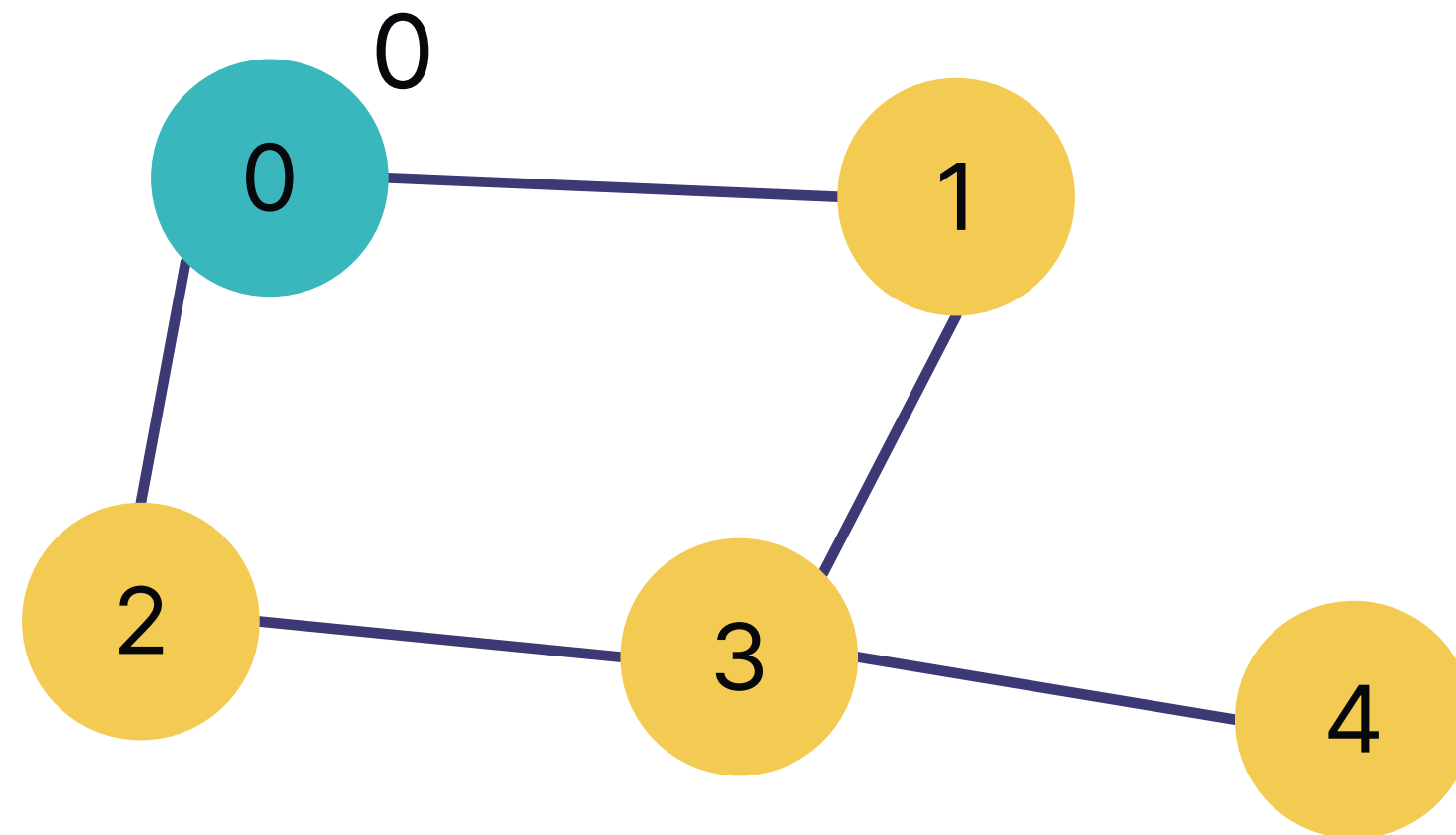
토끼 - 하트여왕





[예제] 촌수 계산하기

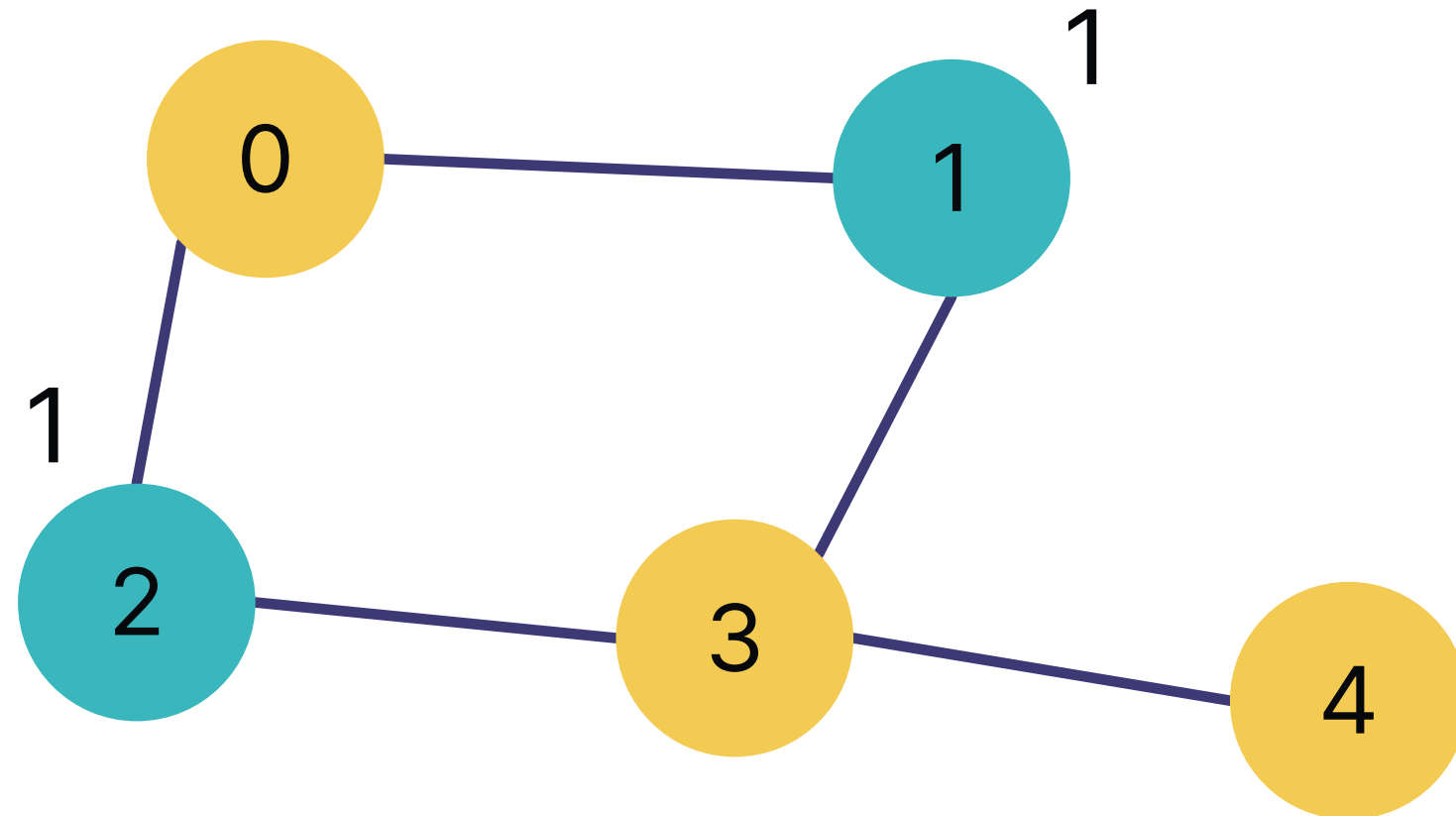
새로운 SNS 서비스 **엘리스친구**는 두 명의 사용자 간의 거리를 촌수로 표시합니다.
직접연결이 되어있는 친구라면 **1촌**, **N명의 친구를 거쳐** 연결되어있다면 **N+1촌**입니다.
두 사용자 간의 촌수를 구하세요.





[예제] 촌수 계산하기

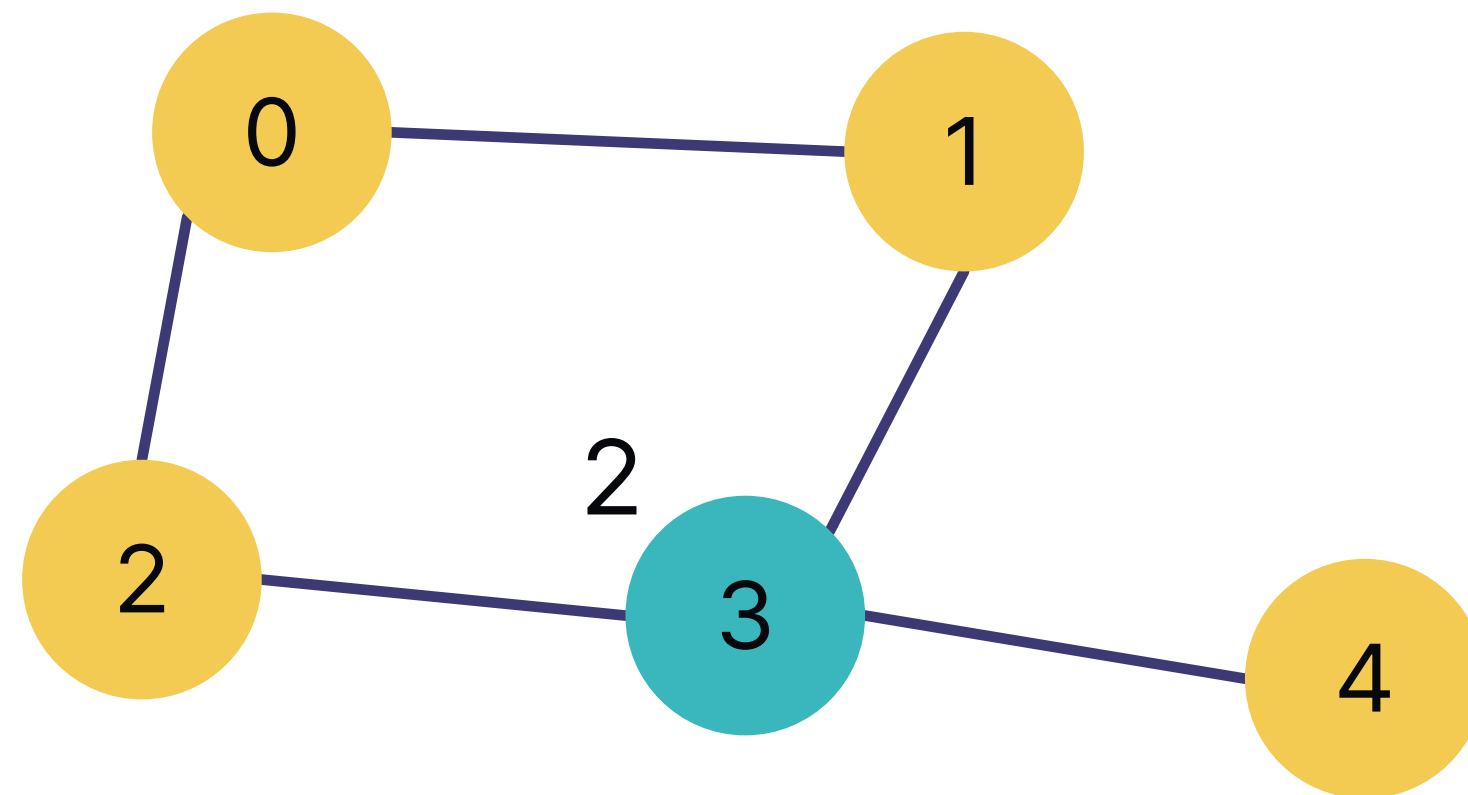
새로운 SNS 서비스 **엘리스친구**는 두 명의 사용자 간의 거리를 촌수로 표시합니다.
직접연결이 되어있는 친구라면 **1촌**, **N명의 친구를 거쳐** 연결되어있다면 **N+1촌**입니다.
두 사용자 간의 촌수를 구하세요.





[예제] 촌수 계산하기

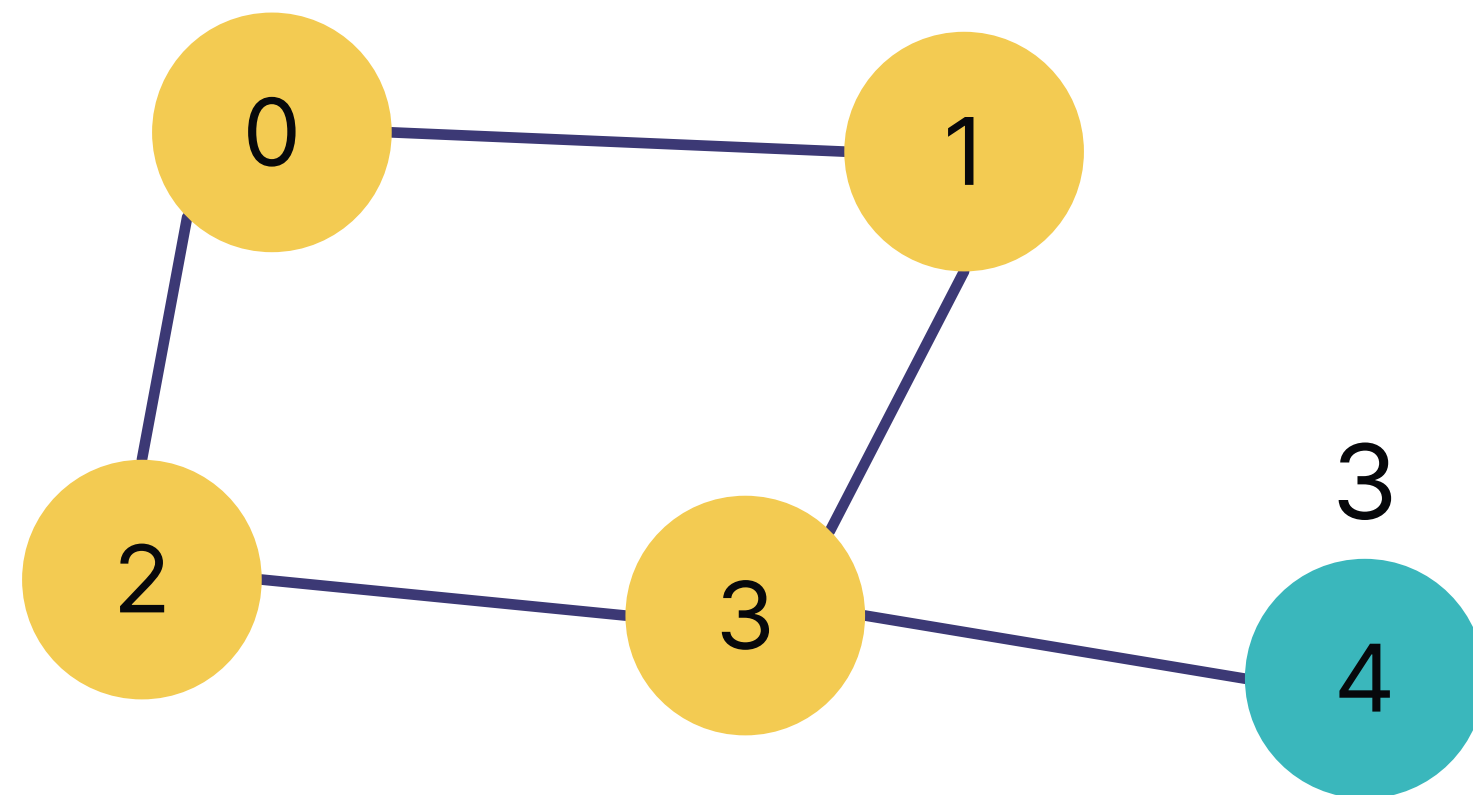
새로운 SNS 서비스 **엘리스친구**는 두 명의 사용자 간의 거리를 촌수로 표시합니다.
직접연결이 되어있는 친구라면 **1촌**, **N명의 친구를 거쳐** 연결되어있다면 **N+1촌**입니다.
두 사용자 간의 촌수를 구하세요.





[예제] 촌수 계산하기

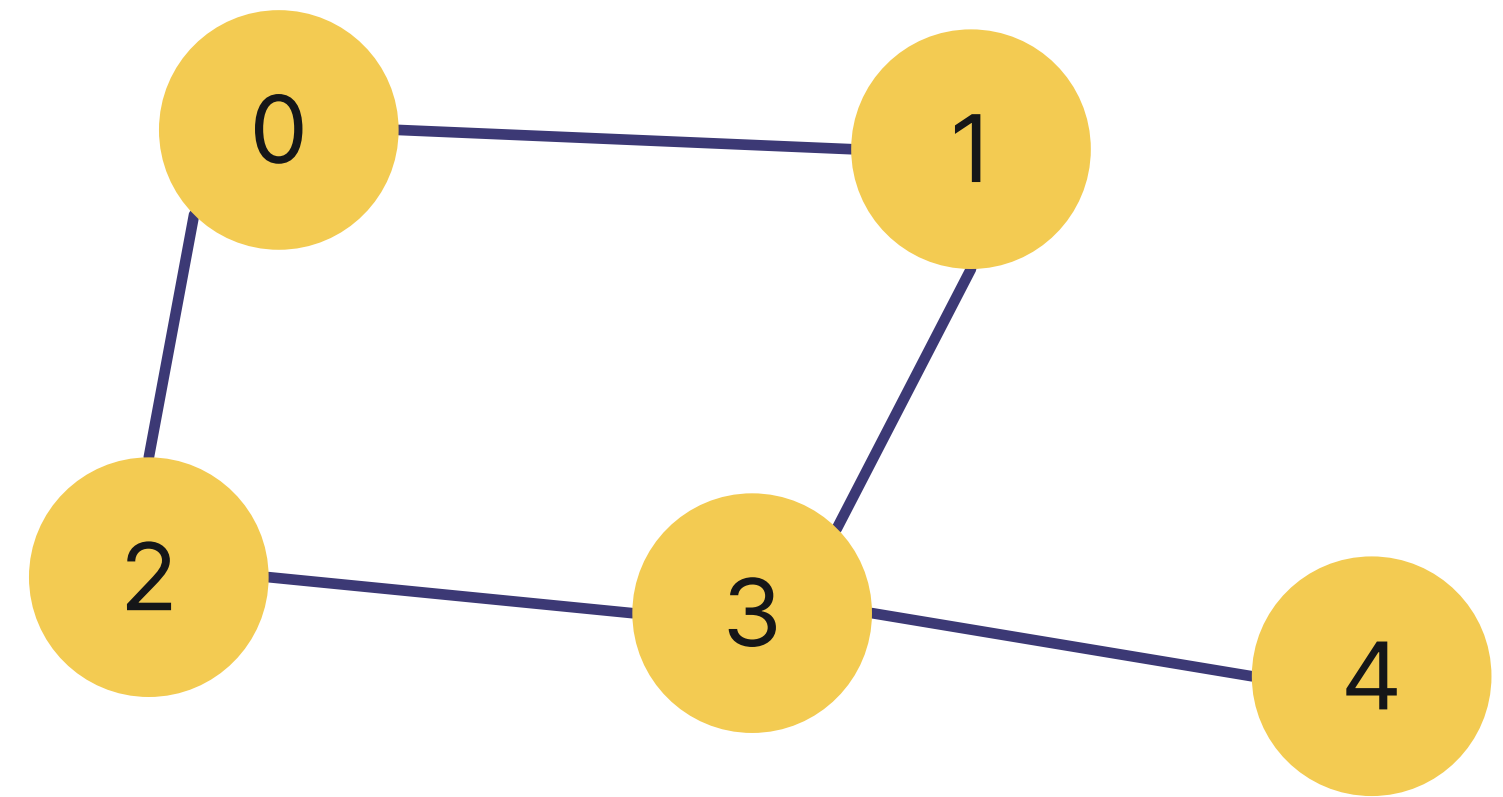
새로운 SNS 서비스 **엘리스친구**는 두 명의 사용자 간의 거리를 촌수로 표시합니다.
직접연결이 되어있는 친구라면 **1촌**, **N명의 친구를 거쳐** 연결되어있다면 **N+1촌**입니다.
두 사용자 간의 촌수를 구하세요.





[예제] 촌수 계산하기

```
def bfs(x) :  
    queue = deque([x])  
    visit = [0]*n  
    while queue :  
        cur = queue.popleft()  
        for i in range(n) :  
            if friends[cur][i] == 1 and visit[i] == 0:  
                visit[i] = visit[cur] + 1  
                queue.append(i)
```





- 그래프는 정점과 간선으로 이루어져 있습니다.
- 인접행렬, 인접리스트 등의 방법으로 그래프를 구현할 수 있습니다.
- 그래프 탐색방법으로는 BFS와 DFS가 있습니다.
 - BFS는 너비우선탐색으로 같은 루트를 공유하는 노드를 우선적으로 탐색합니다.
 - DFS는 특정 분기를 모두 탐색을 마친 후 다음 분기로 넘어가 탐색합니다.